Apostila de Estudos
Desenvolvedor Back-end Java

Do Java e POO ao Spring Boot, PostgreSQL, Testes e Docker

Roadmap: Java → Java moderno → Git/GitHub → Maven → HTTP/REST → Spring Boot → JPA/Hibernate → PostgreSQL → Testes → Docker → API profissional → Segurança → Projeto final → Empregabilidade → Apêndice (SOLID, algoritmos, concorrência)

Projeto principal: Sistema de Biblioteca (evolui a cada capítulo)
Um mini-projeto independente por capítulo
Foco: vaga de estágio / Back-end Java Júnior

Versão 1.1 · revisada tecnicamente (novembro de 2026)

Sumário

Comece aqui: como usar esta apostila 8

Visão geral . . . 8
Como cada capítulo está organizado . . . 8
O Sistema Biblioteca como fio condutor . . . 8
Ambiente de desenvolvimento . . . 9
Como estudar (a regra 70/30) . . . 9
Mapa de tempo sugerido . . . 9
Versões e como confiar neste material . . . 10
O que um júnior precisa demonstrar . . . 10

1. Java: revisão direcionada 12

Explicação geral . . . 12
1.1 Classes, objetos e encapsulamento . . . 12
1.2 Herança, polimorfismo e classes abstratas . . . 13
1.3 Interfaces . . . 14
1.4 equals(), hashCode() e toString() . . . 14
1.5 Exceções . . . 15
1.6 Generics . . . 16
1.7 Collections . . . 17
1.8 String, StringBuilder e enum . . . 18
1.9 Datas com java.time . . . 18
1.10 Autoavaliação: o seu Projeto Diagnóstico . . . 18
Resumo do capítulo . . . 19
Exercícios teóricos . . . 19
Exercícios práticos . . . 19
Projeto principal: Sistema Biblioteca . . . 20
Mini-projeto: Sistema de Contas e Extrato . . . 20

2. Java moderno 21

Explicação geral . . . 21
2.1 Lambdas . . . 21
2.2 Interfaces funcionais . . . 22
2.3 Streams . . . 22
2.4 Optional . . . 24
2.5 Records . . . 24
2.6 var, switch expressions, text blocks e pattern matching . . . 25
2.7 Mais da API moderna . . . 25
Resumo do capítulo . . . 25

Exercícios teóricos . . . 26
Exercícios práticos . . . 26
Projeto principal: Sistema Biblioteca . . . 26
Mini-projeto: Analisador de Vendas . . . 27

3. Git e GitHub 28

Explicação geral . . . 28
3.1 O modelo mental do Git . . . 28
3.2 Configuração inicial . . . 28
3.3 Comandos essenciais . . . 28
3.4 Bons commits . . . 29
3.5 .gitignore: o que não versionar . . . 29
3.6 Branches e merge . . . 30
3.7 Desfazendo coisas . . . 30
3.8 Trabalhando com GitHub . . . 30
3.9 README profissional . . . 31
3.10 Tópicos úteis que costumam salvar o dia . . . 31
3.11 Boas práticas . . . 31
Resumo do capítulo . . . 32
Exercícios teóricos . . . 32
Exercícios práticos . . . 32
Projeto principal: Sistema Biblioteca . . . 32
Mini-projeto: Repositório de Anotações de Estudo . . . 33

4. Maven 34

Explicação geral . . . 34
4.1 Convenção sobre configuração . . . 34
4.2 O pom.xml . . . 34
4.3 Repositórios . . . 35
4.4 Ciclo de vida (lifecycle) . . . 35
4.5 Plugins . . . 36
4.6 Herança de POM e BOM . . . 36
4.7 Maven Wrapper . . . 37
4.8 Problemas comuns . . . 37
Resumo do capítulo . . . 37
Exercícios teóricos . . . 37
Exercícios práticos . . . 37
Projeto principal: Sistema Biblioteca . . . 38
Mini-projeto: Biblioteca de Utilitários (artefato reutilizável) . . . 38

5. HTTP e REST 39

Explicação geral . . . 39
5.1 Como o HTTP funciona . . . 39
5.2 Métodos HTTP . . . 40
5.3 Status codes . . . 40
5.4 JSON . . . 41
5.5 O que é REST . . . 41
5.6 Boas práticas de API . . . 42
5.7 Ferramentas para testar APIs . . . 42
5.8 Documentando o contrato (OpenAPI) . . . 43
Resumo do capítulo . . . 43
Exercícios teóricos . . . 43
Exercícios práticos . . . 44
Projeto principal: Sistema Biblioteca . . . 44
Mini-projeto: Consumidor de API pública (Painel de CEP e Clima) . . . 44

6. Spring Boot 46

Explicação geral . . . 46
6.0 Antes de começar: versões e mudança do modelo . . . 46
6.1 Criando o projeto (Spring Initializr) . . . 46
6.2 IoC, DI e Beans . . . 47
6.3 Controllers . . . 48
6.4 Services . . . 49
6.5 Repositories (em memória, por enquanto) . . . 50
6.6 Configuração: application.properties e YAML . . . 50
6.7 Starters e autoconfiguração . . . 51
6.8 Ciclo de uma requisição . . . 51
6.9 Problemas comuns (e como ler o erro) . . . 51
Resumo do capítulo . . . 52
Exercícios teóricos . . . 52
Exercícios práticos . . . 52
Projeto principal: Sistema Biblioteca . . . 53
Mini-projeto: API de Tarefas (To-Do) . . . 53

7. JPA e Hibernate 54

Explicação geral . . . 54
7.1 Dependências e configuração . . . 54
7.2 Entidades e mapeamento básico . . . 55
7.3 Relacionamentos . . . 56
7.4 Lazy x Eager e o problema N+1 . . . 57
7.5 Spring Data JPA: repositories . . . 58

7.6 Transações . . . 59
7.7 Auditoria e boas práticas . . . 60
Resumo do capítulo . . . 60
Exercícios teóricos . . . 60
Exercícios práticos . . . 61
Projeto principal: Sistema Biblioteca . . . 61
Mini-projeto: API de Cursos e Alunos . . . 61

8. PostgreSQL com Spring Boot 62

Explicação geral . . . 62
8.1 Subindo o PostgreSQL . . . 62
8.2 Conectando a aplicação . . . 62
8.3 Modelando o esquema para a aplicação . . . 63
8.4 Migrações com Flyway . . . 64
8.5 SQL que todo back-end precisa dominar . . . 64
8.6 Dados iniciais e acesso seguro . . . 65
8.7 Problemas comuns . . . 66
Resumo do capítulo . . . 66
Exercícios teóricos . . . 66
Exercícios práticos . . . 67
Projeto principal: Sistema Biblioteca . . . 67
Mini-projeto: Controle Financeiro Pessoal (somente banco + JDBC/JPA) . . . 67

9. Testes: JUnit e Mockito 69

Explicação geral . . . 69
9.1 JUnit 5: o básico . . . 69
9.2 Mockito: isolando dependências . . . 71
9.3 Testes de integração com Spring . . . 72
9.4 Boas práticas . . . 74
Resumo do capítulo . . . 74
Exercícios teóricos . . . 74
Exercícios práticos . . . 75
Projeto principal: Sistema Biblioteca . . . 75
Mini-projeto: Biblioteca de Regras de Negócio (kata de testes) . . . 75

10. Docker 76

Explicação geral . . . 76
10.1 Instalação e primeiros comandos . . . 76
10.2 Dockerfile . . . 77
10.3 Volumes, redes e variáveis de ambiente . . . 78
10.4 Docker Compose . . . 79

10.5 Boas práticas . . . 80
10.6 Problemas comuns . . . 80
Resumo do capítulo . . . 80
Exercícios teóricos . . . 80
Exercícios práticos . . . 81
Projeto principal: Sistema Biblioteca . . . 81
Mini-projeto: Stack Completa com Cache (API + PostgreSQL + Redis) . . . 81

11. API profissional: DTOs, validação e erros 83

Explicação geral . . . 83
11.1 DTOs (Data Transfer Objects) . . . 83
11.2 Validação com Bean Validation . . . 84
11.3 Tratamento global de exceções . . . 85
11.4 Documentação com OpenAPI (Swagger) . . . 88
11.5 Paginação, filtros e logs . . . 88
Resumo do capítulo . . . 89
Exercícios teóricos . . . 89
Exercícios práticos . . . 90
Projeto principal: Sistema Biblioteca . . . 90
Mini-projeto: API de Reservas de Salas . . . 90

12. Segurança básica com Spring Security 91

Explicação geral . . . 91
12.1 Senhas: o que nunca fazer . . . 91
12.2 Configurando o Spring Security . . . 92
12.3 JWT: autenticação por token . . . 93
12.4 Perfis de acesso na Biblioteca . . . 95
12.5 As falhas mais comuns (OWASP) . . . 95
12.6 Testando segurança . . . 96
Resumo do capítulo . . . 96
Exercícios teóricos . . . 97
Exercícios práticos . . . 97
Projeto principal: Sistema Biblioteca . . . 97
Mini-projeto: Cofre de Notas Pessoais . . . 98

13. Projeto principal: do código ao portfólio 99

Explicação geral . . . 99
13.1 Escopo final do Sistema Biblioteca . . . 99
13.2 Arquitetura e organização . . . 101
13.3 Roteiro de implementação (semanas 11 e 12) . . . 101
13.4 Integração contínua com GitHub Actions . . . 102

13.5 Checklist de qualidade final . . . 103
13.6 Estrutura sugerida do README final . . . 104
Resumo do capítulo . . . 104
Exercícios teóricos . . . 104
Exercícios práticos . . . 105
Projeto principal: Sistema Biblioteca (versão 1.0) . . . 105
Mini-projeto: "Clone" de um Sistema do seu Interesse . . . 105

14. Empregabilidade (dezembro) 106

Explicação geral . . . 106
14.1 Currículo de uma página . . . 106
14.2 GitHub como vitrine . . . 107
14.3 LinkedIn . . . 107
14.4 Onde procurar e como se candidatar . . . 108
14.5 Preparação para entrevistas técnicas . . . 108
14.6 Comportamento: o que também decide a vaga . . . 110
14.7 Plano de dezembro (4 semanas) . . . 110
Resumo do capítulo . . . 111
Exercícios teóricos . . . 111
Exercícios práticos . . . 111
Projeto principal: Sistema Biblioteca . . . 112
Mini-projeto: Kit de Candidatura . . . 112

15. Apêndice: fundamentos que completam a base 113

Explicação geral . . . 113
15.1 SOLID: cinco princípios de design . . . 113
15.2 Estruturas de dados e algoritmos para entrevistas . . . 114
15.3 Concorrência básica . . . 117
15.4 Lombok . . . 118
15.5 Extras do Spring que aparecem no mercado . . . 118
15.6 Terminal e Linux básico . . . 119
Resumo do capítulo . . . 119
Exercícios teóricos . . . 120
Exercícios práticos . . . 120
Projeto principal: Sistema Biblioteca (refatoração opcional) . . . 120
Mini-projeto: Kit de Algoritmos com Testes . . . 121

Comece aqui: como usar esta apostila

Visão geral

Esta apostila foi montada a partir do seu **Roadmap de Back-end Java** e do seu **Projeto Diagnóstico (Sistema de Biblioteca)**. Ela parte do que você já sabe (Java e POO da graduação, SQL de Banco de Dados 1 e 2) e leva você, passo a passo, até o perfil de **Desenvolvedor Back-end Java Júnior**: alguém capaz de construir, testar, versionar e empacotar uma API REST com Spring Boot e PostgreSQL.

O objetivo não é decorar sintaxe. É conseguir, numa entrevista técnica ou no primeiro dia de estágio, explicar **o que** você fez, **por que** fez assim e **como** consertar quando quebra.

Como cada capítulo está organizado

Todo capítulo segue a mesma estrutura:

1. **Explicação geral**: o que é a tecnologia, por que ela existe e onde ela se encaixa no mercado.
2. **Subtópicos**: conceitos explicados com exemplos em código.
3. **Resumo**: a versão de uma página para revisar antes de entrevistas.
4. **Exercícios teóricos**: perguntas que você deveria conseguir responder em voz alta.
5. **Exercícios práticos**: pequenas tarefas de código para fixar cada conceito.
6. **Projeto principal: Sistema Biblioteca**: o que você acrescenta ao projeto que já começou.
7. **Mini-projeto do capítulo**: um projeto pequeno e independente, para treinar a matéria em outro contexto.

O Sistema Biblioteca como fio condutor

O projeto principal evolui junto com a apostila. A cada capítulo ele ganha uma camada nova, de modo que no final você terá a história completa de um projeto profissional no GitHub, com commits que mostram sua evolução.

Capítulo	Evolução do Sistema Biblioteca
1 e 2	Versão console em Java puro: POO, exceções, Collections, Streams, Optional, Records
3	Repositório organizado no GitHub: branches, Pull Requests, README, .gitignore
4	Migração para projeto Maven, com testes executáveis por `mvn test`
5	Desenho da API REST da biblioteca (contrato dos endpoints)
6	API Spring Boot em memória: Controller → Service → Repository
7 e 8	Persistência com JPA/Hibernate e PostgreSQL, migrações com Flyway
9	Testes unitários (JUnit + Mockito) e de integração
10	Aplicação + banco rodando com Docker Compose
11 e 12	DTOs, validação, tratamento global de erros, documentação e segurança básica
13	Versão final: vários exemplares, multa, reserva, CI e README profissional
15	Apêndice: SOLID, algoritmos para entrevistas, concorrência e extras (refatoração opcional do projeto)

> **DICA** Quando um capítulo disser "Projeto principal", o foco é sempre acrescentar algo ao Sistema Biblioteca. Quando disser "Mini-projeto", é um projeto separado, em uma pasta/repositório próprio, sobre outro assunto, para você praticar sem o peso do projeto grande.

Ambiente de desenvolvimento

Instale antes de começar:

Ferramenta	Para quê	Observação
JDK 21 (LTS)	Compilar e rodar Java	Use uma versão LTS (17 ou 21). Confira com `java -version`
IntelliJ IDEA Community	IDE principal	VS Code com extensões Java também serve
Git	Controle de versão	Configure nome e e-mail
Maven	Build e dependências	O IntelliJ já traz um; instale o seu para usar no terminal
Docker Desktop (ou Docker Engine)	Containers	Banco de dados a partir do capítulo 8; empacotamento no capítulo 10
PostgreSQL	Banco de dados	Pode ser via Docker (recomendado)
Postman, Insomnia ou Bruno	Testar APIs	Escolha um; o `curl` também serve
DBeaver ou pgAdmin	Cliente de banco	Para inspecionar tabelas

Como estudar (a regra 70/30)

Seu roadmap define: **70% prática e 30% teoria**. Na prática, isso significa:

- Leia um subtópico e **digite o código você mesmo** (não copie e cole). Digitar força o cérebro a processar.
- Depois de entender, **quebre de propósito**: remova um `@Autowired`, troque um tipo, apague um `import`. Ler a mensagem de erro é uma das habilidades mais valiosas de um júnior.
- Faça commits pequenos e frequentes, com mensagens claras. O GitHub é seu portfólio.
- Ao terminar um capítulo, tente **explicar em voz alta** o resumo, sem olhar. O que travou é o que você ainda não domina.
- Não avance acumulando cursos. Se o exercício prático do capítulo não saiu, volte antes de seguir.

> **ATENÇÃO** Evite o "ciclo do tutorial": assistir a vídeos o dia todo sem escrever código próprio. A sensação de progresso é falsa. Reserve pelo menos metade de cada sessão de estudo para o teclado.

Mapa de tempo sugerido

Seu roadmap prevê 12 semanas de estudo e dezembro para empregabilidade. Esta é uma divisão realista dos capítulos:

Período	Capítulos	Entrega esperada
Semana 1	1	Diagnóstico concluído e revisado
Semana 2	2	Biblioteca refatorada com Streams, Optional e Records
Semana 3	3 e 4	Projeto no GitHub, em Maven

Período	Capítulos	Entrega esperada
Semana 4	5	Contrato REST da biblioteca desenhado e testado no Postman
Semanas 5 e 6	6	API Spring Boot funcionando em memória
Semanas 7 e 8	7 e 8	Dados persistidos no PostgreSQL
Semana 9	9	Cobertura de testes nos Services
Semana 10	10 e 11	`docker compose up` sobe tudo; DTOs, validação e erros padronizados
Semana 11	12	Segurança básica (BCrypt, JWT, perfis)
Semana 12	13	Projeto de portfólio finalizado
Dezembro	14 e 15	Currículo, GitHub, LinkedIn e candidaturas; apêndice em paralelo

> **ATENÇÃO** Os capítulos 11, 12, 13 e o apêndice (15) **vão além do roadmap original**. Se o tempo apertar, priorize nesta ordem: 11 (DTOs e erros), 13 (projeto final) e só então 12 (segurança). É aceitável fazer apenas 12.1 e 12.2 (BCrypt e perfis) e deixar o JWT completo para depois. Se precisar, estenda o cronograma em 2 a 3 semanas: é melhor que entregar um projeto final apressado.

Se você atrasar numa semana, tudo bem. É melhor dominar a base do que cumprir o calendário com buracos.

Versões e como confiar neste material

Os exemplos assumem **Java 17 ou 21** e **Spring Boot 3.x ou 4.x**. O Spring Boot 3.5 já saiu do suporte gratuito e o Initializr oferece a série 4.x; por isso, números de versão de bibliotecas (springdoc, jjwt, Testcontainers, JaCoCo, plugins do Maven) são **exemplos** que você deve conferir no Maven Central. Onde o comportamento muda entre versões, o texto avisa.

Esta é a **versão 1.1** da apostila, revisada tecnicamente. O que foi verificado e como:

Item	Como foi verificado
Esquema SQL do capítulo 8, relatórios e índice parcial	Executados em um PostgreSQL 16
Migração de transição do capítulo 13	Executada sobre dados de exemplo, sem perda de dados
Exceções do domínio, teste de concorrência (cap. 13), algoritmos do apêndice	Compilados e executados com JDK 21
Trechos que usam Spring, JPA, Security, Mockito e Testcontainers	**Revisados por leitura**, sem compilar. Teste cada um no seu projeto e consulte a documentação da sua versão se algo divergir

Se você encontrar um erro, trate-o como parte do aprendizado: leia a mensagem, consulte a documentação oficial e corrija. Esse é o dia a dia de um desenvolvedor.

O que um júnior precisa demonstrar

Antes de ler, tenha em mente o "perfil de saída" que as vagas costumam pedir:

- Java sólido, POO, Collections, tratamento de exceções, noções de Streams e Lambdas.
- Spring Boot: construir uma API REST em camadas.
- Banco relacional (SQL) e JPA/Hibernate.
- Git e GitHub no dia a dia.

- Testes unitários com JUnit e Mockito.
- Noções de Docker.
- Comunicação clara: explicar o que fez e pedir ajuda direito.

Bons estudos. Vamos começar pelo seu alicerce: o Java.

1. Java: revisão direcionada

Explicação geral

Java é uma linguagem **compilada para bytecode** que roda na **JVM** (Java Virtual Machine). Isso é o que permite o famoso "escreva uma vez, rode em qualquer lugar". É fortemente tipada, orientada a objetos e é uma das linguagens mais usadas em back-end corporativo (bancos, e-commerce, ERPs, logística). Quase toda vaga júnior de back-end em Java cobra o conteúdo deste capítulo.

Como você já viu Java e POO na graduação, aqui o foco é **revisar com olhar profissional**: o que mais aparece em código de verdade e em entrevistas, e o que o seu Projeto Diagnóstico da Biblioteca exige.

Conceito	Para que serve	Aparece no Sistema Biblioteca como
Classe e objeto	Modelar entidades do mundo real	`Livro`, `Usuario`, `Emprestimo`
Encapsulamento	Proteger o estado interno	atributos `private` + getters
Herança e polimorfismo	Reutilizar e generalizar comportamento	`Pessoa` → `Aluno`, `Professor`
Interface	Definir contratos	`Repositorio<T>`
Exceções	Tratar situações de erro	`LivroIndisponivelException`
Collections	Guardar grupos de objetos	`List<Livro>`, `Map<Integer, Usuario>`

1.1 Classes, objetos e encapsulamento

Uma **classe** é o molde; um **objeto** é uma instância dele. **Encapsulamento** significa esconder os atributos (`private`) e expor apenas operações que façam sentido, garantindo que o objeto nunca fique em estado inválido.

```
public class Livro {
    private final int id;
    private String titulo;
    private String autor;
    private int ano;
    private boolean disponivel;

    public Livro(int id, String titulo, String autor, int ano) {
        if (titulo == null || titulo.isBlank()) {
            throw new IllegalArgumentException("Título é obrigatório");
        }
        this.id = id;
        this.titulo = titulo;
        this.autor = autor;
        this.ano = ano;
        this.disponivel = true;           // todo livro nasce disponível
    }

    public void emprestar() {
        if (!disponivel) {
            throw new LivroIndisponivelException(titulo);
        }
        disponivel = false;
    }

    public void devolver() { disponivel = true; }

    public int getId() { return id; }
    public String getTitulo() { return titulo; }
    public String getAutor() { return autor; }
    public int getAno() { return ano; }
```

```
    public boolean isDisponivel() { return disponivel; }
}
```

Repare em duas decisões de design: não existe `setDisponivel(boolean)` (a mudança de estado acontece por `emprestar()` e `devolver()`, que contêm a regra de negócio) e o `id` é `final` (não muda depois de criado).

> **DICA** Regra de ouro: **métodos que expressam intenção** (`emprestar()`) são melhores que setters genéricos (`setDisponivel(false)`). Quem lê o código entende o negócio.

Construtores, `this` e sobrecarga

O construtor garante que o objeto nasça válido. Você pode ter vários (sobrecarga) e encadeá-los com `this(...)`.

```
public Livro(String titulo, String autor) {
    this(0, titulo, autor, 0);   // reaproveita o construtor principal
}
```

`static` e `final`

- `static`: pertence à classe, não ao objeto (contadores, métodos utilitários, constantes).
- `final`: variável não pode ser reatribuída; método não pode ser sobrescrito; classe não pode ser herdada.

```
public class Biblioteca {
    private static int proximoIdLivro = 1;
    public static final int PRAZO_DIAS = 14;

    public int gerarIdLivro() { return proximoIdLivro++; }
}
```

1.2 Herança, polimorfismo e classes abstratas

Herança (`extends`) permite que uma classe reutilize atributos e métodos de outra. **Polimorfismo** permite tratar objetos de classes diferentes por meio de um tipo comum, com cada um respondendo ao mesmo método à sua maneira.

```
public abstract class Pessoa {
    protected final String nome;
    protected final String email;

    protected Pessoa(String nome, String email) {
        this.nome = nome;
        this.email = email;
    }

    public abstract int limiteEmprestimos();      // cada filho define
    public String getNome() { return nome; }
}

public class Aluno extends Pessoa {
    public Aluno(String nome, String email) { super(nome, email); }
    @Override public int limiteEmprestimos() { return 3; }
}

public class Professor extends Pessoa {
    public Professor(String nome, String email) { super(nome, email); }
    @Override public int limiteEmprestimos() { return 6; }
}

List<Pessoa> pessoas = List.of(new Aluno("Ana", "a@x.com"), new Professor("Bia", "b@x.com"));
for (Pessoa p : pessoas) {
    System.out.println(p.getNome() + " pode pegar " + p.limiteEmprestimos());  // polimorfismo
```

```
}
```

Uma **classe abstrata** não pode ser instanciada e pode misturar métodos prontos com métodos abstratos. Use quando existe um "esqueleto" comum com partes a completar.

> **DICA** Prefira **composição** a herança quando a relação for "tem um" (um `Emprestimo` **tem um** `Livro`) e não "é um". Herança em excesso gera hierarquias rígidas e difíceis de mudar.

1.3 Interfaces

Uma **interface** é um contrato: define *o que* deve ser feito, sem dizer *como*. Uma classe pode implementar várias interfaces (Java não tem herança múltipla de classes, mas tem de contratos).

```
public interface Repositorio<T, ID> {
    void salvar(T entidade);
    Optional<T> buscarPorId(ID id);
    List<T> listarTodos();
}

public class LivroRepositorioMemoria implements Repositorio<Livro, Integer> {
    private final Map<Integer, Livro> dados = new HashMap<>();

    @Override public void salvar(Livro l) { dados.put(l.getId(), l); }
    @Override public Optional<Livro> buscarPorId(Integer id) {
        return Optional.ofNullable(dados.get(id));
    }
    @Override public List<Livro> listarTodos() { return new ArrayList<>(dados.values()); }
}
```

Esse padrão será exatamente o que o Spring Data JPA faz por você mais adiante. Entender interfaces agora economiza muita confusão depois.

Desde o Java 8, interfaces também podem ter métodos `default` (com implementação) e `static`. Desde o Java 9, `private`.

Característica	Classe abstrata	Interface
Estado (atributos de instância)	Sim	Não (apenas constantes)
Construtor	Sim	Não
Herança múltipla	Não	Sim (implementa várias)
Quando usar	Base comum com código compartilhado	Contrato/capacidade

1.4 `equals()`, `hashCode()` e `toString()`

Por padrão, `equals` compara **referências** (se é o mesmo objeto na memória). Para entidades, normalmente queremos comparar por **identidade de negócio** (o `id`). Se você sobrescrever `equals`, **deve** sobrescrever `hashCode` também, senão `HashMap` e `HashSet` se comportam de forma errada.

```
@Override
public boolean equals(Object o) {
    if (this == o) return true;
    if (!(o instanceof Livro outro)) return false;   // pattern matching (Java 16+)
    return id == outro.id;
}

@Override
public int hashCode() { return Integer.hashCode(id); }

@Override
public String toString() {
    return "Livro{id=%d, titulo='%s', autor='%s', disponivel=%s}"
```

```
            .formatted(id, titulo, autor, disponivel);
}
```

> **ATENÇÃO** Contrato: se `a.equals(b)` é verdadeiro, então `a.hashCode() == b.hashCode()`. O inverso não precisa valer. Violar isso é um dos erros clássicos perguntados em entrevistas.

Um `toString()` útil é essencial para depurar e para logs: mostre os campos que ajudam a identificar o objeto, mas nunca dados sensíveis (senhas).

1.5 Exceções

Exceção é um evento que interrompe o fluxo normal do programa. Em Java há dois grandes grupos:

Tipo	Herda de	Exemplos	Compilador obriga a tratar?
Checked	`Exception`	`IOException`, `SQLException`	Sim (`try/catch` ou `throws`)
Unchecked	`RuntimeException`	`NullPointerException`, `IllegalArgumentException`	Não
Errors	`Error`	`OutOfMemoryError`, `StackOverflowError`	Não (não trate)

try, catch, finally, throw e throws

```
public Livro buscarLivro(int id) {          // exceção unchecked: não precisa de throws
    return livros.stream()
            .filter(l -> l.getId() == id)
            .findFirst()
            .orElseThrow(() -> new LivroNaoEncontradoException(id));      // lança
}

try {
    Livro l = biblioteca.buscarLivro(10);
    l.emprestar();
} catch (LivroNaoEncontradoException e) {
    System.out.println("Não existe: " + e.getMessage());
} catch (LivroIndisponivelException e) {
    System.out.println("Já emprestado: " + e.getMessage());
} finally {
    System.out.println("Operação finalizada");       // sempre executa
}
```

Criando exceções próprias

```
// Exceções de domínio da Biblioteca: UNCHECKED (é o padrão que o Spring usa nos caps. 6 a 12)
public class LivroNaoEncontradoException extends RuntimeException {
    public LivroNaoEncontradoException(long id) {
        super("Livro não encontrado: id=" + id);
    }
}

public class LivroIndisponivelException extends RuntimeException {
    public LivroIndisponivelException(String titulo) {
        super("Livro indisponível: " + titulo);
    }
}

// Exemplo de CHECKED (usado no mini-projeto bancário): o chamador é obrigado a tratar
public class SaldoInsuficienteException extends Exception {
    public SaldoInsuficienteException(double saldo, double valor) {
        super("Saldo %.2f insuficiente para sacar %.2f".formatted(saldo, valor));
    }
}
```

ATENÇÃO Ao longo desta apostila, as exceções do **Sistema Biblioteca** são sempre *unchecked* (`RuntimeException`). Se `LivroNaoEncontradoException` fosse *checked*, o Service do capítulo 6 não compilaria sem `throws`, e o rollback de transações do capítulo 7 não funcionaria por padrão.

Checked ou unchecked?

- **Checked**: quando o chamador consegue e **deve** reagir (ex.: arquivo não existe, tentar outro caminho).
- **Unchecked**: erros de regra de negócio e de programação. É o que o **Spring** prefere: exceções de domínio como `RuntimeException`, tratadas em um único lugar (veremos o `@RestControllerAdvice` no capítulo 11).

O `finally` e o `return`

O bloco `finally` executa **mesmo que o `try` tenha um `return`**: o valor de retorno é calculado, o `finally` roda e só então o método devolve. Só não roda se a JVM for encerrada (`System.exit`) ou o processo morrer.

```
static int teste() {
    try { return 1; }
    finally { System.out.println("finally roda antes de devolver"); }
}
```

ATENÇÃO Nunca coloque `return` dentro do `finally`: ele sobrescreve o retorno do `try` e **engole exceções**.

try-with-resources

Para recursos que precisam ser fechados (arquivos, conexões), implemente `AutoCloseable` e use:

```
try (BufferedReader br = Files.newBufferedReader(Path.of("livros.csv"))) {
    String linha;
    while ((linha = br.readLine()) != null) { /* ... */ }
} catch (IOException e) {
    System.err.println("Falha ao ler: " + e.getMessage());
}   // br.close() é chamado automaticamente
```

ATENÇÃO Anti-padrões: `catch (Exception e) {}` vazio (engole o erro), colocar `try/catch` em tudo "só por garantia", e usar exceção para controle de fluxo normal. Trate o erro **onde dá para fazer algo útil**; caso contrário, deixe subir.

1.6 Generics

Generics permitem escrever código reutilizável e **seguro em tempo de compilação**. Sem eles, você precisaria de `Object` e *casts* perigosos.

```
public class Caixa<T> {
    private T conteudo;
    public void guardar(T c) { conteudo = c; }
    public T pegar() { return conteudo; }
}

Caixa<Livro> caixa = new Caixa<>();
caixa.guardar(new Livro(1, "Java Efetivo", "Bloch", 2018));

// método genérico
public static <T> T primeiro(List<T> lista) { return lista.get(0); }
```

Curinga (`?`): `List<? extends Pessoa>` aceita lista de qualquer subtipo de `Pessoa` (para leitura). Você verá isso pouco no dia a dia; basta reconhecê-lo.

1.7 Collections

O **Java Collections Framework** oferece estruturas prontas. Escolher a certa é decisão de projeto.

Interface	Implementações comuns	Característica	Use quando
`List`	`ArrayList`, `LinkedList`	Ordenada, aceita repetidos, acesso por índice	Lista de livros em ordem de cadastro
`Set`	`HashSet`, `LinkedHashSet`, `TreeSet`	Sem repetidos	Garantir unicidade (e-mails)
`Map`	`HashMap`, `LinkedHashMap`, `TreeMap`	Chave → valor	Busca rápida por id
`Queue`/`Deque`	`ArrayDeque`, `PriorityQueue`	Fila/pilha	Fila de reservas

```
List<Livro> livros = new ArrayList<>();
Set<String> emails = new HashSet<>();
Map<Integer, Usuario> usuariosPorId = new HashMap<>();

usuariosPorId.put(1, new Usuario(1, "Ana", "ana@x.com"));
Usuario u = usuariosPorId.get(1);                       // O(1) em média
boolean novo = emails.add("ana@x.com");                 // false se já existia

for (Map.Entry<Integer, Usuario> e : usuariosPorId.entrySet()) {
    System.out.println(e.getKey() + " -> " + e.getValue());
}
```

Como escolher a implementação

Comparação	Diferença que importa
`ArrayList` vs `LinkedList`	`ArrayList` guarda os elementos num vetor: acesso por índice é rápido (O(1)) e inserir no meio é mais lento. `LinkedList` encadeia nós: inserir/remover nas pontas é barato, mas acessar por índice é O(n). **Na prática, use** `ArrayList` quase sempre
`HashMap` vs `TreeMap`	`HashMap` busca em O(1) médio, sem ordem garantida. `TreeMap` mantém as chaves **ordenadas** (O(log n)). `LinkedHashMap` mantém a ordem de inserção
`HashSet` vs `TreeSet`	Mesma ideia: `HashSet` é mais rápido; `TreeSet` mantém ordenado

Como o `HashMap` funciona por dentro: calcula o `hashCode()` da chave, usa-o para escolher um "balde" (*bucket*) e, dentro do balde, compara com `equals()`. Se duas chaves caem no mesmo balde (**colisão**), ficam encadeadas. Por isso `equals` e `hashCode` precisam ser coerentes.

Boas práticas com Collections

- Declare pelo **tipo da interface**: `List<Livro> l = new ArrayList<>();`.
- Use `List.of(...)`, `Set.of(...)`, `Map.of(...)` para coleções **imutáveis** pequenas.
- Não exponha a lista interna da classe. Devolva uma cópia ou uma visão imutável:

```
public List<Livro> getLivros() {
    return Collections.unmodifiableList(livros);   // protege o encapsulamento
}
```

- `HashMap`/`HashSet` dependem de `equals` e `hashCode` corretos nas chaves.
- Para ordenar: `Comparable` (ordem natural) ou `Comparator` (ordem sob medida).

```
livros.sort(Comparator.comparing(Livro::getTitulo).thenComparing(Livro::getAno));
```

1.8 String, StringBuilder e `enum`

Strings são imutáveis: cada concatenação cria um novo objeto. Em laços longos, use `StringBuilder`.

```
StringBuilder sb = new StringBuilder();
for (Livro l : livros) {
    sb.append(l.getTitulo()).append("; ");
}
String resultado = sb.toString();

String a = "java", b = "java";
a.equals(b);           // true  (compare conteúdo com equals, nunca com ==)
"  Olá ".trim();       // "Olá"
"a,b,c".split(",");    // ["a","b","c"]
String.join(" | ", "x", "y");
```

Enum representa um conjunto fixo de valores e pode ter atributos e métodos.

```
public enum StatusEmprestimo {
    ATIVO("Em andamento"), DEVOLVIDO("Concluído"), ATRASADO("Atrasado");

    private final String descricao;
    StatusEmprestimo(String descricao) { this.descricao = descricao; }
    public String getDescricao() { return descricao; }
}

StatusEmprestimo s = StatusEmprestimo.ATIVO;
switch (s) {
    case ATIVO -> System.out.println("Ainda com o usuário");
    case DEVOLVIDO -> System.out.println("Já devolvido");
    case ATRASADO -> System.out.println("Cobrar multa");
}
```

Por que `enum` e não constantes `int`? Com `int STATUS_ATIVO = 1` o compilador aceita qualquer número (`status = 99`), os valores não se descrevem sozinhos no log e o `switch` não avisa se faltar um caso. O `enum` é um **tipo**: só aceita os valores declarados, tem nome legível, pode ter atributos e métodos, e o `switch` com setas exige cobrir todos os casos.

1.9 Datas com `java.time`

Esqueça `Date` e `Calendar`. Use a API moderna (imutável e clara):

```
LocalDate hoje = LocalDate.now();
LocalDate prazo = hoje.plusDays(14);
long diasAtraso = ChronoUnit.DAYS.between(prazo, LocalDate.now());
boolean atrasado = LocalDate.now().isAfter(prazo);
LocalDateTime agora = LocalDateTime.now();
```

1.10 Autoavaliação: o seu Projeto Diagnóstico

O seu PDF de diagnóstico define o que será avaliado. Use esta checklist para revisar o projeto que você já fez, **sem ajuda**, e marque honestamente:

- ☐ Atributos `private`, getters apenas quando necessário, sem setters que quebrem regras.
- ☐ Construtores validando dados e deixando o objeto em estado consistente.
- ☐ `equals`, `hashCode` e `toString` nas entidades (baseados no `id`).
- ☐ Ao emprestar, o livro fica indisponível; ao devolver, volta a ficar disponível.

- ☐ Usuários e livros com IDs únicos e gerados automaticamente.
- ☐ Pelo menos duas exceções próprias, lançadas **onde** o erro acontece e tratadas **onde** dá para reagir.
- ☐ `Main` só cuida de menu/entrada; regras ficam em `Biblioteca`/`Service`.
- ☐ Collections escolhidas com consciência (por que `Map` aqui e `List` ali?).
- ☐ Pacotes separados: `model`, `service`, `exception`, `ui`.
- ☐ Commits pequenos e README explicando como rodar.

O que você não marcou é seu plano de estudo da semana.

Resumo do capítulo

- Java compila para bytecode e roda na JVM; é tipado e orientado a objetos.
- **Encapsulamento**: atributos `private` e métodos que expressam regras (`emprestar()`), não setters genéricos.
- **Herança** modela "é um"; **composição** modela "tem um" (prefira composição quando possível).
- **Interface** é contrato; **classe abstrata** é base com código compartilhado.
- `equals` e `hashCode` andam juntos; `toString` ajuda a depurar.
- **Checked** exceptions obrigam o tratamento; **unchecked** (`RuntimeException`) são preferidas para erros de negócio em frameworks.
- Não engula exceções; trate onde há ação útil.
- Generics dão segurança de tipos; Collections: `List` (ordem), `Set` (unicidade), `Map` (chave-valor).
- Strings são imutáveis; use `StringBuilder` em laços; `enum` para conjuntos fixos; `java.time` para datas.

Exercícios teóricos

1. Qual a diferença entre classe e objeto? Dê um exemplo do Sistema Biblioteca.
2. Por que preferir `livro.emprestar()` a `livro.setDisponivel(false)`?
3. Explique encapsulamento, herança e polimorfismo com suas palavras.
4. Quando usar classe abstrata e quando usar interface?
5. O que acontece se você sobrescrever `equals` mas não `hashCode` e colocar objetos em um `HashSet`?
6. Qual a diferença entre `throw` e `throws`?
7. Cite uma situação para exceção checked e outra para unchecked.
8. Por que `catch (Exception e) { }` vazio é perigoso?
9. Qual a diferença entre `ArrayList` e `LinkedList`? E entre `HashMap` e `TreeMap`?
10. Por que `==` não deve ser usado para comparar `String`?
11. O que é um `enum` e por que é melhor que constantes `int`?
12. O que o `finally` garante? Ele roda mesmo com `return` dentro do `try`?

Exercícios práticos

1. Crie a classe `ContaBancaria` com saldo privado, `depositar`, `sacar` e uma exceção `SaldoInsuficienteException` (checked). Trate-a no `main`.
2. Crie `Forma` (abstrata) com `area()` e as classes `Retangulo`, `Circulo` e `Triangulo`. Coloque todas em uma `List<Forma>` e some as áreas.
3. Crie a interface `Notificador` com `enviar(String msg)` e duas implementações (`EmailNotificador` e `SmsNotificador`, imprimindo no console). Troque de implementação sem mudar quem usa.

4. Crie `Produto` com `equals/hashCode` por código. Mostre que um `HashSet<Produto>` não aceita código repetido.
5. Leia um arquivo `.csv` de livros com try-with-resources, gerando uma `List<Livro>`. Trate arquivo inexistente.
6. Dado um `Map<String, Integer>` de notas, imprima ordenado por nota decrescente (use `Comparator`).
7. Enum `DiaSemana` com método `ehFimDeSemana()`.
8. Calcule a multa de atraso: R$ 1,50 por dia, usando `LocalDate` e `ChronoUnit`.

Projeto principal: Sistema Biblioteca

Você já fez a versão console. Agora faça a **revisão técnica** dela:

1. Passe a checklist da seção 1.10 e corrija o que faltou.
2. Adicione o `enum StatusEmprestimo` (ATIVO, DEVOLVIDO, ATRASADO) e use-o em `Emprestimo`.
3. Crie `Emprestimo.devolver()` que registra a data real e muda o status; calcule multa com `java.time`.
4. Adicione a regra: **limite de 3 empréstimos ativos por usuário**, com `LimiteEmprestimosException`.
5. Garanta que `getLivros()` e `getUsuarios()` da `Biblioteca` não exponham as listas internas.
6. Atualize o README (como compilar, como usar o menu, regras de negócio).

Mini-projeto: Sistema de Contas e Extrato

Crie, em outro repositório, um mini banco em console:

- Classes `Conta` (abstrata), `ContaCorrente` (com limite de cheque especial) e `ContaPoupanca` (com rendimento).
- Interface `Tributavel` implementada pelas contas que pagam taxa.
- Operações: depositar, sacar, transferir. Cada operação gera um registro em um extrato (`List<Transacao>`).
- Exceções próprias: `SaldoInsuficienteException` (checked) e `ContaNaoEncontradaException` (unchecked).
- Menu no console e `Map<String, Conta>` para localizar por número.
- Desafio: ordenar o extrato por data e imprimir com `StringBuilder`.

2. Java moderno

Explicação geral

A partir do Java 8 (2014), a linguagem ganhou recursos que mudaram a forma de escrever código: **lambdas**, **Streams**, **Optional**. Versões seguintes trouxeram **records**, `var`, **switch expressions**, **text blocks**, **pattern matching** e **sealed classes**. Código de empresas, projetos Spring e entrevistas usam esses recursos o tempo todo. Dominá-los é o que diferencia "sei Java da faculdade" de "escrevo Java atual".

A ideia central é o estilo **declarativo**: em vez de dizer *como* percorrer e filtrar uma lista com `for` e `if`, você diz *o quê* deseja.

Recurso	Desde	Resolve
Lambdas e interfaces funcionais	Java 8	Passar comportamento como argumento
Streams	Java 8	Filtrar, transformar e agregar coleções
Optional	Java 8	Evitar `NullPointerException`
`var`	Java 10	Inferência de tipo local
Switch expressions e text blocks	Java 14 / 15	Código mais enxuto
Records	Java 16	Classes de dados imutáveis sem boilerplate
Pattern matching `instanceof`	Java 16	Elimina cast manual
Sealed classes	Java 17	Hierarquias fechadas

> **DICA** Use sempre uma versão **LTS** (Long-Term Support): 17 ou 21. Spring Boot 3 exige no mínimo Java 17.

2.1 Lambdas

Uma **lambda** é uma função anônima: um bloco de código que pode ser passado como valor. Sintaxe: `(parâmetros) -> expressão`.

```
// Antes (classe anônima)
Comparator<Livro> c1 = new Comparator<Livro>() {
    @Override public int compare(Livro a, Livro b) {
        return a.getTitulo().compareTo(b.getTitulo());
    }
};

// Agora (lambda)
Comparator<Livro> c2 = (a, b) -> a.getTitulo().compareTo(b.getTitulo());

// Ainda mais curto (method reference)
Comparator<Livro> c3 = Comparator.comparing(Livro::getTitulo);

livros.forEach(l -> System.out.println(l.getTitulo()));
livros.removeIf(l -> !l.isDisponivel());
```

Method reference (`Classe::metodo`) é um atalho quando a lambda só chama um método: `String::toUpperCase`, `System.out::println`, `Livro::new`.

2.2 Interfaces funcionais

Uma **interface funcional** tem exatamente **um método abstrato**; é onde uma lambda "se encaixa". O pacote `java.util.function` traz as principais:

Interface	Método	Recebe → devolve	Exemplo
`Predicate<T>`	`test`	T → boolean	`l -> l.isDisponivel()`
`Function<T,R>`	`apply`	T → R	`Livro::getTitulo`
`Consumer<T>`	`accept`	T → nada	`System.out::println`
`Supplier<T>`	`get`	nada → T	`() -> new ArrayList<>()`
`UnaryOperator<T>`	`apply`	T → T	`String::trim`
`BiFunction<T,U,R>`	`apply`	(T,U) → R	`(a, b) -> a + b`

```
Predicate<Livro> disponivel = Livro::isDisponivel;
Predicate<Livro> recente = l -> l.getAno() >= 2020;
Predicate<Livro> filtro = disponivel.and(recente);      // composição

Function<Livro, String> titulo = Livro::getTitulo;
Function<String, Integer> tamanho = String::length;
Function<Livro, Integer> tamanhoTitulo = titulo.andThen(tamanho);
```

Você pode criar a sua com `@FunctionalInterface`:

```
@FunctionalInterface
public interface Regra<T> {
    boolean validar(T valor);
}
```

> **DICA** Dentro de uma lambda você só pode usar variáveis locais que sejam **efetivamente finais** (não reatribuídas). Se precisar acumular, use o retorno do Stream, não uma variável externa.

2.3 Streams

Um **Stream** é uma sequência de elementos sobre a qual você aplica um *pipeline* de operações. Ele **não guarda dados** nem modifica a coleção original.

Estrutura: **fonte → operações intermediárias → operação terminal**.

```
List<String> titulos = livros.stream()           // fonte
        .filter(Livro::isDisponivel)             // intermediária
        .sorted(Comparator.comparing(Livro::getAno).reversed())
        .map(Livro::getTitulo)                   // intermediária
        .toList();                               // terminal
```

Operações intermediárias são **preguiçosas**: só executam quando a terminal é chamada.

> **DICA** `stream.toList()` (Java 16+) devolve uma lista **imutável**; `Collectors.toList()` devolve uma lista comum. Tentar `add`/`remove` na primeira lança `UnsupportedOperationException`.

Principais operações

Operação	Tipo	O que faz
`filter(Predicate)`	intermediária	Mantém quem atende à condição

Operação	Tipo	O que faz
`map(Function)`	intermediária	Transforma cada elemento
`flatMap`	intermediária	"Achata" listas de listas
`sorted`, `distinct`, `limit`, `skip`	intermediária	Ordena, remove repetidos, pagina
`forEach`	terminal	Executa ação em cada item
`collect`, `toList()`	terminal	Constrói o resultado
`count`, `min`, `max`	terminal	Agregações (`min`/`max` recebem um `Comparator`)
`sum`, `average`	terminal	Só em streams **primitivos** (`IntStream`, `DoubleStream`...): use `mapToInt`/`mapToDouble` antes
`anyMatch`, `allMatch`, `noneMatch`	terminal	Testes booleanos
`findFirst`, `findAny`	terminal	Devolve `Optional`
`reduce`	terminal	Combina tudo em um valor

Exemplos com a Biblioteca

```
// Buscar por trecho do título (ignorando maiúsculas)
List<Livro> achados = livros.stream()
        .filter(l -> l.getTitulo().toLowerCase().contains(termo.toLowerCase()))
        .toList();

// Quantos livros disponíveis?
long qtd = livros.stream().filter(Livro::isDisponivel).count();

// Algum livro de 2023?
boolean existe = livros.stream().anyMatch(l -> l.getAno() == 2023);

// Média de ano de publicação
double media = livros.stream().mapToInt(Livro::getAno).average().orElse(0);
```

Collectors: agrupar e resumir

```
// Livros agrupados por autor
Map<String, List<Livro>> porAutor = livros.stream()
        .collect(Collectors.groupingBy(Livro::getAutor));

// Contagem por autor
Map<String, Long> contagem = livros.stream()
        .collect(Collectors.groupingBy(Livro::getAutor, Collectors.counting()));

// Particionar em duas listas (true/false)
Map<Boolean, List<Livro>> particao = livros.stream()
        .collect(Collectors.partitioningBy(Livro::isDisponivel));

// Mapa id -> livro (lança IllegalStateException se houver chave repetida!)
Map<Integer, Livro> porId = livros.stream()
        .collect(Collectors.toMap(Livro::getId, Function.identity()));

// Com chave possivelmente repetida, informe como resolver o conflito
Map<String, Livro> porTitulo = livros.stream()
        .collect(Collectors.toMap(Livro::getTitulo, Function.identity(), (a, b) -> a));

// Texto único: "Java; Spring; Docker"
String lista = livros.stream().map(Livro::getTitulo).collect(Collectors.joining("; "));
```

> **ATENÇÃO** Evite Streams quando um `for` simples é mais legível, e **nunca** altere estado externo dentro de `map`/`filter`. Também não reutilize um Stream: após a operação terminal ele é consumido (`IllegalStateException`).

Streams paralelos

`list.parallelStream()` divide o trabalho entre vários núcleos. Só vale a pena para grandes volumes e operações caras; em geral, **não use** em código de aplicação web.

2.4 Optional

`Optional<T>` é um "contêiner" que pode ou não ter um valor. Ele torna explícito, **na assinatura do método**, que o resultado pode estar ausente, em vez de devolver `null`.

```
public Optional<Livro> buscarPorId(int id) {
    return livros.stream().filter(l -> l.getId() == id).findFirst();
}

// Consumindo:
Livro l = biblioteca.buscarPorId(5)
        .orElseThrow(() -> new LivroNaoEncontradoException(5));

String titulo = biblioteca.buscarPorId(5)
        .map(Livro::getTitulo)
        .orElse("Desconhecido");

biblioteca.buscarPorId(5).ifPresent(x -> System.out.println(x));
biblioteca.buscarPorId(5).ifPresentOrElse(
        x -> System.out.println("Achei " + x),
        () -> System.out.println("Não achei"));
```

Método	Quando usar
`Optional.of(x)`	Valor com certeza não nulo
`Optional.ofNullable(x)`	Valor que pode ser nulo
`Optional.empty()`	Sem valor
`orElse(v)`	Valor padrão simples
`orElseGet(supplier)`	Valor padrão custoso (calculado só se vazio)
`orElseThrow(...)`	Ausência é erro
`map`, `filter`, `flatMap`	Encadear transformações

> **ATENÇÃO** Boas práticas: use `Optional` **apenas como retorno de método**. Não use em atributos de classe, parâmetros ou coleções. Evite `opt.get()` sem verificar; prefira `orElseThrow`.

2.5 Records

Um **record** é uma classe de dados **imutável** em uma linha. O compilador gera construtor, getters (sem o prefixo `get`), `equals`, `hashCode` e `toString`.

```
public record LivroDTO(int id, String titulo, String autor, int ano) { }

LivroDTO dto = new LivroDTO(1, "Clean Code", "Martin", 2008);
System.out.println(dto.titulo());       // acessor: titulo(), não getTitulo()
System.out.println(dto);                // LivroDTO[id=1, titulo=Clean Code, ...]
```

Você pode validar no construtor compacto e adicionar métodos:

```
public record Emprestimo(int usuarioId, int livroId, LocalDate data) {
    public Emprestimo {                                  // construtor compacto
        Objects.requireNonNull(data, "data é obrigatória");
    }
    public LocalDate prazo() { return data.plusDays(14); }
}
```

Records são perfeitos para **DTOs** (objetos de transferência de dados) em APIs REST, que você usará bastante no Spring. **Não** os use como entidades JPA (que precisam ser mutáveis e ter construtor padrão).

2.6 `var`, switch expressions, text blocks e pattern matching

```
// var: inferência de tipo local (o tipo continua estático)
var livros = new ArrayList<Livro>();
var nome = "Ana";

// switch expression: devolve valor, sem break
String msg = switch (status) {
    case ATIVO -> "Com o usuário";
    case DEVOLVIDO -> "Devolvido";
    case ATRASADO -> "Em atraso";
};

// text block: strings de várias linhas (ótimo para SQL e JSON)
String sql = """
        SELECT id, titulo
        FROM livro
        WHERE disponivel = true
        ORDER BY titulo
        """;

// pattern matching de instanceof
if (obj instanceof Livro l && l.isDisponivel()) {
    System.out.println(l.getTitulo());
}
```

Sealed classes (noção)

Restringem quem pode herdar uma classe/interface, útil para modelar hierarquias fechadas:

```
public sealed interface Pagamento permits Pix, Cartao, Boleto { }
public record Pix(String chave) implements Pagamento { }
public record Cartao(String numero) implements Pagamento { }
public record Boleto(String codigo) implements Pagamento { }
```

2.7 Mais da API moderna

- **Coleções imutáveis**: `List.of`, `Set.of`, `Map.of`, `Map.entry`.
- **Métodos novos de String**: `isBlank`, `strip`, `repeat`, `lines`, `formatted`.
- **Comparators**: `Comparator.comparing(...).thenComparing(...).reversed()`.
- `Objects`: `requireNonNull`, `equals`, `hash`, `toString`.
- `Files` **e** `Path`: leitura/escrita de arquivos em poucas linhas (`Files.readAllLines`, `Files.writeString`).

Resumo do capítulo

- Lambdas passam comportamento como valor; method references são atalhos.
- Interfaces funcionais (`Predicate`, `Function`, `Consumer`, `Supplier`) são o "encaixe" das lambdas.
- Streams: **fonte → intermediárias (preguiçosas) → terminal**. Não alteram a coleção original e só podem ser consumidos uma vez.
- `Collectors.groupingBy`, `partitioningBy`, `toMap` e `joining` resolvem a maioria dos agrupamentos.

- `Optional` representa ausência de valor no **retorno** de métodos; prefira `orElseThrow`, `map` e `ifPresent` a `get()`.
- Records criam DTOs imutáveis; `var`, switch expressions, text blocks e pattern matching reduzem código repetitivo.
- Use versões LTS (17/21).

Exercícios teóricos

1. Qual a diferença entre operações intermediárias e terminais em um Stream?
2. Por que um Stream não pode ser reutilizado?
3. O que é uma interface funcional? Cite três do pacote `java.util.function`.
4. Qual a diferença entre `map` e `flatMap`?
5. Por que `Optional` não deve ser usado como atributo de classe?
6. Diferencie `orElse` e `orElseGet`.
7. Quais métodos um `record` gera automaticamente? Quando não usá-lo?
8. Por que a variável usada dentro de uma lambda precisa ser "efetivamente final"?
9. O que significa a avaliação "preguiçosa" (lazy) dos Streams?
10. Quando um `for` tradicional é preferível a um Stream?

Exercícios práticos

Use esta lista: `List<Integer> nums = List.of(5, 12, 8, 3, 21, 7, 12, 40, 15);`

1. Filtre os pares, eleve ao quadrado e some o resultado.
2. Devolva os 3 maiores números distintos em ordem decrescente.
3. Dada uma lista de `String` (nomes), devolva uma só `String` com os nomes em maiúsculas separados por vírgula.
4. Crie `record Funcionario(String nome, String setor, double salario)` e, com uma lista deles: (a) salário médio por setor; (b) o funcionário mais bem pago de cada setor; (c) quantos ganham acima de 5.000.
5. Escreva `Optional<Funcionario> buscarPorNome(String nome)` e consuma-o de três formas diferentes.
6. Crie uma interface funcional `Validador<T>` e valide e-mails com uma lambda.
7. Reescreva um `for` com `if` aninhado que você já tem no projeto da Biblioteca usando Streams, e compare a legibilidade.
8. Converta um `switch` antigo para switch expression.

Projeto principal: Sistema Biblioteca

Refatore a Biblioteca para o estilo moderno:

1. `buscarLivro(String termo)` com Stream e `filter`, ignorando maiúsculas e minúsculas.
2. `Optional<Livro> buscarPorId(int)` e `Optional<Usuario> buscarUsuarioPorId(int)`; quem usa lança a exceção própria com `orElseThrow`.
3. Relatórios com Streams: livros disponíveis, livros por autor (`groupingBy`), usuários com empréstimos ativos, ranking dos 3 livros mais emprestados.

4. Use `record` para um `ResumoEmprestimoDTO(String usuario, String livro, LocalDate prazo)` e imprima o relatório de empréstimos com ele.
5. Novo item no menu: **9 - Relatórios** (usando os Streams acima).

Mini-projeto: Analisador de Vendas

Em outro repositório, crie um programa que leia um CSV de vendas (`data;produto;categoria;quantidade;preco`) e produza, usando **apenas Streams**:

- Faturamento total e ticket médio.
- Faturamento por categoria, ordenado do maior para o menor.
- Top 5 produtos mais vendidos (por quantidade).
- Mês com maior faturamento (use `java.time`).
- Saída formatada com text blocks e `formatted`.

Modele as linhas como `record Venda(...)` e trate linhas inválidas sem derrubar o programa.

3. Git e GitHub

Explicação geral

Git é um sistema de controle de versão distribuído: registra a história do seu código, permite voltar no tempo, trabalhar em ideias paralelas (branches) e colaborar sem sobrescrever o trabalho dos outros. **GitHub** é uma plataforma que hospeda repositórios Git na nuvem e adiciona colaboração (Pull Requests, Issues, Actions). Não são a mesma coisa: Git é a ferramenta; GitHub é um serviço (existem também GitLab e Bitbucket).

Para quem busca a primeira vaga, o GitHub é a **vitrine**: recrutadores olham seus repositórios, seus commits e seus READMEs. Em qualquer empresa, Git é obrigatório desde o primeiro dia.

3.1 O modelo mental do Git

Todo arquivo passa por três áreas locais e, depois, chega ao repositório remoto:

Área	O que é	Comando para mover
Working directory	Seus arquivos como estão agora	edição normal
Staging area (index)	O que entrará no próximo commit	`git add`
Repositório local	A história de commits	`git commit`
Repositório remoto (GitHub)	Cópia na nuvem	`git push` / `git pull`

Um **commit** é uma fotografia do projeto com autor, data e mensagem. Uma **branch** é apenas um ponteiro móvel para um commit. `HEAD` indica onde você está agora.

3.2 Configuração inicial

```
git config --global user.name "Seu Nome"
git config --global user.email "seu@email.com"
git config --global init.defaultBranch main
git config --list
```

Para autenticar no GitHub, use **SSH** (recomendado) ou um *Personal Access Token*. Senha de conta não funciona mais no `push`.

```
ssh-keygen -t ed25519 -C "seu@email.com"
# copie o conteúdo de ~/.ssh/id_ed25519.pub e cole em GitHub > Settings > SSH keys
ssh -T git@github.com
```

3.3 Comandos essenciais

```
git init                      # cria um repositório no diretório atual
git clone <url>               # copia um repositório remoto
git status                    # o que mudou?
git add arquivo.java          # prepara um arquivo
git add .                     # prepara tudo (cuidado)
git commit -m "feat: adiciona cadastro de livro"
git log --oneline --graph     # histórico resumido
git diff                      # mudanças ainda não preparadas
git diff --staged             # mudanças preparadas
git push origin main          # envia commits
git pull                      # traz e integra commits remotos
git fetch                     # só traz, sem integrar
```

O fluxo diário

1. `git pull` (atualiza antes de começar).
2. Edita o código.
3. `git status` e `git diff` (revise o que mudou).
4. `git add` do que pertence ao mesmo assunto.
5. `git commit -m "mensagem clara"`.
6. `git push`.

3.4 Bons commits

Um commit deve ser **pequeno, coeso e descrever uma mudança**. Evite "ajustes", "update" ou "final2". Um padrão muito usado é o **Conventional Commits**:

Prefixo	Uso	Exemplo
`feat`	Nova funcionalidade	`feat: permite devolver livro`
`fix`	Correção de bug	`fix: impede empréstimo de livro indisponível`
`refactor`	Reorganização sem mudar comportamento	`refactor: extrai BibliotecaService`
`docs`	Documentação	`docs: atualiza README`
`test`	Testes	`test: cobre regra de limite de empréstimos`
`chore`	Tarefas de manutenção	`chore: atualiza dependências`

> **DICA** Escreva a mensagem no imperativo e em até ~70 caracteres na primeira linha. Se precisar explicar o porquê, deixe uma linha em branco e escreva o corpo.

3.5 `.gitignore`: o que não versionar

Nunca versione: binários compilados, arquivos da IDE, senhas, chaves e dados locais.

```
# Java / Maven
target/
*.class
*.jar

# IDEs
.idea/
*.iml
.vscode/

# Segredos e ambiente
.env
application-local.properties

# Sistema
.DS_Store
```

> **ATENÇÃO** Se você commitou uma senha ou chave por engano, **apagar no commit seguinte não basta**: ela continua no histórico. Revogue/troque a credencial imediatamente.

Para ignorar algo já rastreado: `git rm -r --cached target/` e commite.

3.6 Branches e merge

Branches permitem desenvolver uma funcionalidade isolada sem bagunçar a principal.

```
git branch                        # lista
git switch -c feature/emprestimo  # cria e muda para a nova branch
git switch main                   # volta para a main
git merge feature/emprestimo      # integra a feature na main
git branch -d feature/emprestimo  # apaga a branch já integrada
```

Conflitos de merge

Acontecem quando duas branches alteram **o mesmo trecho**. O Git marca assim:

```
<<<<<<< HEAD
    private int limite = 3;
=======
    private int limite = 5;
>>>>>>> feature/limite
```

Para resolver: edite o arquivo escolhendo (ou combinando) o resultado, remova os marcadores, depois `git add arquivo` e `git commit`. A IDE tem uma ferramenta visual de merge que ajuda.

Rebase (noção)

`git rebase main` "reaplica" seus commits por cima da main, deixando o histórico linear. É poderoso, mas **nunca faça rebase em branch já compartilhada** com outras pessoas.

3.7 Desfazendo coisas

Situação	Comando
Descartar mudanças não commitadas num arquivo	`git restore arquivo`
Tirar arquivo do staging	`git restore --staged arquivo`
Corrigir a última mensagem de commit (ainda não enviado)	`git commit --amend`
Desfazer um commit já publicado, criando um novo que o reverte	`git revert <hash>`
Guardar trabalho pela metade	`git stash` / `git stash pop`
Voltar a branch para um commit (destrutivo)	`git reset --hard <hash>`

> **ATENÇÃO** `git reset --hard` e `git push --force` podem **destruir trabalho**. Em dúvida, prefira `revert` e crie uma branch de segurança antes.

3.8 Trabalhando com GitHub

```
# Conectar um repositório local a um remoto vazio no GitHub
git remote add origin git@github.com:seu-usuario/sistema-biblioteca.git
git branch -M main
git push -u origin main
```

Pull Request (PR)

Um PR é um pedido para integrar sua branch na `main`, com espaço para **revisão de código**. Fluxo comum:

1. Crie uma branch: `git switch -c feature/limite-emprestimos`.
2. Faça commits e dê `git push -u origin feature/limite-emprestimos`.

3. No GitHub, abra o Pull Request descrevendo **o quê** e **por quê**.
4. Receba/faça revisão; ajuste com novos commits.
5. Faça o merge e apague a branch.

Mesmo sozinho, usar PRs nos seus projetos treina o fluxo que as empresas usam e deixa um histórico organizado.

Modelos de fluxo de branches

- **GitHub Flow** (simples): `main` sempre estável + branches curtas de feature + PR. Ideal para você.
- **Git Flow** (mais pesado): `main`, `develop`, `feature/*`, `release/*`, `hotfix/*`.

Outros recursos do GitHub

- **Issues**: registre tarefas e bugs (use como seu quadro de planejamento).
- **README.md**: a cara do projeto (veja seção 3.9).
- **Releases/Tags**: marque versões (`git tag v1.0.0`).
- **GitHub Actions**: automação (testes a cada push; capítulo 13).
- **Fork**: cópia de um repositório de terceiros na sua conta, para contribuir.

3.9 README profissional

O README é a primeira (e às vezes única) coisa que o recrutador lê. Estrutura recomendada:

1. **Título e descrição** em 2 ou 3 linhas (o problema que resolve).
2. **Tecnologias** utilizadas.
3. **Funcionalidades** e regras de negócio principais.
4. **Como executar** (pré-requisitos e comandos exatos).
5. **Endpoints** (para APIs) ou prints do sistema.
6. **Estrutura do projeto** (pastas/camadas).
7. **Decisões e aprendizados** (o que você faria diferente).
8. **Autor/contato**.

3.10 Tópicos úteis que costumam salvar o dia

- `git reflog`: lista onde o `HEAD` esteve. É a rede de segurança para recuperar commits "perdidos" depois de um `reset --hard` ou de um rebase errado.
- **Fim de linha (Windows × Linux)**: Windows usa CRLF, Linux usa LF. Em equipes mistas (e dentro de contêineres Docker) isso causa diffs enormes e scripts `.sh` que não rodam. Crie um `.gitattributes` com `* text=auto eol=lf`.
- `LICENSE`: um projeto público sem licença não pode, legalmente, ser reutilizado. Para portfólio, MIT é uma escolha comum.
- `git bisect`: busca binária no histórico para achar o commit que introduziu um bug.

3.11 Boas práticas

- Faça commits frequentes e pequenos; um commit = uma ideia.
- Nunca trabalhe direto na `main` em projetos sérios: use branches + PR.
- Rode o projeto/testes antes de commitar.
- Não versione `target/`, `.idea/` nem segredos.

- Perfil do GitHub: foto, bio, e um README de perfil; fixe (pin) seus 4 a 6 melhores repositórios.
- Mantenha o histórico honesto: é normal ter commits de correção; não é necessário "esconder" o aprendizado.

Resumo do capítulo

- Git é local e distribuído; GitHub hospeda e colabora.
- Fluxo: working directory → `add` → staging → `commit` → repositório local → `push` → remoto.
- Branch é um ponteiro; `switch -c` cria, `merge` integra; conflitos se resolvem editando e commitando.
- Commits pequenos e descritivos (Conventional Commits); `.gitignore` para `target/`, IDE e segredos.
- `restore` descarta, `revert` desfaz com segurança, `reset --hard` e `push --force` são perigosos.
- PRs organizam revisão; README é a vitrine do projeto.

Exercícios teóricos

1. Qual a diferença entre Git e GitHub?
2. Explique working directory, staging area e repositório local.
3. O que é uma branch? Por que usar uma por funcionalidade?
4. Qual a diferença entre `git fetch` e `git pull`?
5. Quando ocorre um conflito de merge e como resolvê-lo?
6. Qual a diferença entre `git revert` e `git reset --hard`?
7. Por que não se deve versionar `target/` ou arquivos `.env`?
8. O que é um Pull Request e que vantagens ele traz mesmo em projeto solo?
9. Cite três boas práticas para mensagens de commit.

Exercícios práticos

1. Crie um repositório local, faça 5 commits (um por arquivo/ideia) e visualize com `git log --oneline --graph`.
2. Crie a branch `feature/teste`, altere um arquivo, volte para `main` e faça o merge.
3. **Provoque um conflito de propósito**: altere a mesma linha em duas branches, faça o merge e resolva.
4. Crie o `.gitignore` de um projeto Java/Maven e confirme com `git status` que `target/` não aparece.
5. Use `git stash` para guardar mudanças, trocar de branch e recuperá-las.
6. Desfaça um commit com `git revert` e veja o que aparece no log.
7. Publique um repositório no GitHub via SSH e abra um Pull Request de uma branch para a `main`.
8. Faça um *fork* de um repositório público, clone-o e altere o README.

Projeto principal: Sistema Biblioteca

1. Garanta que o repositório do Sistema Biblioteca está no GitHub, com `.gitignore` correto.
2. Crie o fluxo: `main` estável + uma branch por funcionalidade. Implemente em **branches separadas** (e abra PRs para cada uma): `feature/limite-emprestimos`, `feature/relatorios` e `docs/readme`.
3. Escreva um **README** seguindo a seção 3.9, com as regras de negócio do projeto.
4. Crie **Issues** para o roadmap do projeto (Maven, API, banco, testes, Docker) e use um quadro (Projects) para acompanhar.

5. Marque uma tag `v0.1.0` na versão console concluída.

Mini-projeto: Repositório de Anotações de Estudo

Crie um repositório `meus-estudos-java` que será o seu "diário técnico":

- Pastas por tema (`01-java-basico`, `02-collections`, `03-streams`...).
- Cada pasta com exemplos de código e um `README.md` curto resumindo o conceito com suas palavras.
- No mínimo 15 commits em Conventional Commits, 3 branches integradas por PR e uma tag `v1.0`.
- Uma Issue por tema que você ainda quer estudar.
- Perfil do GitHub: README de perfil (repositório com o seu próprio nome de usuário) com apresentação, tecnologias e links.

4. Maven

Explicação geral

Maven é a ferramenta de **build e gerenciamento de dependências** mais usada no ecossistema Java. Sem ela, para usar uma biblioteca (por exemplo, o Spring) você precisaria baixar JARs manualmente, colocar no classpath e ainda descobrir quais outras bibliotecas ela exige. Com Maven, você **declara** o que precisa em um arquivo `pom.xml` e ele resolve, baixa e organiza tudo; também compila, testa e empacota a aplicação com comandos padronizados.

Todo projeto Spring Boot de mercado usa Maven (ou Gradle, alternativa com a mesma ideia). Entender o `pom.xml` é requisito básico para qualquer vaga.

4.1 Convenção sobre configuração

Maven define uma **estrutura padrão de pastas**. Se você a seguir, quase não precisa configurar nada:

```
sistema-biblioteca/
├── pom.xml
├── src/
│   ├── main/
│   │   ├── java/        ← código-fonte da aplicação
│   │   └── resources/   ← arquivos de configuração (application.properties)
│   └── test/
│       ├── java/        ← código de testes
│       └── resources/
└── target/              ← gerado pelo Maven (não versionar!)
```

4.2 O `pom.xml`

POM significa *Project Object Model*. Exemplo mínimo para a Biblioteca:

```
<project xmlns="http://maven.apache.org/POM/4.0.0">
  <modelVersion>4.0.0</modelVersion>

  <groupId>com.exemplo</groupId>
  <artifactId>sistema-biblioteca</artifactId>
  <version>1.0.0-SNAPSHOT</version>
  <packaging>jar</packaging>

  <properties>
    <maven.compiler.release>21</maven.compiler.release>
    <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
  </properties>

  <dependencies>
    <dependency>
      <groupId>org.junit.jupiter</groupId>
      <artifactId>junit-jupiter</artifactId>
      <version>5.11.4</version>      <!-- exemplo: confira a versão atual no Maven Central -->
      <scope>test</scope>
    </dependency>
  </dependencies>

  <build>
    <plugins>
      <!-- Necessário para executar testes JUnit 5 em projetos SEM o Spring Boot parent.
           Surefire antigo (2.x) ignora JUnit 5 e o build passa com "No tests run". -->
      <plugin>
        <groupId>org.apache.maven.plugins</groupId>
        <artifactId>maven-surefire-plugin</artifactId>
        <version>3.5.2</version>
```

```
      </plugin>
    </plugins>
  </build>
</project>
```

Coordenadas (GAV)

Todo artefato é identificado por três informações:

Campo	Significado	Exemplo
`groupId`	Organização/domínio (normalmente domínio invertido)	`com.exemplo`
`artifactId`	Nome do projeto/biblioteca	`sistema-biblioteca`
`version`	Versão	`1.0.0-SNAPSHOT`

`SNAPSHOT` indica uma versão em desenvolvimento (instável). Versões finais não têm o sufixo (`1.0.0`).

Escopos (`scope`) das dependências

Escopo	Disponível em	Exemplo típico
`compile` (padrão)	Compilação, testes e execução	Spring Web
`provided`	Compilação e testes (o ambiente fornece em execução)	Servlet API
`runtime`	Só na execução	Driver JDBC (PostgreSQL)
`test`	Só nos testes	JUnit, Mockito

Dependências transitivas

Se você adiciona a biblioteca A e ela depende de B, o Maven traz B automaticamente. Para ver a árvore completa (e descobrir conflitos de versão):

```
mvn dependency:tree
```

4.3 Repositórios

Maven baixa as dependências do **Maven Central** (https://central.sonatype.com ou https://mvnrepository.com para pesquisar). Elas ficam em cache local em `~/.m2/repository`, então só são baixadas uma vez. Se algo corromper, apagar a pasta da biblioteca em `~/.m2` e rodar de novo costuma resolver.

4.4 Ciclo de vida (lifecycle)

O Maven organiza o build em **fases** executadas em ordem. Ao chamar uma fase, todas as anteriores rodam antes.

Fase	O que faz
`validate`	Verifica se o projeto está correto
`compile`	Compila `src/main/java`
`test`	Executa os testes unitários
`package`	Gera o `.jar` em `target/`
`verify`	Executa verificações/testes de integração

Fase	O que faz
`install`	Instala o artefato no repositório local `~/.m2`
`deploy`	Publica em repositório remoto

Há ainda o ciclo **clean**, que apaga a pasta `target/`.

Comandos do dia a dia

```
mvn clean            # apaga target/
mvn compile          # compila
mvn test             # compila e roda os testes
mvn clean package    # build limpo + gera o jar (roda os testes)
mvn clean package -DskipTests   # gera o jar sem rodar testes (use com moderação)
mvn clean install    # empacota e instala no repositório local
mvn dependency:tree  # árvore de dependências
```

Para rodar o jar gerado: `java -jar target/sistema-biblioteca-1.0.0-SNAPSHOT.jar` (precisa de um `Main-Class` configurado ou, no Spring Boot, do plugin dele).

4.5 Plugins

Plugins adicionam capacidades ao build. Cada fase do ciclo é executada por plugins ligados a ela (por exemplo, `maven-compiler-plugin` compila, `maven-surefire-plugin` executa os testes).

```
<build>
  <plugins>
    <plugin>
      <groupId>org.apache.maven.plugins</groupId>
      <artifactId>maven-jar-plugin</artifactId>
      <version>3.4.1</version>
      <configuration>
        <archive>
          <manifest>
            <mainClass>com.exemplo.biblioteca.ui.Main</mainClass>
          </manifest>
        </archive>
      </configuration>
    </plugin>
  </plugins>
</build>
```

Com o `mainClass` definido, `java -jar target/sistema-biblioteca-1.0.0-SNAPSHOT.jar` funciona para o projeto console. No Spring Boot, o `spring-boot-maven-plugin` cuida disso e gera um "fat jar" executável com tudo dentro.

4.6 Herança de POM e BOM

- **Parent POM**: seu projeto pode herdar configurações de outro. O Spring Boot usa `spring-boot-starter-parent` para fixar as versões compatíveis de centenas de bibliotecas. Por isso nas dependências Spring você **não informa a versão**.
- **BOM** (Bill of Materials): lista de versões compatíveis importada via `dependencyManagement`.
- **Propriedades**: evite repetir versões; use `<properties>` e `${...}`.

```
<properties>
  <junit.version>5.10.2</junit.version>
</properties>
...
<version>${junit.version}</version>
```

4.7 Maven Wrapper

O wrapper (`mvnw` / `mvnw.cmd`) fixa a versão do Maven no próprio projeto. Quem clonar o repositório pode rodar `./mvnw clean package` sem ter o Maven instalado. O Spring Initializr (capítulo 6) já o inclui. **Versione** o wrapper.

4.8 Problemas comuns

Sintoma	Causa provável	O que fazer
`release version 21 not supported`	JDK instalado é menor que o configurado	Ajuste o JDK da IDE/`JAVA_HOME` ou o `release`
Dependência "não encontrada"	Typo no GAV ou sem internet	Confira no mvnrepository; force `mvn -U`
Testes não executam (`No tests run`)	Nome fora do padrão (`*Test`), pasta errada, **ou Surefire antigo sem suporte a JUnit 5**	Siga `src/test/java` e sufixo `Test`; declare `maven-surefire-plugin` 3.x
`package` falha por testes	Teste quebrado	Corrija o teste; não use `-DskipTests` para esconder
Comportamento estranho do build	Cache sujo	`mvn clean` e, se preciso, limpe `~/.m2` da lib

Resumo do capítulo

- Maven automatiza **build, testes, empacotamento e dependências**.
- Estrutura padrão: `src/main/java`, `src/main/resources`, `src/test/java`, `target/`.
- `pom.xml` define coordenadas (GAV), propriedades, dependências e plugins.
- Escopos: `compile`, `provided`, `runtime`, `test`.
- Ciclo: `validate → compile → test → package → verify → install → deploy`; `clean` apaga `target/`.
- `spring-boot-starter-parent` gerencia versões; o wrapper `mvnw` dispensa instalação do Maven.

Exercícios teóricos

1. Que problemas o Maven resolve em relação a baixar JARs manualmente?
2. O que significam `groupId`, `artifactId` e `version`?
3. Qual a diferença entre `mvn package` e `mvn install`?
4. Para que serve o escopo `test`? E o `runtime`?
5. O que são dependências transitivas? Como visualizá-las?
6. O que acontece ao rodar `mvn test`? Quais fases são executadas?
7. Por que `target/` não deve ir para o Git?
8. Para que serve o Maven Wrapper?
9. Qual o papel do `spring-boot-starter-parent`?

Exercícios práticos

1. Crie um projeto Maven do zero (sem IDE) com a estrutura padrão e um `HelloWorld`. Compile e rode com `mvn`.

2. Adicione a dependência `commons-lang3` e use `StringUtils.capitalize(...)`. Rode `mvn dependency:tree`.
3. Adicione JUnit 5 com escopo `test`, escreva um teste simples e execute `mvn test`.
4. Quebre o teste de propósito e veja o `mvn package` falhar. Conserte.
5. Configure o `maven-jar-plugin` com `mainClass` e gere um jar executável.
6. Use `<properties>` para centralizar uma versão usada em duas dependências.
7. Gere o wrapper com `mvn wrapper:wrapper` e rode `./mvnw clean test`.

Projeto principal: Sistema Biblioteca

1. **Migre** a versão console para um projeto Maven com `groupId` seu e `artifactId sistema-biblioteca`.
2. Configure Java 21 (ou 17) no `pom.xml`.
3. Organize os pacotes em `src/main/java`: `model`, `service`, `exception`, `ui`.
4. Adicione JUnit 5 e escreva **3 testes** para `Livro` e `Biblioteca` (por exemplo: emprestar muda a disponibilidade; emprestar livro indisponível lança exceção).
5. Gere o jar executável e rode `java -jar`. Documente no README os comandos `mvn clean test` e `mvn clean package`.
6. Faça commit em uma branch `chore/maven` e abra um PR.

Mini-projeto: Biblioteca de Utilitários (artefato reutilizável)

Crie um projeto Maven que seja uma **biblioteca**, não uma aplicação:

- Artefato `utils-texto` com classes estáticas: `Cpf.valido(String)`, `Slug.de(String)` (transforma "Olá Mundo!" em "ola-mundo"), `Palindromo.ehPalindromo(String)`.
- Pelo menos 10 testes JUnit cobrindo casos normais e extremos (nulo, vazio, espaços).
- Rode `mvn clean install` para instalá-la no repositório local.
- Crie **outro projeto Maven** que declare `utils-texto` como dependência e a utilize. Assim você entende na prática o que significa "publicar e consumir um artefato".

5. HTTP e REST

Explicação geral

Quase todo back-end moderno existe para ser **consumido por outro programa**: um site em React, um aplicativo de celular, outro serviço. A conversa entre eles acontece pela web, usando o protocolo **HTTP**, e o estilo de organização mais comum dessa conversa é **REST**. Uma **API REST** expõe *recursos* (livros, usuários, empréstimos) por meio de URLs e manipula-os com os verbos HTTP, trocando dados normalmente em **JSON**.

Antes de escrever uma linha de Spring, você precisa dominar esse vocabulário, porque o Spring é, em essência, uma forma de **transformar métodos Java em endpoints HTTP**.

```
Cliente (Postman, React, app)           Servidor (sua API Spring Boot)
        |    --- Requisição HTTP --->            |
        |    GET /livros/5                       |
        |    <--- Resposta HTTP ---              |
        |    200 OK  { "id": 5, "titulo": ... }  |
```

5.1 Como o HTTP funciona

HTTP é um protocolo **requisição-resposta** e **sem estado** (*stateless*): cada requisição é independente, o servidor não "lembra" da anterior. Por isso, autenticação precisa ser reenviada a cada chamada (via token).

Anatomia de uma requisição

```
POST /api/livros HTTP/1.1
Host: localhost:8080
Content-Type: application/json
Authorization: Bearer eyJhbGciOi...

{
  "titulo": "Java Efetivo",
  "autor": "Joshua Bloch",
  "ano": 2018
}
```

Parte	Descrição
Método	O que quer fazer (GET, POST...)
URL/Path	Qual recurso (`/api/livros`)
Headers	Metadados: tipo do conteúdo, autenticação, idioma
Body	Dados enviados (em POST, PUT, PATCH)

Anatomia de uma resposta

```
HTTP/1.1 201 Created
Content-Type: application/json
Location: /api/livros/7

{ "id": 7, "titulo": "Java Efetivo", "autor": "Joshua Bloch", "ano": 2018, "disponivel": true }
```

Composta por **status code**, **headers** e **body**.

Partes de uma URL

`https://api.exemplo.com:8080/api/livros/5?disponivel=true&pagina=0#topo`

Parte	Valor
Esquema	`https`
Host e porta	`api.exemplo.com:8080`
Path	`/api/livros/5`
Path variable	`5` (identifica o recurso)
Query string	`?disponivel=true&pagina=0` (filtros, paginação)

5.2 Métodos HTTP

Método	Uso	Body?	Seguro?	Idempotente?
`GET`	Ler/consultar	Não	Sim	Sim
`POST`	Criar um recurso (ou executar uma ação)	Sim	Não	Não
`PUT`	Substituir um recurso por completo	Sim	Não	Sim
`PATCH`	Alterar parcialmente	Sim	Não	Em geral, não garantido
`DELETE`	Remover	Normalmente não	Não	Sim

- **Seguro**: não altera o estado do servidor.
- **Idempotente**: chamar uma ou dez vezes produz o mesmo resultado final. `DELETE /livros/5` repetido continua resultando em "livro 5 não existe". Já `POST /livros` repetido cria vários livros.

> **DICA** Pergunta clássica de entrevista: "Qual a diferença entre PUT e PATCH?" Resposta: PUT envia o recurso **completo** e o substitui; PATCH envia **apenas os campos** a alterar.

5.3 Status codes

O código diz ao cliente o que aconteceu. Agrupam-se por centena:

Faixa	Significado
1xx	Informativo
2xx	Sucesso
3xx	Redirecionamento
4xx	**Erro do cliente** (pedido inválido)
5xx	**Erro do servidor** (falha na sua aplicação)

Os que você precisa saber de cor:

Código	Nome	Quando usar
200	OK	Leitura/atualização bem-sucedida
201	Created	Recurso criado (envie o header `Location`)
204	No Content	Sucesso sem corpo (ex.: DELETE)

Código	Nome	Quando usar
400	Bad Request	Dados inválidos/malformados
401	Unauthorized	Não autenticado (sem login/token válido)
403	Forbidden	Autenticado, mas sem permissão
404	Not Found	Recurso não existe
409	Conflict	Conflito de regra (e-mail duplicado, livro já emprestado)
422	Unprocessable Entity	Sintaxe ok, mas regra de validação falhou (usado por alguns times no lugar de 400)
500	Internal Server Error	Falha inesperada no servidor

> **ATENÇÃO** Erro comum de iniciante: devolver `200 OK` com a mensagem de erro dentro do corpo. O status code **é** parte do contrato. Um livro inexistente deve responder `404`, não `200`.

5.4 JSON

JSON (*JavaScript Object Notation*) é o formato de troca de dados mais usado em APIs.

```
{
  "id": 7,
  "titulo": "Java Efetivo",
  "autor": { "nome": "Joshua Bloch", "pais": "EUA" },
  "ano": 2018,
  "disponivel": true,
  "tags": ["java", "boas práticas"]
}
```

Tipos: objeto `{}`, array `[]`, string, número, booleano e `null`. Chaves **sempre** entre aspas duplas; sem vírgula sobrando no final; não aceita comentários.

No Java, a conversão entre objeto e JSON (serialização/desserialização) é feita pela biblioteca **Jackson**, que o Spring já usa por baixo dos panos. Um `record` ou classe simples vira JSON automaticamente.

5.5 O que é REST

REST (*Representational State Transfer*) é um conjunto de princípios arquiteturais, não um protocolo. Os mais importantes na prática:

1. **Recursos como substantivos**: a URL identifica "coisas" (`/livros`), não ações (`/buscarLivros`).
2. **Verbos HTTP** expressam a ação.
3. **Stateless**: cada requisição carrega tudo de que o servidor precisa.
4. **Representações**: o mesmo recurso pode ser entregue em JSON, XML etc.
5. **Interface uniforme**: padrões consistentes entre todos os endpoints.
6. Cliente e servidor **independentes** (o front pode mudar sem afetar o back).

Desenhando URLs

Ruim	Bom	Por quê
`GET /getLivros`	`GET /livros`	Verbo já está no método HTTP
`POST /criarLivro`	`POST /livros`	Idem
`GET /livro?id=5`	`GET /livros/5`	Identificador no path

Ruim	Bom	Por quê
`POST /deletarLivro/5`	`DELETE /livros/5`	Use o verbo certo
`/Livros_Cadastrados`	`/livros`	Minúsculas, plural, sem underscore

Mapeamento CRUD

Operação	Requisição	Sucesso
Listar	`GET /livros`	200
Buscar um	`GET /livros/{id}`	200 / 404
Criar	`POST /livros`	201 + `Location`
Atualizar (completo)	`PUT /livros/{id}`	200 / 404
Atualizar (parcial)	`PATCH /livros/{id}`	200 / 404
Remover	`DELETE /livros/{id}`	204 / 404

Recursos aninhados e ações

Relacionamentos naturais viram sub-recursos:

```
GET  /usuarios/3/emprestimos        → empréstimos do usuário 3
POST /emprestimos                   → realiza um empréstimo (body: usuarioId, livroId)
POST /emprestimos/12/devolucao      → ação de devolver (modelada como sub-recurso)
GET  /livros?autor=Bloch&disponivel=true&pagina=0&tamanho=10
```

Evite aninhar mais de dois níveis. Para ações que não cabem no CRUD (devolver, cancelar), modele como um **sub-recurso** ou um `POST` com nome claro.

5.6 Boas práticas de API

- **Versionamento**: `/api/v1/livros`, para evoluir sem quebrar clientes.
- **Paginação e ordenação**: nunca devolva 100 mil registros: `?pagina=0&tamanho=20&ordenar=titulo`.
- **Formato de erro padronizado**: o cliente precisa entender o que falhou.

```
{
  "status": 404,
  "erro": "Livro não encontrado",
  "mensagem": "Não existe livro com id=99",
  "caminho": "/api/v1/livros/99",
  "timestamp": "2026-11-05T14:30:00"
}
```

- **Não exponha entidades internas**: use DTOs (capítulo 11).
- **Nunca devolva** senha, hash, tokens ou dados que o cliente não deveria ver.
- Use **HTTPS** em produção.
- Nomes e formatos consistentes (camelCase no JSON; datas em ISO-8601: `2026-11-05`).
- **CORS**: navegadores bloqueiam chamadas entre origens diferentes (ex.: front em `localhost:3000` chamando API em `localhost:8080`) a menos que o servidor autorize explicitamente.

5.7 Ferramentas para testar APIs

curl (terminal)

```
curl http://localhost:8080/api/v1/livros
curl -X POST http://localhost:8080/api/v1/livros \
```

```
     -H "Content-Type: application/json" \
     -d '{"titulo":"Java Efetivo","autor":"Bloch","ano":2018}'
curl -i -X DELETE http://localhost:8080/api/v1/livros/7     # -i mostra status e headers
```

Postman / Insomnia / Bruno

Interface gráfica para montar requisições, organizar **coleções** e usar **variáveis de ambiente** (por exemplo `{{baseUrl}}`). Exporte a coleção e versione no repositório: é ótima documentação viva.

Chamando uma API em Java (cliente HTTP)

```
public static void main(String[] args) throws IOException, InterruptedException {
    HttpClient client = HttpClient.newHttpClient();
    HttpRequest req = HttpRequest.newBuilder()
            .uri(URI.create("https://viacep.com.br/ws/88301000/json/"))
            .header("Accept", "application/json")
            .GET()
            .build();

    // send() declara IOException e InterruptedException (checked): trate ou propague
    HttpResponse<String> resp = client.send(req, HttpResponse.BodyHandlers.ofString());
    System.out.println(resp.statusCode());   // 200
    System.out.println(resp.body());         // JSON em texto
}
```

5.8 Documentando o contrato (OpenAPI)

OpenAPI (antigo Swagger) é um padrão para descrever uma API em um arquivo (YAML/JSON). Ferramentas geram uma página interativa onde se pode testar cada endpoint. No Spring, a biblioteca `springdoc-openapi` faz isso automaticamente a partir do código (capítulo 11). Pensar no contrato **antes** de codar (abordagem *design first*) evita retrabalho.

Resumo do capítulo

- HTTP é requisição-resposta e **stateless**; requisição = método + URL + headers + body; resposta = status + headers + body.
- Métodos: GET (ler), POST (criar), PUT (substituir), PATCH (alterar parcial), DELETE (remover). Idempotência: GET, PUT e DELETE são idempotentes.
- Status: 2xx sucesso (200, 201, 204), 4xx erro do cliente (400, 401, 403, 404, 409), 5xx erro do servidor (500).
- REST: recursos como substantivos no plural, verbo no método HTTP, identificador no path, filtros na query.
- Boas práticas: versionamento, paginação, erros padronizados, DTOs, HTTPS, ISO-8601.
- Teste com curl/Postman e documente com OpenAPI.

Exercícios teóricos

1. O que significa dizer que o HTTP é *stateless*? Qual a consequência para autenticação?
2. Diferencie `PUT` de `PATCH` e `PUT` de `POST`.
3. O que é idempotência? Dê um exemplo de método idempotente e um que não é.
4. Qual a diferença entre `401` e `403`?
5. Quando usar `201` em vez de `200`? E `204`?
6. Por que `POST /deletarLivro/5` é uma má URL REST?
7. Qual a diferença entre path variable e query parameter? Quando usar cada um?

8. Que problema a paginação resolve?
9. O que é CORS e por que ele só afeta navegadores?
10. Quais regras de um JSON válido costumam causar erros em iniciantes?

Exercícios práticos

1. Com `curl` ou Postman, consulte a API pública `https://viacep.com.br/ws/88301000/json/` e identifique status, headers e body.
2. Use a API `https://jsonplaceholder.typicode.com` para fazer GET, POST, PUT, PATCH e DELETE em `/posts` e anote os status devolvidos.
3. Provoque um `404` e um `400` de propósito em alguma API pública e observe a resposta.
4. Escreva em JSON uma lista de 3 livros, cada um com um objeto `autor` aninhado e um array de `tags`. Valide em um validador de JSON.
5. Crie uma coleção no Postman com variável `{{baseUrl}}` e 5 requisições.
6. Reescreva estas URLs ruins no padrão REST: `GET /listarUsuarios`, `POST /novoEmprestimo`, `GET /buscarLivroPorAutor?autor=x`, `POST /excluirUsuario/3`.
7. Em Java, chame a API ViaCEP com `HttpClient` e imprima a cidade a partir do JSON (pode usar `String.contains` ou a biblioteca Jackson).

Projeto principal: Sistema Biblioteca

Desenhe o **contrato da API** antes de programá-la. Crie o arquivo `docs/API.md` no repositório com a tabela de endpoints:

Recurso	Método e URL	Descrição	Sucesso	Erros
Livros	`GET /api/v1/livros`	Lista com filtros e paginação	200	—
Livros	`GET /api/v1/livros/{id}`	Busca por id	200	404
Livros	`POST /api/v1/livros`	Cadastra	201	400
Usuários	`POST /api/v1/usuarios`	Cadastra	201	400, 409 (e-mail)
Empréstimos	`POST /api/v1/emprestimos`	Empresta	201	404, 409
Empréstimos	`POST /api/v1/emprestimos/{id}/devolucao`	Devolve	200	404, 409
Empréstimos	`GET /api/v1/usuarios/{id}/emprestimos`	Histórico do usuário	200	404

Complete o documento com: exemplos de JSON de entrada e saída de cada endpoint, o **formato de erro padrão** e o mapeamento de cada exceção própria (`LivroNaoEncontradoException` → 404, `LivroIndisponivelException` → 409, `UsuarioNaoEncontradoException` → 404). Isso será seguido quando você implementar o Spring.

Mini-projeto: Consumidor de API pública (Painel de CEP e Clima)

Em outro projeto Maven, escreva um programa de console que:

- Pede um CEP, consulta o **ViaCEP** com `HttpClient` e mostra logradouro, bairro, cidade e UF.

- Trata erros de forma elegante: CEP inválido (400 ou campo `erro`), sem internet, resposta inesperada.
- Converte o JSON em um `record Endereco(...)` usando **Jackson** (dependência Maven `jackson-databind`).
- Guarda as últimas 10 consultas em memória e permite listá-las.
- Desafio: cache simples em `Map` para não repetir chamadas ao mesmo CEP.

6. Spring Boot

Explicação geral

Spring é o framework mais usado para back-end em Java. Ele resolve a parte repetitiva e "de infraestrutura" de uma aplicação (criar objetos, conectar peças, expor HTTP, acessar banco, segurança) para que você foque na **regra de negócio**. **Spring Boot** é a camada que torna o Spring fácil de usar: **autoconfiguração**, servidor web embutido (Tomcat) e dependências organizadas em *starters*. Em minutos você tem uma API rodando com `java -jar`.

Duas ideias sustentam todo o Spring e são as perguntas de entrevista mais frequentes:

- **IoC (Inversão de Controle)**: em vez de *você* criar e ligar os objetos (`new`), o **Spring** cria e gerencia.
- **DI (Injeção de Dependência)**: o Spring entrega a cada objeto as dependências de que ele precisa.

A arquitetura em camadas que você vai usar:

```
Cliente HTTP
   |
Controller   ← recebe requisição HTTP, valida entrada, devolve resposta
   |
Service      ← regras de negócio (o "cérebro")
   |
Repository   ← acesso a dados
   |
Banco de dados
```

6.0 Antes de começar: versões e mudança do modelo

> **ATENÇÃO** **Versões.** Os exemplos desta apostila seguem o estilo do Spring Boot 3.x/4.x com Java 17 ou 21. Em novembro de 2026, o Spring Initializr oferece Boot 4.x, e a série 3.5 já saiu do suporte gratuito. Gere o projeto com a **versão estável mais recente** e confira os números de versão de bibliotecas de terceiros (springdoc, Testcontainers, JaCoCo, jjwt) no Maven Central. No Boot 4 mudaram, entre outros pontos: o Jackson 3 é o padrão (pacote `tools.jackson`), `@MockBean` foi substituído por `@MockitoBean`, e os starters de teste ficaram por funcionalidade. Em caso de erro de dependência ou de pacote, consulte o guia oficial de migração do Spring Boot 4.

Nota de transição do modelo. No capítulo 1, `Livro` tinha `int id` informado no construtor. A partir daqui o **banco gera o id** (`Long`), então o id deixa de ser argumento do construtor e passa a ser atribuído pela camada de persistência. O construtor passa a ser `Livro(String titulo, String autor, Integer ano)`. O mesmo vale para `Usuario` e `Emprestimo`. As exceções de domínio continuam *unchecked* e passam a receber `Long`.

6.1 Criando o projeto (Spring Initializr)

Acesse **https://start.spring.io** e escolha:

Campo	Valor sugerido
Project	Maven
Language	Java
Spring Boot	A versão estável mais recente (3.x ou superior)

Campo	Valor sugerido
Group / Artifact	`com.exemplo` / `biblioteca-api`
Packaging / Java	Jar / 21 (ou 17)
Dependências	Web (API REST), Validation, DevTools. (Lombok é opcional e **não** é usado nesta apostila: escrevemos getters à mão ou usamos records)

Gere, extraia o `.zip` e abra na IDE. A estrutura:

```
biblioteca-api/
├── pom.xml
├── mvnw / mvnw.cmd
└── src/
    ├── main/java/com/exemplo/bibliotecaapi/BibliotecaApiApplication.java
    ├── main/resources/application.properties
    └── test/java/.../BibliotecaApiApplicationTests.java
```

```
@SpringBootApplication
public class BibliotecaApiApplication {
    public static void main(String[] args) {
        SpringApplication.run(BibliotecaApiApplication.class, args);
    }
}
```

`@SpringBootApplication` combina três coisas: `@SpringBootConfiguration` (uma especialização de `@Configuration`), `@EnableAutoConfiguration` e `@ComponentScan` (procura componentes no pacote da classe principal e **abaixo dele**; por isso mantenha todas as classes dentro desse pacote base).

Rode com `./mvnw spring-boot:run` e acesse `http://localhost:8080`.

6.2 IoC, DI e Beans

Um **bean** é um objeto criado e gerenciado pelo contêiner do Spring (*ApplicationContext*). Classes anotadas com estereótipos viram beans automaticamente:

Anotação	Camada	Função
`@Component`	Genérica	Bean qualquer
`@Service`	Serviço	Regras de negócio
`@Repository`	Dados	Acesso a dados (traduz exceções de persistência)
`@RestController`	Web	Controller que responde JSON
`@Configuration` + `@Bean`	Configuração	Declara beans manualmente

Injeção de dependência

A forma recomendada é por **construtor**:

```
@Service
public class LivroService {

    private final LivroRepository repository;     // dependência

    public LivroService(LivroRepository repository) {   // o Spring injeta aqui
        this.repository = repository;
    }
}
```

Com um único construtor, `@Autowired` é dispensável. Vantagens da injeção por construtor: campos `final` (imutáveis), fácil de testar (basta passar um *mock*), e o objeto nunca nasce incompleto.

> **ATENÇÃO** Evite injetar por campo (`@Autowired private X x;`). Funciona, mas esconde dependências e dificulta os testes.

Sem Spring, você escreveria `new LivroService(new LivroRepository())` em todo lugar. Com DI, você declara *o que precisa* e o contêiner monta o grafo de objetos.

Escopo dos beans

Por padrão são **singleton**: uma única instância compartilhada na aplicação. Por isso beans **não devem guardar estado mutável por usuário** (risco de concorrência).

`@Bean` e `@Configuration`

Para objetos de bibliotecas de terceiros, que você não pode anotar:

```
@Configuration
public class AppConfig {
    @Bean
    public Clock clock() {
        return Clock.systemDefaultZone();   // injetável onde precisar da data/hora
    }
}
```

6.3 Controllers

O `@RestController` recebe requisições e devolve dados (o Spring converte objetos em JSON via Jackson).

```
@RestController
@RequestMapping("/api/v1/livros")
public class LivroController {

    private final LivroService service;

    public LivroController(LivroService service) { this.service = service; }

    @GetMapping
    public List<LivroResponse> listar() {
        return service.listar();
    }

    @GetMapping("/{id}")
    public LivroResponse buscar(@PathVariable Long id) {
        return service.buscarPorId(id);
    }

    @PostMapping
    public ResponseEntity<LivroResponse> criar(@RequestBody LivroRequest request) {
        LivroResponse criado = service.criar(request);
        URI location = URI.create("/api/v1/livros/" + criado.id());
        return ResponseEntity.created(location).body(criado);     // 201 + Location
    }

    @PutMapping("/{id}")
    public LivroResponse atualizar(@PathVariable Long id, @RequestBody LivroRequest request) {
        return service.atualizar(id, request);
    }

    @DeleteMapping("/{id}")
    @ResponseStatus(HttpStatus.NO_CONTENT)                        // 204
    public void remover(@PathVariable Long id) {
        service.remover(id);
    }
}
```

Anotações de mapeamento e parâmetros

Anotação	Função	Exemplo
`@RequestMapping("/x")`	Prefixo da URL da classe	`/api/v1/livros`
`@GetMapping`, `@PostMapping`, `@PutMapping`, `@PatchMapping`, `@DeleteMapping`	Verbo HTTP do método	
`@PathVariable`	Valor do path	`/livros/{id}`
`@RequestParam`	Parâmetro de query	`?autor=Bloch`
`@RequestBody`	Corpo JSON → objeto Java	
`@RequestHeader`	Valor de um header	
`ResponseEntity<T>`	Controle total de status, headers e body	

```
// ALTERNATIVA ao listar() acima (não adicione os dois: dois @GetMapping no mesmo caminho
// causam "Ambiguous mapping" e a aplicação não sobe)
@GetMapping
public List<LivroResponse> buscar(
        @RequestParam(required = false) String autor,
        @RequestParam(defaultValue = "true") boolean disponivel) { ... }
```

DTOs mínimos (records)

Os exemplos acima usam `LivroRequest` e `LivroResponse`. Por enquanto, crie-os como records simples no pacote `dto` (o capítulo 11 acrescenta validação):

```
public record LivroRequest(String titulo, String autor, Integer ano) { }

public record LivroResponse(Long id, String titulo, String autor, Integer ano, boolean disponivel) {
    public static LivroResponse de(Livro l) {
        return new LivroResponse(l.getId(), l.getTitulo(), l.getAutor(),
                l.getAno(), l.isDisponivel());
    }
}
```

> **DICA** O Controller deve ser **fino**: recebe a requisição, chama o Service e devolve o resultado. Nenhuma regra de negócio aqui.

6.4 Services

O Service concentra as **regras de negócio** e orquestra repositórios. É também onde você lança as exceções do domínio.

```
@Service
public class EmprestimoService {

    private final LivroRepository livros;
    private final UsuarioRepository usuarios;
    private final EmprestimoRepository emprestimos;

    public EmprestimoService(LivroRepository livros, UsuarioRepository usuarios,
                             EmprestimoRepository emprestimos) {
        this.livros = livros;
        this.usuarios = usuarios;
        this.emprestimos = emprestimos;
    }
```

```
    public Emprestimo emprestar(Long usuarioId, Long livroId) {
        Usuario usuario = usuarios.findById(usuarioId)
                .orElseThrow(() -> new UsuarioNaoEncontradoException(usuarioId));
        Livro livro = livros.findById(livroId)
                .orElseThrow(() -> new LivroNaoEncontradoException(livroId));

        livro.emprestar();      // já lança LivroIndisponivelException se não estiver disponível
        return emprestimos.save(new Emprestimo(usuario, livro, LocalDate.now()));
    }
}
```

6.5 Repositories (em memória, por enquanto)

Neste capítulo, o repositório guarda os dados em um `Map`. O objetivo é que, no capítulo 7, ele seja substituído pelo Spring Data JPA **sem mexer no Service**. Para isso funcionar de verdade, precisamos de duas coisas: (1) o Service depende de uma **interface**, e (2) os métodos têm **os mesmos nomes** que o Spring Data usa (`save`, `findById`, `findAll`, `deleteById`).

```
// O contrato: o Service só conhece esta interface
public interface LivroRepository {
    Livro save(Livro livro);
    Optional<Livro> findById(Long id);
    List<Livro> findAll();
    void deleteById(Long id);
}

// Implementação em memória (será APAGADA no capítulo 7)
@Repository
public class LivroRepositoryMemoria implements LivroRepository {
    private final Map<Long, Livro> dados = new ConcurrentHashMap<>();
    private final AtomicLong sequencia = new AtomicLong(1);

    @Override public Livro save(Livro livro) {
        // setId só existe nesta fase
        if (livro.getId() == null) livro.setId(sequencia.getAndIncrement());
        dados.put(livro.getId(), livro);
        return livro;
    }
    @Override public Optional<Livro> findById(Long id) { return Optional.ofNullable(dados.get(id)); }
    @Override public List<Livro> findAll() { return new ArrayList<>(dados.values()); }
    @Override public void deleteById(Long id) { dados.remove(id); }
}
```

No capítulo 7, você apaga `LivroRepositoryMemoria` e faz `LivroRepository extends JpaRepository<Livro, Long>`. Como os nomes dos métodos coincidem, **o Service não muda**.

6.6 Configuração: `application.properties` e YAML

A configuração externa fica em `src/main/resources`. Pode ser `.properties` ou `.yml`:

```
server.port=8080
spring.application.name=biblioteca-api
logging.level.com.exemplo=DEBUG
```

```
server:
  port: 8080
spring:
  application:
    name: biblioteca-api
logging:
  level:
    com.exemplo: DEBUG
```

Lendo valores de configuração

```
@Value("${biblioteca.prazo-dias:14}")      // 14 é o valor padrão
private int prazoDias;
```

Para vários valores, use um `record` com `@ConfigurationProperties`:

```
@ConfigurationProperties(prefix = "biblioteca")
// dinheiro: BigDecimal
public record BibliotecaProperties(int prazoDias, BigDecimal multaPorDia, int limiteEmprestimos) { }
// habilite com @ConfigurationPropertiesScan na classe principal
```

Profiles

Profiles separam configurações por ambiente (`dev`, `test`, `prod`). Crie `application-dev.properties` e `application-prod.properties` e ative com:

```
java -jar app.jar --spring.profiles.active=prod
# ou variável de ambiente: SPRING_PROFILES_ACTIVE=prod
```

> **ATENÇÃO** **Nunca** coloque senhas no repositório. Use variáveis de ambiente: `spring.datasource.password=${DB_PASSWORD}`.

6.7 Starters e autoconfiguração

Um *starter* é uma dependência "guarda-chuva" que traz tudo que você precisa para uma finalidade:

Starter	Para quê
`spring-boot-starter-web`	API REST (Tomcat + Jackson + Spring MVC)
`spring-boot-starter-validation`	Bean Validation
`spring-boot-starter-data-jpa`	JPA/Hibernate + Spring Data
`spring-boot-starter-test`	JUnit, Mockito, AssertJ, MockMvc
`spring-boot-starter-security`	Spring Security
`spring-boot-starter-actuator`	Monitoramento (`/actuator/health`)

A **autoconfiguração** detecta o que está no classpath e configura sozinha (por exemplo, se há driver do PostgreSQL e `spring.datasource.url`, cria o `DataSource`). Para ver o que foi ativado, use `debug=true` nas propriedades.

6.8 Ciclo de uma requisição

1. O cliente chama `POST /api/v1/livros`.
2. O Tomcat embutido entrega a requisição ao `DispatcherServlet`.
3. O Spring MVC localiza o método do Controller pelo mapeamento.
4. O Jackson converte o JSON no objeto do `@RequestBody`.
5. O Controller chama o Service, que chama o Repository.
6. O retorno é convertido em JSON e a resposta HTTP volta ao cliente.

6.9 Problemas comuns (e como ler o erro)

Mensagem	Causa	Solução
`No qualifying bean of type ...`	Classe sem `@Service`/`@Component`, ou fora do pacote base	Anote e/ou mova para dentro do pacote base

Mensagem	Causa	Solução
`Port 8080 was already in use`	Outra aplicação usa a porta	Pare a outra ou mude `server.port`
`HttpMessageNotReadableException` (400)	JSON malformado ou tipos incompatíveis	Revise o JSON e o DTO
`404` em endpoint que "deveria existir"	URL, verbo ou `@RequestMapping` incorretos	Confira o caminho completo
`Circular dependency`	Beans dependendo um do outro	Reestruture responsabilidades

Resumo do capítulo

- Spring Boot = Spring + autoconfiguração + servidor embutido + starters.
- **IoC**: o contêiner cria e gerencia os objetos. **DI**: ele os injeta (prefira injeção por construtor).
- Estereótipos (`@Component`, `@Service`, `@Repository`, `@RestController`) registram beans; beans são singletons por padrão.
- Controller recebe/responde HTTP; Service tem a regra de negócio; Repository acessa dados.
- `@GetMapping`, `@PostMapping` etc. + `@PathVariable`, `@RequestParam`, `@RequestBody` + `ResponseEntity`.
- Configuração em `application.properties/yml`, com Profiles por ambiente e segredos em variáveis de ambiente.

Exercícios teóricos

1. Explique IoC e DI com suas palavras. Que problema resolvem?
2. Por que a injeção por construtor é preferível à injeção por campo?
3. Qual a diferença entre `@Component`, `@Service`, `@Repository` e `@Controller`?
4. O que faz `@SpringBootApplication`? Por que as classes precisam ficar abaixo do pacote principal?
5. O que é um bean singleton? Que cuidado isso exige?
6. Qual a diferença entre `@PathVariable` e `@RequestParam`?
7. Por que o Controller não deve conter regra de negócio?
8. O que é um starter e o que é autoconfiguração?
9. Para que servem os Profiles? Como proteger uma senha de banco?
10. Descreva o caminho completo de uma requisição `POST` dentro do Spring MVC.

Exercícios práticos

1. Gere um projeto no Spring Initializr, rode e crie um endpoint `GET /ola` que devolve "Olá, Spring!".
2. Crie `GET /ola/{nome}` e `GET /soma?a=1&b=2` usando `@PathVariable` e `@RequestParam`.
3. Crie `POST /echo` que recebe um JSON `{"mensagem":"..."}` e devolve o mesmo objeto (use um `record`).
4. Monte a trinca Controller → Service → Repository para uma entidade `Tarefa` (em memória) com CRUD completo.
5. Faça o `POST` devolver `201` com o header `Location`, e o `DELETE` devolver `204`.
6. Crie propriedades `app.nome` e `app.versao` no `application.properties` e exponha-as em `GET /info` via `@ConfigurationProperties`.

7. Crie `application-dev.properties` e `application-prod.properties` com portas diferentes e ative cada profile.
8. Remova `@Service` de uma classe e leia a mensagem de erro de inicialização. Depois conserte.

Projeto principal: Sistema Biblioteca

Transforme a Biblioteca em uma **API Spring Boot em memória**, seguindo o contrato do capítulo 5:

1. Crie o projeto `biblioteca-api` (Spring Initializr) com pacotes `controller`, `service`, `repository`, `model`, `exception`, `dto`.
2. Reaproveite o domínio do projeto console: `Livro`, `Usuario`, `Emprestimo` e as exceções próprias.
3. Implemente os endpoints de **livros** e **usuários** (CRUD) e os de **empréstimo/devolução**.
4. Repositórios em memória (`Map`) **com interface e nomes** `save/findById/findAll/deleteById`, como na seção 6.5, e DTOs mínimos como na 6.3. Aplique a nota de transição da seção 6.0 ao modelo.
5. Configure `biblioteca.prazo-dias`, `biblioteca.multa-por-dia` e `biblioteca.limite-emprestimos` no `application.yml` com `@ConfigurationProperties`.
6. Teste todos os endpoints no Postman e exporte a coleção para `docs/postman/`.
7. Por enquanto, exceções viram `500`; no capítulo 11 você as mapeará para `404` e `409`.

Mini-projeto: API de Tarefas (To-Do)

Em outro repositório, construa uma API REST completa e simples:

- Entidade `Tarefa(id, titulo, descricao, concluida, prazo, prioridade)` com `enum Prioridade`.
- Endpoints: CRUD, `PATCH /tarefas/{id}/concluir`, `GET /tarefas?concluida=false&prioridade=ALTA`.
- Camadas bem separadas e injeção por construtor.
- Dois profiles (`dev` com logs detalhados, `prod` com logs enxutos).
- README com a tabela de endpoints e exemplos de `curl`.
- Desafio: endpoint `GET /tarefas/atrasadas` que usa Streams para filtrar por `prazo`.

7. JPA e Hibernate

Explicação geral

Até aqui, seus dados viviam em um `Map` na memória: ao reiniciar a aplicação, tudo se perdia. Para persistir em um banco relacional, o jeito "antigo" era escrever SQL e converter `ResultSet` em objetos na mão (JDBC). Isso é repetitivo e propenso a erros. A solução é o **ORM** (*Object-Relational Mapping*): você mapeia classes Java para tabelas, e o framework gera o SQL.

- **JPA** (Jakarta Persistence) é a **especificação**: um conjunto de interfaces e anotações (`@Entity`, `@Id`...).
- **Hibernate** é a **implementação** mais usada da JPA.
- **Spring Data JPA** é a camada do Spring que escreve os repositórios para você, a partir de uma interface.

```
Service → Repository (Spring Data JPA) → JPA/Hibernate → JDBC → PostgreSQL
```

Seu conhecimento de SQL continua sendo essencial: o Hibernate gera SQL, e quem não entende o que ele gera cai em problemas de desempenho (como o N+1, visto adiante).

7.1 Dependências e configuração

No `pom.xml` (sem versão, o parent do Spring Boot cuida disso):

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-data-jpa</artifactId>
</dependency>
<dependency>
  <groupId>org.postgresql</groupId>
  <artifactId>postgresql</artifactId>
  <scope>runtime</scope>
</dependency>
```

Neste capítulo, comece com o banco em memória **H2** (`com.h2database:h2`, escopo `runtime`): você não precisa instalar nada, e o Hibernate cria as tabelas a partir das entidades. No capítulo 8 você troca para PostgreSQL e passa a criar o esquema com migrações.

```
# Capítulo 7 (H2 em memória): o Hibernate cria e apaga as tabelas
spring:
  datasource:
    url: jdbc:h2:mem:biblioteca
  jpa:
    hibernate:
      ddl-auto: create-drop      # só para estudo; veja a tabela abaixo
    show-sql: true
    properties:
      hibernate:
        format_sql: true

# A PARTIR DO CAPÍTULO 8 (PostgreSQL + Flyway): o esquema vem das migrações
spring:
  datasource:
    url: jdbc:postgresql://localhost:5432/biblioteca
    username: ${DB_USER:postgres}
    password: ${DB_PASSWORD:postgres}
  jpa:
    hibernate:
      ddl-auto: validate
```

> **ATENÇÃO** Com `validate` e **sem** esquema criado (por Flyway ou script), a aplicação falha ao subir com "missing table". Por isso o capítulo 7 usa `create-drop` e só o capítulo 8 passa para `validate`.

ddl-auto	Efeito	Uso
`none`	Não mexe no esquema	Produção
`validate`	Confere se as tabelas batem com as entidades	Produção (com migrações)
`update`	Altera o esquema automaticamente	Só estudo
`create` / `create-drop`	Recria o esquema a cada execução	Testes e protótipos

> **ATENÇÃO** Nunca use `update` ou `create` em produção: você pode perder dados. O caminho profissional é versionar o esquema com **migrações** (Flyway), visto no capítulo 8.

7.2 Entidades e mapeamento básico

Uma **entidade** é uma classe ligada a uma tabela.

```
@Entity
@Table(name = "livro")
public class Livro {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false, length = 200)
    private String titulo;

    @Column(nullable = false, length = 120)
    private String autor;

    private Integer ano;

    @Column(nullable = false)
    private boolean disponivel = true;

    protected Livro() { }          // construtor sem argumentos exigido pela JPA

    public Livro(String titulo, String autor, Integer ano) {
        this.titulo = titulo;
        this.autor = autor;
        this.ano = ano;
    }

    // getters e regras de negócio (emprestar, devolver)
}
```

Anotação	Função
`@Entity`	Marca a classe como entidade
`@Table(name=...)`	Nome da tabela (opcional)
`@Id`	Chave primária
`@GeneratedValue`	Estratégia de geração do id
`@Column`	Nome, tamanho, obrigatoriedade, unicidade
`@Enumerated(EnumType.STRING)`	Grava o enum como texto
`@Transient`	Campo não persistido

Anotação	Função
`@Temporal`	Só para `Date` antigo; com `java.time` não precisa

Estratégias de geração de id

- `IDENTITY`: o banco gera (`SERIAL`/`IDENTITY`). Simples e comum no PostgreSQL.
- `SEQUENCE`: usa uma sequence do banco; permite *batch* eficiente.
- `UUID`: identificadores únicos globais.

> **ATENÇÃO** Use `EnumType.STRING`, nunca `ORDINAL` (o padrão): com `ORDINAL`, inserir um valor no meio do enum corrompe os dados já gravados.

Regras para entidades

- Construtor sem argumentos (pode ser `protected`).
- Classe não `final`, e atributos não `final` (o Hibernate precisa criar proxies).
- Implemente `equals`/`hashCode` com cuidado: baseado no id (tratando `null` antes de persistir) ou em uma chave de negócio.
- **Records não servem como entidades**; use-os como DTOs.

7.3 Relacionamentos

Em SQL, relacionamentos são chaves estrangeiras. Em JPA, são referências entre objetos.

`@ManyToOne` e `@OneToMany` (o mais comum)

Muitos empréstimos pertencem a um usuário; um usuário tem muitos empréstimos.

```
@Entity
public class Emprestimo {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @ManyToOne(fetch = FetchType.LAZY, optional = false)
    @JoinColumn(name = "usuario_id")            // coluna FK na tabela emprestimo
    private Usuario usuario;

    @ManyToOne(fetch = FetchType.LAZY, optional = false)
    @JoinColumn(name = "livro_id")
    private Livro livro;

    private LocalDate dataEmprestimo;
    private LocalDate dataDevolucao;

    @Enumerated(EnumType.STRING)
    private StatusEmprestimo status;
}

@Entity
public class Usuario {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String nome;

    @OneToMany(mappedBy = "usuario")            // lado inverso (não tem a FK)
    private List<Emprestimo> emprestimos = new ArrayList<>();
}
```

Conceitos importantes:

- **Lado dono**: quem tem a chave estrangeira (`@ManyToOne` + `@JoinColumn`). O `mappedBy` indica o lado inverso.
- **Unidirecional x bidirecional**: só mapeie os dois lados se realmente precisar navegar nos dois sentidos; é menos código e menos armadilhas.
- **Mantenha os dois lados sincronizados** quando o relacionamento for bidirecional, com um método auxiliar. O que vale para o banco é o lado dono, mas o objeto em memória fica inconsistente se você atualizar só um lado:

```
// dentro de Usuario
public void adicionarEmprestimo(Emprestimo e) {
    emprestimos.add(e);
    e.setUsuario(this);
}
```

`@ManyToMany`

Ex.: um livro tem vários autores, um autor tem vários livros (tabela de junção).

```
@Entity
public class Livro {
    @ManyToMany
    @JoinTable(name = "livro_autor",
               joinColumns = @JoinColumn(name = "livro_id"),
               inverseJoinColumns = @JoinColumn(name = "autor_id"))
    private Set<Autor> autores = new HashSet<>();
}

@Entity
public class Autor {
    @ManyToMany(mappedBy = "autores")
    private Set<Livro> livros = new HashSet<>();
}
```

Se a relação precisar de atributos próprios (data, quantidade), crie uma **entidade de junção** (como `Emprestimo`, que liga `Usuario` e `Livro` e tem datas) em vez de `@ManyToMany`.

`@OneToOne`

```
@Entity
public class Usuario {
    @OneToOne(cascade = CascadeType.ALL, orphanRemoval = true)
    @JoinColumn(name = "carteirinha_id")
    private Carteirinha carteirinha;
}
```

Cascade e orphanRemoval

`cascade` propaga operações do pai para os filhos (`PERSIST`, `MERGE`, `REMOVE`, `ALL`). `orphanRemoval = true` apaga filhos removidos da coleção. Use com critério: `CascadeType.REMOVE` em relações muito amplas pode apagar dados em cadeia.

7.4 Lazy x Eager e o problema N+1

- **LAZY**: a associação só é carregada do banco quando você a acessa.
- **EAGER**: carregada junto com a entidade, sempre.

Padrões: `@ManyToOne` e `@OneToOne` são **EAGER**; `@OneToMany` e `@ManyToMany` são **LAZY**. A boa prática é declarar tudo como `LAZY` e buscar o que precisa de forma explícita.

O problema N+1

Imagine listar 100 empréstimos e, para cada um, mostrar o nome do usuário (associação lazy). Se os 100 empréstimos forem de 100 usuários **diferentes**:

```
1 consulta:  SELECT * FROM emprestimo              → 100 linhas
+ 100 consultas: SELECT * FROM usuario WHERE id=?  (uma por usuário diferente)
= 101 consultas
```

Isso é o **N+1**: uma consulta para a lista, mais uma para cada associação. (O número exato depende dos dados: o Hibernate guarda na sessão os usuários já carregados, então usuários repetidos não geram nova consulta. Mesmo assim, o crescimento acompanha o tamanho da lista.) Em volumes grandes derruba o desempenho. Soluções:

```
// JOIN FETCH traz tudo em uma consulta
@Query("SELECT e FROM Emprestimo e JOIN FETCH e.usuario JOIN FETCH e.livro")
List<Emprestimo> listarComDetalhes();

// ou @EntityGraph
@EntityGraph(attributePaths = {"usuario", "livro"})
List<Emprestimo> findByStatus(StatusEmprestimo status);
```

> **DICA** Ative `spring.jpa.show-sql=true` durante o desenvolvimento e **conte as consultas**. É assim que se descobre o N+1.

LazyInitializationException

Acontece quando você acessa uma associação lazy **fora** de uma transação/sessão (por exemplo, no Controller, depois que o Service terminou). Solução: busque o necessário no Service (com `JOIN FETCH`) e devolva um **DTO**, não a entidade.

7.5 Spring Data JPA: repositories

Basta declarar uma interface; o Spring gera a implementação em tempo de execução:

```
public interface LivroRepository extends JpaRepository<Livro, Long> {
}
```

Isso já dá: `save`, `findById`, `findAll`, `deleteById`, `count`, `existsById` e paginação.

```
Livro salvo = repository.save(new Livro("Java Efetivo", "Bloch", 2018));
Optional<Livro> achado = repository.findById(1L);
List<Livro> todos = repository.findAll();
repository.deleteById(1L);
```

Query methods (consultas pelo nome do método)

```
public interface LivroRepository extends JpaRepository<Livro, Long> {

    List<Livro> findByAutor(String autor);
    List<Livro> findByDisponivelTrue();
    List<Livro> findByTituloContainingIgnoreCase(String trecho);
    List<Livro> findByAnoBetween(int de, int ate);
    Optional<Livro> findByTitulo(String titulo);
    boolean existsByTitulo(String titulo);
    long countByDisponivelTrue();
    List<Livro> findByAutorOrderByAnoDesc(String autor);
}
```

O Spring interpreta `findBy` + propriedade + palavra-chave (`Containing`, `Between`, `GreaterThan`, `IgnoreCase`, `OrderBy`...).

JPQL e `@Query`

JPQL é uma linguagem parecida com SQL, mas escrita sobre **entidades e atributos**, não tabelas e colunas.

```
@Query("SELECT l FROM Livro l WHERE l.autor = :autor AND l.disponivel = true")
List<Livro> disponiveisDoAutor(@Param("autor") String autor);

// Passe o enum como parâmetro, em vez de comparar com um literal de texto
@Query("SELECT e FROM Emprestimo e WHERE e.usuario.id = :uid AND e.status = :status")
List<Emprestimo> findDoUsuarioComStatus(@Param("uid") Long uid,
                                        @Param("status") StatusEmprestimo status);
// uso: repo.findDoUsuarioComStatus(5L, StatusEmprestimo.ATIVO);

// SQL nativo, quando necessário
@Query(value = "SELECT * FROM livro WHERE ano > :ano", nativeQuery = true)
List<Livro> recentes(@Param("ano") int ano);
```

Use `:parametro` nomeado. **Nunca concatene strings** na consulta (risco de SQL Injection).

Paginação e ordenação

```
Page<Livro> pagina = repository.findAll(
        PageRequest.of(0, 10, Sort.by("titulo").ascending()));

pagina.getContent();        // itens da página
pagina.getTotalElements();  // total geral
pagina.getTotalPages();

Page<Livro> findByAutor(String autor, Pageable pageable);   // também em query methods
```

No Controller, o Spring aceita `Pageable` direto: `GET /livros?page=0&size=10&sort=titulo,asc`.

7.6 Transações

Uma **transação** garante que um conjunto de operações seja **atômico**: ou tudo acontece, ou nada. É o "A" do ACID (Atomicidade, Consistência, Isolamento, Durabilidade) que você viu em Banco de Dados.

```
@Service
public class EmprestimoService {

    @Transactional                      // se algo lançar exceção, faz rollback de tudo
    public Emprestimo emprestar(Long usuarioId, Long livroId) {
        Usuario u = usuarios.findById(usuarioId).orElseThrow(...);
        Livro l = livros.findById(livroId).orElseThrow(...);
        l.emprestar();                  // altera o estado
        return emprestimos.save(new Emprestimo(u, l));
    }                                   // commit aqui
}
```

Pontos-chave:

- Dentro de uma transação, entidades gerenciadas têm suas alterações **detectadas automaticamente** (*dirty checking*): mudou o objeto, o Hibernate faz o `UPDATE` no commit, mesmo sem chamar `save`.
- Por padrão, o *rollback* ocorre para `RuntimeException` e `Error`, **não** para exceções *checked*. Outra razão para preferir exceções de domínio unchecked.
- `@Transactional(readOnly = true)` em consultas permite otimizações.
- Coloque `@Transactional` no **Service**, nunca no Controller.
- A anotação funciona via proxy: uma chamada de método **dentro da mesma classe** (`this.outroMetodo()`) não passa pelo proxy e ignora a transação.

Ciclo de vida de uma entidade

Estado	Significado
Transient	`new Livro()` ainda não ligado ao banco
Managed (persistent)	Gerenciado pelo contexto de persistência; mudanças são rastreadas
Detached	Já foi gerenciado, mas a sessão fechou
Removed	Marcado para exclusão

7.7 Auditoria e boas práticas

- Campos de auditoria: `@CreatedDate`, `@LastModifiedDate`. Exigem **três** passos: `@EnableJpaAuditing` em uma classe de configuração, `@EntityListeners(AuditingEntityListener.class)` na entidade e os campos anotados.
- Controle de concorrência otimista: campo `@Version` evita que duas pessoas sobrescrevam a mesma linha sem perceber (ex.: dois empréstimos simultâneos do mesmo livro).
- Prefira **lazy** + buscas explícitas; evite expor entidades no Controller (use DTOs).
- Nunca confie em `ddl-auto=update` em produção.
- Cuidado com `toString`/`equals` incluindo coleções lazy: podem disparar consultas ou laços infinitos em relações bidirecionais.

Resumo do capítulo

- **JPA** é a especificação, **Hibernate** a implementação, **Spring Data JPA** gera repositórios a partir de interfaces.
- Entidades: `@Entity`, `@Id`, `@GeneratedValue`, `@Column`; use `EnumType.STRING`.
- Relacionamentos: `@ManyToOne` (lado dono, com FK), `@OneToMany(mappedBy)`, `@ManyToMany` (ou entidade de junção), `@OneToOne`.
- Prefira **LAZY**; evite o **N+1** com `JOIN FETCH` ou `@EntityGraph`.
- Repositórios: `JpaRepository`, query methods, `@Query` (JPQL) e paginação com `Pageable`.
- `@Transactional` no Service garante atomicidade; rollback por padrão para exceções unchecked.

Exercícios teóricos

1. Qual a diferença entre JPA, Hibernate e Spring Data JPA?
2. O que é um ORM? Quais vantagens e desvantagens em relação ao JDBC puro?
3. Por que usar `EnumType.STRING` em vez de `ORDINAL`?
4. O que significa "lado dono" de um relacionamento? Para que serve o `mappedBy`?
5. Quando usar entidade de junção em vez de `@ManyToMany`?
6. Explique LAZY e EAGER. Qual a prática recomendada?
7. O que é o problema N+1 e como resolvê-lo?
8. O que causa `LazyInitializationException`?
9. O que faz `@Transactional`? Quando ocorre rollback por padrão?
10. O que é *dirty checking*?
11. Por que records não são adequados como entidades?
12. Por que `ddl-auto=update` é perigoso em produção?

Exercícios práticos

1. Mapeie a entidade `Tarefa` (do mini-projeto do capítulo 6) com JPA e use H2 para persistir.
2. Crie 6 query methods em um repository (ex.: por título contendo trecho, por intervalo de datas, contagem por status).
3. Escreva duas consultas com `@Query` (JPQL), uma delas com `JOIN`.
4. Implemente paginação e ordenação em um endpoint (`?page=0&size=5&sort=titulo,asc`).
5. Crie `Categoria` (1) → `Tarefa` (N) com `@ManyToOne` e liste as tarefas de uma categoria.
6. Reproduza o N+1: crie 20 tarefas em categorias distintas, liste e **conte as consultas SQL**. Corrija com `JOIN FETCH` e compare.
7. Provoque um `LazyInitializationException` de propósito e corrija retornando um DTO.
8. Escreva um método `@Transactional` que faz duas operações e lança exceção na segunda. Confirme o rollback.

Projeto principal: Sistema Biblioteca

1. Adicione `spring-boot-starter-data-jpa` e o H2, com `ddl-auto: create-drop` (PostgreSQL e `validate` ficam para o capítulo 8).
2. Converta `Livro`, `Usuario` e `Emprestimo` em **entidades JPA**: ids, `@Column`, `@Enumerated(STRING)` para o status.
3. Relacionamentos: `Emprestimo` `@ManyToOne` para `Usuario` e `Livro` (lazy); `Usuario` `@OneToMany(mappedBy)` para empréstimos.
4. **Desafio opcional:** crie uma entidade `Autor` e relacione com `Livro` (`@ManyToMany`), justificando a escolha no README. Atenção: o esquema do capítulo 8 usa a coluna `autor` (texto) em `livro`; se você adotar `Autor` agora, o capítulo 13 traz a migração que faz essa troca. Se preferir o caminho mais simples, mantenha `String autor` até o capítulo 13.
5. Troque os repositórios em memória por interfaces `JpaRepository`, **sem alterar a lógica do Service**.
6. Adicione query methods: buscar livros por título/autor, usuários por e-mail, empréstimos ativos de um usuário.
7. Marque `emprestar` e `devolver` como `@Transactional` e garanta que o limite de empréstimos seja validado dentro da transação.
8. Adicione paginação em `GET /livros`. Verifique o SQL gerado e elimine qualquer N+1 na listagem de empréstimos.

Mini-projeto: API de Cursos e Alunos

Em outro repositório, modele uma escola:

- Entidades `Aluno`, `Curso`, `Professor` e `Matricula` (entidade de junção com `dataMatricula` e `nota`).
- Relacionamentos: `Professor` 1–N `Curso`; `Aluno` N–N `Curso` via `Matricula`.
- Endpoints: matricular aluno, lançar nota, listar alunos de um curso (paginado), média de notas de um curso (JPQL com `AVG`).
- Regras: aluno não pode se matricular duas vezes no mesmo curso; curso tem limite de vagas.
- Entregue o `README` com um diagrama das entidades e a contagem de consultas SQL das principais operações.

8. PostgreSQL com Spring Boot

Explicação geral

PostgreSQL é um banco de dados relacional open source, robusto e muito usado em empresas. Você já estudou SQL em Banco de Dados 1 e 2: aqui o foco é **aplicar esse conhecimento dentro do back-end**, ligando a aplicação Spring Boot a um PostgreSQL real e tratando o banco como parte do projeto (esquema versionado, índices, conexões, dados de teste).

Duas mudanças de mentalidade em relação ao ambiente acadêmico:

- O esquema do banco **evolui junto com o código** e precisa ser versionado (migrações), assim como o Java.
- A aplicação **não cria tabelas "no braço"**: o esquema é responsabilidade das migrações, e o Hibernate apenas as valida.

8.1 Subindo o PostgreSQL

A forma mais prática é com Docker (vamos aprofundar no capítulo 10):

```
docker run --name pg-biblioteca \
  -e POSTGRES_DB=biblioteca \
  -e POSTGRES_USER=postgres \
  -e POSTGRES_PASSWORD=postgres \
  -p 5432:5432 \
  -v pg-biblioteca-data:/var/lib/postgresql/data \
  -d postgres:16
```

Parte	Significado
`-e POSTGRES_*`	Variáveis que criam banco, usuário e senha
`-p 5432:5432`	Mapeia a porta do contêiner para a máquina
`-v ...:/var/lib/postgresql/data`	**Volume**: preserva os dados se o contêiner for recriado
`postgres:16`	Imagem e versão

Conecte com `psql` ou com um cliente gráfico (DBeaver, pgAdmin):

```
docker exec -it pg-biblioteca psql -U postgres -d biblioteca
```

Comandos úteis do `psql`: `\l` (bancos), `\c biblioteca` (conectar), `\dt` (tabelas), `\d livro` (descrever tabela), `\q` (sair).

8.2 Conectando a aplicação

Dependência (já vista no capítulo 7) e configuração:

```
spring:
  datasource:
    url: jdbc:postgresql://localhost:5432/biblioteca
    username: ${DB_USER:postgres}
    password: ${DB_PASSWORD:postgres}
  jpa:
    hibernate:
      ddl-auto: validate
    open-in-view: false
```

A URL JDBC segue o padrão `jdbc:postgresql://host:porta/banco`. Valores sensíveis vêm de **variáveis de ambiente**, com um padrão só para desenvolvimento local.

> **DICA** Desative `spring.jpa.open-in-view` (o padrão é `true`). Ele mantém a sessão aberta durante toda a requisição e esconde problemas de N+1.

Pool de conexões (HikariCP)

Abrir uma conexão com o banco é caro. O Spring Boot usa o **HikariCP**, um *pool* que reaproveita conexões:

```
spring:
  datasource:
    hikari:
      maximum-pool-size: 10
      connection-timeout: 30000
```

Se o pool esvaziar (todas as conexões ocupadas), novas requisições esperam e dão erro de timeout. Conexões presas por consultas lentas são uma causa clássica de lentidão.

8.3 Modelando o esquema para a aplicação

Revise o que você viu em Banco de Dados, agora com olhar de back-end:

Conceito	Na prática
Chave primária	`BIGSERIAL` ou `BIGINT GENERATED ... AS IDENTITY` (casa com `GenerationType.IDENTITY`)
Chave estrangeira	Sempre declare (`REFERENCES`), garante integridade
`NOT NULL`	Reflita no `@Column(nullable = false)`
`UNIQUE`	Ex.: e-mail do usuário; trate a violação como `409 Conflict`
`CHECK`	Regra no banco (ex.: `ano > 0`) como última linha de defesa
Tipos	`VARCHAR(n)`, `TEXT`, `INTEGER`, `NUMERIC(10,2)` para dinheiro, `DATE`, `TIMESTAMP`, `BOOLEAN`
Índices	Acelere colunas usadas em `WHERE`/`JOIN`/`ORDER BY`

> **ATENÇÃO** Nunca use `double`/`float` para dinheiro. Use `BigDecimal` no Java e `NUMERIC(10,2)` no banco.

Exemplo: esquema da biblioteca

```
CREATE TABLE usuario (
    id      BIGSERIAL PRIMARY KEY,
    nome    VARCHAR(120) NOT NULL,
    email   VARCHAR(160) NOT NULL UNIQUE
);

CREATE TABLE livro (
    id          BIGSERIAL PRIMARY KEY,
    titulo      VARCHAR(200) NOT NULL,
    autor       VARCHAR(120) NOT NULL,          -- texto simples; vira tabela "autor" no capítulo 13
    ano         INTEGER CHECK (ano > 0),
    disponivel  BOOLEAN NOT NULL DEFAULT TRUE,
    version     BIGINT NOT NULL DEFAULT 0       -- usado pelo @Version (bloqueio otimista)
);

CREATE TABLE emprestimo (
    id                BIGSERIAL PRIMARY KEY,
    usuario_id        BIGINT NOT NULL REFERENCES usuario(id),
    livro_id          BIGINT NOT NULL REFERENCES livro(id),
    data_emprestimo   DATE NOT NULL,
```

```
    data_devolucao    DATE,
    status            VARCHAR(20) NOT NULL
);

CREATE INDEX idx_emprestimo_usuario ON emprestimo(usuario_id);
CREATE INDEX idx_emprestimo_status  ON emprestimo(status);
```

Repare que `status` é `VARCHAR`: combina com `@Enumerated(EnumType.STRING)`. Este esquema corresponde **exatamente** às entidades do capítulo 7 (`Livro` com `String autor`); com `ddl-auto: validate`, qualquer diferença entre entidade e tabela impede a aplicação de subir. A coluna `version` já nasce aqui para o `@Version` que você adiciona mais adiante neste capítulo.

8.4 Migrações com Flyway

Rodar scripts SQL à mão em cada máquina e em produção leva a esquemas diferentes e a erros. **Flyway** aplica automaticamente, na ordem, scripts versionados e guarda o histórico em uma tabela (`flyway_schema_history`).

Dependências:

```
<dependency>
  <groupId>org.flywaydb</groupId>
  <artifactId>flyway-core</artifactId>
</dependency>
<dependency>
  <groupId>org.flywaydb</groupId>
  <artifactId>flyway-database-postgresql</artifactId>
</dependency>
```

Scripts em `src/main/resources/db/migration/`, com o padrão de nome `V<versão>__<descrição>.sql` (dois underscores):

```
db/migration/
├── V1__criar_tabelas_basicas.sql
├── V2__criar_emprestimo.sql
└── V3__adicionar_indice_email.sql
```

Ao iniciar a aplicação, o Flyway executa as migrações pendentes. Regras de ouro:

- **Nunca edite** uma migração já aplicada (o Flyway detecta alteração pelo *checksum* e recusa). Para mudar, crie uma nova versão (`V4__...`).
- Uma migração = uma mudança coesa.
- Com Flyway cuidando do esquema, use `spring.jpa.hibernate.ddl-auto=validate`.

Exemplo de evolução:

```
-- V4__adicionar_telefone_usuario.sql
ALTER TABLE usuario ADD COLUMN telefone VARCHAR(20);
```

8.5 SQL que todo back-end precisa dominar

Mesmo com JPA, você precisa ler e escrever SQL com segurança. Revise com a Biblioteca:

```
-- Empréstimos ativos com nome do usuário e título do livro
SELECT e.id, u.nome, l.titulo, e.data_emprestimo
FROM emprestimo e
JOIN usuario u ON u.id = e.usuario_id
JOIN livro   l ON l.id = e.livro_id
WHERE e.status = 'ATIVO'
ORDER BY e.data_emprestimo;

-- Quantidade de empréstimos por usuário (mais de 2)
-- Agrupe pelo ID (únicos), não pelo nome: dois usuários homônimos seriam somados juntos
SELECT u.id, u.nome, COUNT(*) AS total
FROM emprestimo e
```

```
JOIN usuario u ON u.id = e.usuario_id
GROUP BY u.id, u.nome
HAVING COUNT(*) > 2
ORDER BY total DESC;

-- Livros que nunca foram emprestados
SELECT l.*
FROM livro l
LEFT JOIN emprestimo e ON e.livro_id = l.id
WHERE e.id IS NULL;

-- Livros mais emprestados (top 5)
SELECT l.id, l.titulo, COUNT(*) AS vezes
FROM emprestimo e JOIN livro l ON l.id = e.livro_id
GROUP BY l.id, l.titulo
ORDER BY vezes DESC
LIMIT 5;
```

Índices e `EXPLAIN`

Um índice funciona como o índice remissivo de um livro: acelera leituras, mas deixa escritas um pouco mais lentas e ocupa espaço. Crie índices em colunas de FK e em colunas muito filtradas.

```
EXPLAIN ANALYZE
SELECT * FROM emprestimo WHERE usuario_id = 5;
```

Se o plano mostrar `Seq Scan` (varredura da tabela inteira) em uma tabela grande, falta um índice. `Index Scan` indica uso de índice.

Transações e concorrência no banco

O PostgreSQL garante ACID. Para evitar duas pessoas emprestando o mesmo livro ao mesmo tempo, você tem opções:

- **Bloqueio otimista**: campo `@Version` na entidade; o segundo a salvar falha. Com Spring Data, a exceção que chega à sua aplicação é a do Spring, `ObjectOptimisticLockingFailureException` (subtipo de `OptimisticLockingFailureException`), e **não** a `jakarta.persistence.OptimisticLockException`. Trate a do Spring (no capítulo 11, mapeie para `409`).
- **Bloqueio pessimista**: `@Lock(LockModeType.PESSIMISTIC_WRITE)` em uma consulta do repositório (`SELECT ... FOR UPDATE`).
- **Restrição única parcial** (recurso do PostgreSQL): impede, no próprio banco, mais de um empréstimo ativo do mesmo livro.

```
CREATE UNIQUE INDEX uq_livro_emprestimo_ativo
    ON emprestimo(livro_id) WHERE status = 'ATIVO';
```

> **ATENÇÃO** Este índice só faz sentido enquanto cada livro for **um único exemplar** (modelo atual, com `disponivel` booleano). No capítulo 13, quando o livro passa a ter vários exemplares (`quantidade_disponivel`), este índice **impediria** emprestar o segundo exemplar: a migração do capítulo 13 o remove e o substitui por `CHECK (quantidade_disponivel >= 0)` somado ao `@Version`.

8.6 Dados iniciais e acesso seguro

- Dados de apoio (ex.: categorias fixas) podem entrar por uma migração (`V5__dados_iniciais.sql`).
- Dados de demonstração para desenvolvimento: use um profile `dev` ou um `CommandLineRunner`.
- **Nunca** monte SQL concatenando entradas do usuário. Use parâmetros (`:nome`, `?`). Isso impede **SQL Injection**.

- Em produção, crie um usuário do banco com **permissões mínimas** (não use o superusuário `postgres`).
- Faça **backup**: `pg_dump -U postgres biblioteca > backup.sql` e restaure com `psql`.

8.7 Problemas comuns

Sintoma	Causa provável	Solução
`Connection refused`	Banco não está rodando ou porta errada	`docker ps`, confira a porta e a URL
`password authentication failed`	Usuário/senha divergentes	Confira variáveis de ambiente
`Schema-validation: missing table`	Esquema não criado	Verifique as migrações Flyway
`Migration checksum mismatch`	Migração já aplicada foi editada	Reverta a edição e crie nova versão
Banco não sobe após trocar `postgres:16` por outra versão major, com o mesmo volume	Dados gravados por uma versão major não são lidos por outra, e imagens mais novas podem mudar o caminho do diretório de dados	Leia a documentação da imagem antes de trocar a major; em estudo, recrie o volume
`duplicate key value violates unique constraint`	Violação de `UNIQUE`	Trate e devolva `409`
Aplicação lenta ao listar	N+1 ou falta de índice	`show-sql`, `EXPLAIN ANALYZE`

Resumo do capítulo

- PostgreSQL via Docker (com **volume** para persistir dados); conexão por `jdbc:postgresql://host:porta/banco`.
- Credenciais em **variáveis de ambiente**; pool de conexões (HikariCP).
- Esquema versionado com **Flyway** (`V1__...sql`); nunca edite migrações já aplicadas; Hibernate em `validate`.
- Reflita restrições (`NOT NULL`, `UNIQUE`, `FK`, `CHECK`) no banco e no mapeamento.
- Domine JOIN, GROUP BY/HAVING, subconsultas, índices e `EXPLAIN ANALYZE`.
- Concorrência: `@Version` (otimista), bloqueio pessimista ou índice único parcial.
- Dinheiro: `NUMERIC` + `BigDecimal`. Nunca concatene SQL.

Exercícios teóricos

1. Por que o esquema do banco deve ser versionado junto com o código?
2. Como o Flyway sabe quais migrações já foram executadas? O que acontece se você edita uma já aplicada?
3. Por que usar `ddl-auto=validate` em vez de `update` quando há Flyway?
4. O que é um pool de conexões e por que ele existe?
5. Para que serve um volume no Docker em um contêiner de banco?
6. Qual a diferença entre bloqueio otimista e pessimista? Dê um exemplo para a Biblioteca.
7. Como um índice acelera consultas? Qual o custo?

8. Por que `NUMERIC`/`BigDecimal` para dinheiro e não `double`?
9. O que é SQL Injection e como evitá-lo?
10. O que significa `Seq Scan` no `EXPLAIN`?

Exercícios práticos

1. Suba um PostgreSQL com Docker (com volume), conecte pelo `psql` e pelo DBeaver.
2. Crie, com SQL, as tabelas `aluno`, `curso` e `matricula` com PKs, FKs, `UNIQUE` e `CHECK`.
3. Escreva 5 consultas: um `JOIN` triplo, um `GROUP BY` com `HAVING`, um `LEFT JOIN` para achar "sem relação", uma subconsulta e um `ORDER BY ... LIMIT`.
4. Crie uma tabela com 100 mil linhas (`generate_series`), meça uma busca com `EXPLAIN ANALYZE`, crie um índice e meça de novo.
5. Configure Flyway em um projeto Spring, crie `V1` e `V2` e veja a tabela `flyway_schema_history`.
6. Edite uma migração aplicada de propósito, leia o erro e conserte do jeito certo.
7. Faça um backup com `pg_dump`, apague o banco e restaure.
8. Provoque uma violação de `UNIQUE` pela API e veja a exceção no log.

Projeto principal: Sistema Biblioteca

1. Suba o PostgreSQL com Docker e configure o `application.yml` (variáveis de ambiente para usuário e senha).
2. Adicione **Flyway** e transforme o esquema da seção 8.3 em migrações `V1` a `V3`.
3. Mude `ddl-auto` para `validate` e `spring.jpa.open-in-view` para `false`.
4. Crie o **índice único parcial** que impede dois empréstimos ativos do mesmo livro e teste com duas requisições simultâneas (lembre: ele será removido no capítulo 13, quando surgirem vários exemplares).
5. Adicione `@Version private Long version;` em `Livro` (a coluna já existe no esquema) e trate `OptimisticLockingFailureException`, que o Spring lança.
6. Crie uma migração `V4__dados_demo.sql` (ou um runner em profile `dev`) com 20 livros, 5 autores e 5 usuários.
7. Escreva três consultas de relatório em SQL (livros mais emprestados, usuários com atraso, livros nunca emprestados) e exponha-as em `GET /api/v1/relatorios/...` com `@Query`.
8. Documente no README como subir o banco e rodar as migrações.

Mini-projeto: Controle Financeiro Pessoal (somente banco + JDBC/JPA)

Em outro repositório:

- Modele `conta`, `categoria` e `lancamento` (receita/despesa) com `NUMERIC(12,2)`, FKs e `CHECK (valor > 0)`.
- Esquema criado por migrações Flyway; dados de exemplo em uma migração.
- Endpoints: lançar receita/despesa, saldo por conta, total por categoria no mês (`GROUP BY` por mês), extrato paginado.
- Transferência entre contas com `@Transactional` (se falhar no meio, nada muda).
- Índices justificados com `EXPLAIN ANALYZE` e registrados no README.

- Script de backup e restauração documentado.

9. Testes: JUnit e Mockito

Explicação geral

Testes automatizados são código que **verifica se o seu código funciona**, e que você pode rodar quantas vezes quiser em segundos. Eles dão confiança para mudar o sistema sem medo: se algo quebrar, um teste vermelho avisa antes de o erro chegar ao usuário. Em empresas, código sem teste costuma ser recusado em revisão, e entrevistas de back-end quase sempre perguntam "como você testa seu Service?".

Existem vários níveis, que formam a **pirâmide de testes**:

```
          /\
         /E2E\          poucos, lentos, caros (sistema inteiro)
        /------\
       /Integração\     alguns (Spring + banco real)
      /------------\
     /  Unitários   \   muitos, rápidos, isolados (uma classe)
    /----------------\
```

Neste capítulo você foca na base da pirâmide (testes unitários com **JUnit 5** e **Mockito**) e conhece o nível de integração com Spring.

9.1 JUnit 5: o básico

O JUnit é o framework padrão de testes em Java. O `spring-boot-starter-test` já traz JUnit, Mockito, AssertJ e MockMvc.

```
class LivroTest {

    @Test
    void deveFicarIndisponivelAoEmprestar() {
        Livro livro = new Livro("Java Efetivo", "Bloch", 2018);

        livro.emprestar();

        assertFalse(livro.isDisponivel());
    }

    @Test
    void deveLancarExcecaoAoEmprestarLivroIndisponivel() {
        Livro livro = new Livro("Java Efetivo", "Bloch", 2018);
        livro.emprestar();

        assertThrows(LivroIndisponivelException.class, livro::emprestar);
    }
}
```

Os testes ficam em `src/test/java`, em um pacote **espelho** da classe testada, e a classe termina em `Test`. Rode com `mvn test` ou pela IDE.

Anatomia: Arrange, Act, Assert (AAA)

```
@Test
void devolverDeveTornarLivroDisponivel() {
    // Arrange: prepara o cenário
    Livro livro = new Livro("Clean Code", "Martin", 2008);
    livro.emprestar();

    // Act: executa a ação testada
    livro.devolver();
```

```
    // Assert: verifica o resultado
    assertTrue(livro.isDisponivel());
}
```

Cada teste deve verificar **uma coisa** e ter um nome que descreva o comportamento (`deveLancarExcecaoQuandoLivroIndisponivel`).

Assertions mais usadas

Assertion	Verifica
`assertEquals(esperado, atual)`	Igualdade (esperado primeiro!)
`assertTrue` / `assertFalse`	Condição booleana
`assertNull` / `assertNotNull`	Nulidade
`assertThrows(Tipo.class, () -> ...)`	Que uma exceção é lançada
`assertAll(...)`	Várias verificações juntas
`assertIterableEquals`	Conteúdo de coleções

```
var ex = assertThrows(LivroIndisponivelException.class, () -> livro.emprestar());
assertEquals("Livro indisponível: Clean Code", ex.getMessage());
```

AssertJ (já incluído) deixa as asserções mais legíveis e é muito usado no mercado:

```
assertThat(livros).hasSize(3).extracting(Livro::getTitulo).contains("Clean Code");
assertThat(livro.isDisponivel()).isFalse();
```

Ciclo de vida e organização

Anotação	Quando roda
`@BeforeEach`	Antes de **cada** teste (prepare objetos novos aqui)
`@AfterEach`	Depois de cada teste
`@BeforeAll` / `@AfterAll`	Uma vez por classe (métodos `static`)
`@DisplayName("...")`	Nome legível no relatório
`@Disabled`	Ignora o teste (use com moderação)
`@Nested`	Agrupa testes por cenário

Testes parametrizados

Evitam repetir o mesmo teste com dados diferentes:

```
@ParameterizedTest
@ValueSource(strings = {"", "  ", "\t"})
void tituloEmBrancoDeveSerRejeitado(String titulo) {
    assertThrows(IllegalArgumentException.class, () -> new Livro(titulo, "Autor", 2020));
}

@ParameterizedTest
@CsvSource({"1,1.5", "5,7.5", "10,15"})
void multaDeveSerUmEMeioPorDia(int dias, double esperado) {
    assertEquals(esperado, calculadora.multa(dias), 0.001);
}
```

9.2 Mockito: isolando dependências

Um teste **unitário** do `EmprestimoService` não deve acessar um banco de verdade: seria lento e frágil, e testaria o banco, não o Service. A solução é substituir as dependências por **mocks** (objetos falsos que você controla).

```
@ExtendWith(MockitoExtension.class)
class EmprestimoServiceTest {

    @Mock private LivroRepository livros;
    @Mock private UsuarioRepository usuarios;
    @Mock private EmprestimoRepository emprestimos;

    @InjectMocks private EmprestimoService service;   // recebe os mocks no construtor

    // Observação: os construtores abaixo seguem o modelo do capítulo 6 (sem id; o banco o gera).
    @Test
    void deveEmprestarLivroDisponivel() {
        // Arrange
        Usuario usuario = new Usuario("Ana", "ana@x.com");
        Livro livro = new Livro("Clean Code", "Martin", 2008);
        when(usuarios.findById(1L)).thenReturn(Optional.of(usuario));
        when(livros.findById(2L)).thenReturn(Optional.of(livro));
        when(emprestimos.save(any(Emprestimo.class))).thenAnswer(i -> i.getArgument(0));

        // Act
        Emprestimo resultado = service.emprestar(1L, 2L);

        // Assert
        assertThat(resultado.getLivro()).isSameAs(livro);
        assertThat(livro.isDisponivel()).isFalse();
        verify(emprestimos).save(any(Emprestimo.class));
    }

    @Test
    void deveLancarExcecaoQuandoLivroNaoExiste() {
        when(usuarios.findById(1L)).thenReturn(Optional.of(new Usuario("Ana", "ana@x.com")));
        when(livros.findById(99L)).thenReturn(Optional.empty());

        assertThrows(LivroNaoEncontradoException.class, () -> service.emprestar(1L, 99L));

        verify(emprestimos, never()).save(any());     // nada deve ser salvo
    }
}
```

Vocabulário do Mockito

Recurso	Para quê
`@Mock`	Cria um objeto falso da dependência
`@InjectMocks`	Cria a classe testada injetando os mocks
`when(x.metodo()).thenReturn(v)`	Define o que o mock devolve
`thenThrow(new Ex())`	Faz o mock lançar exceção
`verify(mock).metodo()`	Confirma que o método foi chamado
`verify(mock, times(2))`, `never()`	Quantidade de chamadas
`any()`, `eq(v)`	Casadores de argumento
`ArgumentCaptor`	Captura o argumento passado para inspeção
`@Spy`	Objeto real com partes "espionadas" (use raramente)

```
ArgumentCaptor<Emprestimo> captor = ArgumentCaptor.forClass(Emprestimo.class);
verify(emprestimos).save(captor.capture());
assertThat(captor.getValue().getStatus()).isEqualTo(StatusEmprestimo.ATIVO);
```

> **ATENÇÃO** Ao usar `any()` em um argumento, os **demais** argumentos do mesmo método também precisam usar casadores (`eq(...)`). Misturar valor literal com casador gera erro.

Dicas importantes

- Mocke **dependências**, nunca a classe que está testando.
- Não mocke tipos simples (`String`, `List`) nem entidades: crie objetos reais.
- Excesso de `verify` deixa o teste preso à implementação. Prefira verificar o **resultado** e use `verify` para efeitos colaterais importantes (salvar, enviar e-mail).
- Mockito moderno é "estrito": falha se você configurar um `when` que nunca é usado, o que ajuda a manter os testes limpos.

9.3 Testes de integração com Spring

Testam várias peças juntas, com o contexto do Spring.

Testando só a camada web: `@WebMvcTest` + MockMvc

Sobe apenas o Controller (sem banco), com o Service mockado. (O pacote dos imports de `@WebMvcTest` e `@MockitoBean` pode variar conforme a versão do Spring Boot; se a IDE não resolver o import, consulte o guia de migração da sua versão.)

```
@WebMvcTest(LivroController.class)
class LivroControllerTest {

    @Autowired private MockMvc mvc;
    @MockitoBean private LivroService service;      // @MockBean em versões mais antigas

    @Test
    void deveRetornar404QuandoLivroNaoExiste() throws Exception {
        when(service.buscarPorId(99L)).thenThrow(new LivroNaoEncontradoException(99L));

        mvc.perform(get("/api/v1/livros/99"))
           .andExpect(status().isNotFound());
    }

    @Test
    void deveCriarLivroERetornar201() throws Exception {
        when(service.criar(any()))
                .thenReturn(new LivroResponse(1L, "Clean Code", "Martin", 2008, true));

        mvc.perform(post("/api/v1/livros")
                .contentType(MediaType.APPLICATION_JSON)
                .content("""
                    {"titulo":"Clean Code","autor":"Martin","ano":2008}
                    """))
           .andExpect(status().isCreated())
           .andExpect(jsonPath("$.titulo").value("Clean Code"));
    }
}
```

Testando a persistência: `@DataJpaTest`

Sobe apenas JPA com um banco em memória e executa seus repositórios de verdade. Por padrão ele substitui o seu banco por um embutido, então o **H2 precisa estar no classpath de teste** (`com.h2database:h2`, escopo `test`) ou você deve usar `@AutoConfigureTestDatabase(replace = NONE)` com um banco real (Testcontainers, adiante).

> **ATENÇÃO** Se o projeto usa Flyway, as migrações **também rodam** no banco do teste. Uma migração com SQL específico do PostgreSQL (como o índice único parcial do capítulo 8) **falha no H2**. Para testar repositórios de um projeto com Flyway, use Testcontainers com PostgreSQL.

```
@DataJpaTest
class LivroRepositoryTest {

    @Autowired private LivroRepository repository;

    @Test
    void deveBuscarLivrosDisponiveisDoAutor() {
        repository.save(new Livro("Clean Code", "Martin", 2008));
        repository.save(new Livro("Clean Architecture", "Martin", 2017));

        List<Livro> achados = repository.findByAutor("Martin");

        assertThat(achados).hasSize(2);
    }
}
```

Testcontainers (PostgreSQL de verdade)

H2 não se comporta exatamente como o PostgreSQL (tipos, índices parciais, funções). O **Testcontainers** sobe um PostgreSQL real em Docker apenas durante o teste, dando mais fidelidade:

```
@DataJpaTest
@AutoConfigureTestDatabase(replace = AutoConfigureTestDatabase.Replace.NONE)
@Testcontainers
class LivroRepositoryPgTest {

    @Container
    @ServiceConnection
    static PostgreSQLContainer<?> pg = new PostgreSQLContainer<>("postgres:16");

    // testes ...
}
```

Exige Docker em execução; é o padrão ouro para testes de repositório.

> **DICA** A API do Testcontainers mudou entre as versões 1.x e 2.x (por exemplo, nomes de módulos e de classes de contêiner). O código acima segue o estilo 1.x. Ao usar uma versão 2.x, consulte o guia de migração oficial do Testcontainers e ajuste os imports e as dependências. Em ambos os casos, o conceito é o mesmo: o teste sobe um PostgreSQL real em Docker.

`@SpringBootTest`

Carrega **a aplicação inteira**. É lento: use pouco, para poucos fluxos críticos (ex.: criar usuário → criar livro → emprestar → devolver).

Anotação	O que sobe	Velocidade	Use para
(nenhuma, só JUnit+Mockito)	Nada	Muito rápida	Service, regras de domínio
`@WebMvcTest`	Camada web	Rápida	Controller, status, JSON, validação
`@DataJpaTest`	JPA + banco	Média	Repositories, queries
`@SpringBootTest`	Aplicação inteira	Lenta	Fluxos ponta a ponta

9.4 Boas práticas

- **F.I.R.S.T.**: *Fast* (rápidos), *Independent* (independentes entre si e sem ordem), *Repeatable* (mesmo resultado sempre), *Self-validating* (passa ou falha, sem inspeção manual), *Timely* (escritos junto com o código).
- Teste **comportamento e regras de negócio**, não getters e setters.
- Cubra os **caminhos felizes e os de erro** (livro indisponível, usuário inexistente, limite excedido).
- Teste **valores de fronteira**: zero, um, limite exato, limite + 1, nulo, vazio.
- Dados de teste com **builders** ou métodos auxiliares para evitar repetição.
- Um teste que falha de vez em quando (*flaky*) é pior que nenhum: investigue datas, ordem e concorrência (injete um `Clock` em vez de usar `LocalDate.now()` direto).
- **TDD** (Test-Driven Development): escrever o teste antes, ver falhar (vermelho), implementar o mínimo (verde), refatorar. Vale experimentar nos exercícios.

Cobertura de código

A cobertura mede quanto do código os testes executam. Use o **JaCoCo** (plugin Maven) para gerar o relatório com `mvn verify`. Cobertura alta **não garante qualidade**, mas cobertura muito baixa indica risco. Mire em cobrir bem os Services (regras de negócio), por exemplo acima de 80%.

```
<plugin>
  <groupId>org.jacoco</groupId>
  <artifactId>jacoco-maven-plugin</artifactId>
  <version>0.8.12</version>
  <executions>
    <execution><goals><goal>prepare-agent</goal></goals></execution>
    <execution>
      <id>report</id><phase>verify</phase><goals><goal>report</goal></goals>
    </execution>
  </executions>
</plugin>
```

O relatório HTML fica em `target/site/jacoco/index.html`.

Resumo do capítulo

- Pirâmide de testes: muitos unitários, alguns de integração, poucos ponta a ponta.
- JUnit 5: `@Test`, `@BeforeEach`, `assertEquals`, `assertThrows`, `@ParameterizedTest`; padrão AAA; AssertJ para legibilidade.
- Mockito isola dependências: `@Mock`, `@InjectMocks`, `when/thenReturn`, `verify`, `ArgumentCaptor`.
- Spring: `@WebMvcTest` (Controller + MockMvc), `@DataJpaTest` (repositórios), `@SpringBootTest` (tudo; usar pouco); Testcontainers para PostgreSQL real.
- Teste regras de negócio, erros e valores de fronteira; testes devem ser rápidos, independentes e repetíveis.
- JaCoCo mede cobertura; qualidade importa mais que o número.

Exercícios teóricos

1. Quais as diferenças entre teste unitário, de integração e ponta a ponta?
2. O que significa AAA? Por que organizar os testes assim?
3. O que é um mock? Por que não usar o repositório real em um teste unitário de Service?
4. Qual a diferença entre `when(...).thenReturn(...)` e `verify(...)`?
5. Quando usar `@WebMvcTest`, `@DataJpaTest` e `@SpringBootTest`?

6. Por que o H2 pode "mentir" em testes de repositório? Como o Testcontainers ajuda?
7. O que são valores de fronteira? Dê três exemplos para a regra "limite de 3 empréstimos".
8. O que torna um teste *flaky*? Como `Clock` injetado ajuda?
9. Cobertura de 100% garante um sistema sem bugs? Explique.
10. Descreva o ciclo do TDD.

Exercícios práticos

1. Escreva 5 testes JUnit para uma classe `Calculadora` (inclua divisão por zero com `assertThrows`).
2. Converta 3 testes repetidos em um `@ParameterizedTest` com `@CsvSource`.
3. Faça TDD: escreva primeiro os testes para `validarCpf(String)` e só depois implemente.
4. Teste um `PedidoService` mockando `PedidoRepository` e um `EmailSender`; use `verify` e `ArgumentCaptor`.
5. Teste os caminhos de erro: dependência devolvendo `Optional.empty()` e mock lançando exceção.
6. Use `@WebMvcTest` para testar um endpoint `GET` (200) e um `POST` (201) com `MockMvc`.
7. Use `@DataJpaTest` para testar dois query methods.
8. Configure o JaCoCo, gere o relatório e aumente a cobertura de uma classe de 40% para mais de 80%.

Projeto principal: Sistema Biblioteca

1. **Testes de domínio** (JUnit puro): `Livro.emprestar/devolver`, cálculo de multa (com valores de fronteira), validações do construtor.
2. **Testes de** `EmprestimoService` com Mockito, cobrindo no mínimo: empréstimo com sucesso, usuário inexistente, livro inexistente, livro indisponível, limite de empréstimos excedido, devolução com atraso (multa), devolução de empréstimo já devolvido.
3. **Testes de Controller** com `@WebMvcTest`: status `201`, `404` e `409` (após o mapeamento do capítulo 11, volte e complete).
4. **Testes de Repository** com `@DataJpaTest` e **Testcontainers (obrigatório aqui)**: o H2 não executa o índice único parcial do capítulo 8. Teste as queries personalizadas e a violação do índice.
5. Um teste `@SpringBootTest` do fluxo completo: cadastrar usuário e livro → emprestar → tentar emprestar de novo (erro) → devolver.
6. Injete um `Clock` para testar regras de prazo sem depender da data real.
7. Configure o JaCoCo e meta de **80%** nos Services; mostre o resultado no README (badge opcional).

Mini-projeto: Biblioteca de Regras de Negócio (kata de testes)

Em outro repositório, pratique TDD em três *katas* clássicos, sempre escrevendo o teste primeiro:

- **FizzBuzz** (aquecimento).
- **Calculadora de frete**: regras por região, peso e valor mínimo para frete grátis; mocke um `TabelaDeFretes`.
- **Carrinho de compras**: adicionar/remover itens, cupom de desconto (percentual e valor fixo, não acumulativos), limite de quantidade por item.
- Mínimo de 25 testes (usando `@Nested` para organizar por cenário), parametrizados onde fizer sentido.
- Relatório JaCoCo acima de 90% e README explicando as decisões.

10. Docker

Explicação geral

Quem nunca ouviu "na minha máquina funciona"? O problema nasce porque cada ambiente tem versões diferentes de Java, de banco, de variáveis e de sistema operacional. **Docker** resolve isso empacotando a aplicação **junto com tudo de que ela precisa** em uma unidade padronizada, o **contêiner**, que roda do mesmo jeito no seu notebook, no servidor da empresa ou na nuvem.

Para o desenvolvedor back-end, Docker tem dois usos principais:

1. **Subir dependências sem instalar nada** (PostgreSQL, Redis, RabbitMQ) com um comando.
2. **Empacotar e entregar a própria aplicação** como uma imagem, para ser implantada em qualquer lugar.

Conceito	O que é	Analogia
Imagem	Pacote somente leitura com o sistema, o Java e a aplicação	A "receita pronta" / molde
Contêiner	Instância em execução de uma imagem	O "bolo" feito a partir do molde
Dockerfile	Arquivo com os passos para construir uma imagem	A receita escrita
Registry	Repositório de imagens (Docker Hub, GHCR)	Uma "loja" de imagens
Volume	Armazenamento persistente fora do contêiner	Um HD externo
Network	Rede virtual entre contêineres	Uma rede local privada
Compose	Define e sobe vários contêineres juntos	Um "maestro"

Contêiner não é máquina virtual

Uma VM carrega um sistema operacional inteiro. Um contêiner **compartilha o kernel** da máquina hospedeira e isola apenas processos e arquivos. Por isso é muito mais leve e inicia em segundos.

10.1 Instalação e primeiros comandos

Instale o **Docker Desktop** (Windows/macOS) ou o **Docker Engine** (Linux) e confirme:

```
docker --version
docker run hello-world
```

Comandos essenciais

```
docker pull postgres:16             # baixa uma imagem
docker images                       # lista imagens locais
docker run -d --name meu-pg -p 5432:5432 -e POSTGRES_PASSWORD=postgres postgres:16
docker ps                           # contêineres em execução
docker ps -a                        # todos, inclusive parados
docker logs -f meu-pg               # acompanha os logs
docker exec -it meu-pg bash         # abre um terminal dentro do contêiner
docker stop meu-pg                  # para
docker start meu-pg                 # inicia de novo
docker rm meu-pg                    # remove o contêiner
docker rmi postgres:16              # remove a imagem
docker system prune                 # limpa itens sem uso (cuidado)
```

Opção do `docker run`	Significado
`-d`	Executa em segundo plano (*detached*)
`--name`	Dá um nome ao contêiner
`-p host:contêiner`	Publica uma porta
`-e CHAVE=valor`	Define variável de ambiente
`-v volume:/caminho`	Monta um volume
`--rm`	Remove o contêiner ao terminar
`--network`	Conecta a uma rede

> **ATENÇÃO** Tags importam: `postgres:latest` muda com o tempo e pode quebrar seu projeto. **Fixe a versão** (`postgres:16`) para ter ambientes reproduzíveis.

10.2 Dockerfile

O Dockerfile descreve, instrução por instrução, como montar a imagem. Exemplo para uma aplicação Spring Boot que já foi compilada (`mvn clean package`):

```
FROM eclipse-temurin:21-jre
WORKDIR /app
COPY target/biblioteca-api-1.0.0.jar app.jar
EXPOSE 8080
ENTRYPOINT ["java", "-jar", "app.jar"]
```

Instrução	Função
`FROM`	Imagem base (aqui, Java 21 só com o JRE)
`WORKDIR`	Diretório de trabalho dentro da imagem
`COPY`	Copia arquivos da sua máquina para a imagem
`RUN`	Executa um comando **durante o build**
`ENV`	Define variável de ambiente
`EXPOSE`	Documenta a porta usada (não publica sozinha)
`ENTRYPOINT` / `CMD`	O que executar **quando o contêiner inicia**

Construa e rode:

```
docker build -t biblioteca-api:1.0 .
docker run -d --name api -p 8080:8080 biblioteca-api:1.0
```

Multi-stage build (o jeito profissional)

O exemplo anterior depende de compilar o projeto antes, fora do Docker. Com **múltiplos estágios**, o próprio build compila a aplicação, e a imagem final contém só o necessário (menor e mais segura):

```
# Estágio 1: compila com Maven + JDK
FROM maven:3.9-eclipse-temurin-21 AS build
WORKDIR /build
COPY pom.xml .
RUN mvn -B dependency:go-offline          # camada de cache das dependências
COPY src ./src
RUN mvn -B clean package -DskipTests

# Estágio 2: imagem final enxuta, só com JRE
FROM eclipse-temurin:21-jre
```

```
WORKDIR /app
RUN addgroup --system app && adduser --system --ingroup app app
COPY --from=build /build/target/*.jar app.jar
USER app                                    # não roda como root
EXPOSE 8080
ENTRYPOINT ["java", "-jar", "app.jar"]
```

Detalhes que fazem diferença:

- **Cache de camadas**: copiar o `pom.xml` e baixar dependências **antes** de copiar o `src` faz o Docker reaproveitar essa camada quando só o código muda, acelerando muito os builds.
- `USER app`: rodar como usuário comum reduz o risco em caso de invasão.
- Imagem final só com JRE: menor e com menos superfície de ataque.

`.dockerignore`

Evita enviar arquivos inúteis ao build (funciona como o `.gitignore`):

```
target/
.git/
.idea/
*.md
.env
```

10.3 Volumes, redes e variáveis de ambiente

Volumes: persistindo dados

Tudo que é gravado dentro de um contêiner **some quando ele é removido**. Para dados importantes (banco), use um volume:

```
docker volume create pg-data
docker run -d --name pg -v pg-data:/var/lib/postgresql/data postgres:16
docker volume ls
```

Há também o **bind mount** (`-v ./pasta:/dentro`), que liga uma pasta da sua máquina ao contêiner; útil em desenvolvimento, por exemplo para scripts SQL.

Redes: contêineres conversando

Contêineres na mesma rede se encontram **pelo nome**. Dentro do contêiner da API, o banco não está em `localhost`, e sim no nome do serviço:

```
Errado:  jdbc:postgresql://localhost:5432/biblioteca
Certo:   jdbc:postgresql://postgres:5432/biblioteca   (postgres = nome do serviço)
```

> **ATENÇÃO** Erro clássico: usar `localhost` na URL do banco dentro do contêiner da aplicação. Para o contêiner, `localhost` é **ele mesmo**, não a máquina nem o outro contêiner.

Variáveis de ambiente

É assim que se configura a mesma imagem para ambientes diferentes, sem recompilar. O Spring Boot converte `SPRING_DATASOURCE_URL` em `spring.datasource.url` automaticamente.

```
docker network create biblioteca-net           # rede definida pelo usuário (permite resolver nomes)

docker run -d --name postgres --network biblioteca-net \
           -e POSTGRES_DB=biblioteca -e POSTGRES_PASSWORD=postgres postgres:16

docker run -d --name api --network biblioteca-net -p 8080:8080 \
           -e SPRING_DATASOURCE_URL=jdbc:postgresql://postgres:5432/biblioteca \
           -e SPRING_DATASOURCE_USERNAME=postgres \
           -e SPRING_DATASOURCE_PASSWORD=postgres \
```

```
biblioteca-api:1.0
```

> **ATENÇÃO** Na rede *bridge* padrão do Docker, os contêineres **não** se resolvem pelo nome. É preciso criar uma rede (`docker network create`) e conectar os dois contêineres a ela com `--network`. O Docker Compose faz isso automaticamente para os serviços do mesmo arquivo.

10.4 Docker Compose

Subir banco e API com vários `docker run` é trabalhoso. O **Compose** descreve tudo em um único arquivo `compose.yaml`:

```yaml
services:
  postgres:
    image: postgres:16
    environment:
      POSTGRES_DB: biblioteca
      POSTGRES_USER: ${DB_USER:-postgres}
      POSTGRES_PASSWORD: ${DB_PASSWORD:-postgres}
    ports:
      - "5432:5432"
    volumes:
      - pg-data:/var/lib/postgresql/data
    healthcheck:
      # $$ escapa o cifrão: a variável é lida dentro do contêiner
      test: ["CMD-SHELL", "pg_isready -U $${POSTGRES_USER} -d $${POSTGRES_DB}"]
      interval: 5s
      timeout: 3s
      retries: 10

  api:
    build: .
    depends_on:
      postgres:
        condition: service_healthy        # espera o banco ficar pronto
    environment:
      SPRING_DATASOURCE_URL: jdbc:postgresql://postgres:5432/biblioteca
      SPRING_DATASOURCE_USERNAME: ${DB_USER:-postgres}
      SPRING_DATASOURCE_PASSWORD: ${DB_PASSWORD:-postgres}
    ports:
      - "8080:8080"

volumes:
  pg-data:
```

Pontos importantes do arquivo:

- Cada chave em `services` é um contêiner; o **nome do serviço** (`postgres`) vira o nome de rede.
- `healthcheck` + `depends_on: condition: service_healthy`: `depends_on` sozinho só espera o contêiner **iniciar**, não o banco estar **pronto** para conexões.
- `${DB_PASSWORD:-postgres}` lê da variável de ambiente ou de um arquivo `.env` (que **não** vai para o Git), com valor padrão.
- O volume `pg-data` preserva os dados entre execuções.

Comandos do Compose

```
docker compose up -d            # sobe tudo em segundo plano
docker compose up -d --build    # reconstrói a imagem da API antes
docker compose ps               # estado dos serviços
docker compose logs -f api      # logs de um serviço
docker compose exec postgres psql -U postgres -d biblioteca
docker compose down             # derruba (mantém os volumes)
docker compose down -v          # derruba e APAGA os volumes (dados perdidos!)
```

Um perfil comum de desenvolvimento: **só o banco** no Compose, e a API rodando pela IDE com `localhost:5432`. Em seguida, para validar o empacotamento, suba tudo.

10.5 Boas práticas

- Um processo principal por contêiner (API e banco em contêineres separados).
- Imagens pequenas, com versão fixa e **multi-stage**.
- Nunca grave senhas na imagem ou no repositório: use variáveis de ambiente ou *secrets*.
- Contêineres são **descartáveis**: não dependa de editar algo dentro de um contêiner em execução.
- Rode como usuário não-root.
- Logs para a saída padrão (`stdout`), de onde o Docker os coleta (o Spring já faz isso).
- Use **Actuator** para um endpoint de saúde (`/actuator/health`) e para o `healthcheck` da API.
- Imagens de terceiros: prefira as oficiais e verifique a origem.

10.6 Problemas comuns

Sintoma	Causa provável	Solução
`port is already allocated`	Porta ocupada (outro PostgreSQL local, por exemplo)	Pare o outro ou mude o mapeamento (`5433:5432`)
API não conecta no banco	`localhost` na URL, ou banco ainda subindo	Use o nome do serviço e `healthcheck`
Dados sumiram	Faltou volume, ou usou `down -v`	Declare volume; cuidado com `-v`
Build lento a cada mudança	`COPY . .` antes do download de dependências	Reordene para aproveitar o cache
`no main manifest attribute`	JAR sem o plugin do Spring Boot	Confirme `spring-boot-maven-plugin` no `pom.xml`
Imagem enorme	JDK completo e arquivos desnecessários	Multi-stage, JRE e `.dockerignore`

Resumo do capítulo

- Docker empacota aplicação e dependências em **imagens**, executadas como **contêineres** (leves, compartilham o kernel).
- Comandos: `pull`, `run`, `ps`, `logs`, `exec`, `stop`, `rm`; fixe sempre a **tag** da imagem.
- **Dockerfile**: `FROM`, `COPY`, `RUN`, `ENTRYPOINT`; multi-stage reduz o tamanho; ordene as camadas para usar cache.
- **Volumes** persistem dados; **redes** permitem comunicação pelo nome do serviço (nunca `localhost` entre contêineres).
- **Compose** sobe API + PostgreSQL com um comando; `healthcheck` garante a ordem correta.
- Segredos em variáveis de ambiente, usuário não-root, `.dockerignore`.

Exercícios teóricos

1. Qual a diferença entre imagem e contêiner?
2. Qual a diferença entre contêiner e máquina virtual?
3. O que acontece com os dados de um contêiner quando ele é removido? Como evitar a perda?
4. Para que serve o multi-stage build? Que vantagens traz?

5. Por que copiar o `pom.xml` e baixar dependências antes de copiar o código-fonte?
6. Por que a API, dentro do contêiner, não deve usar `localhost` para acessar o banco?
7. Qual a diferença entre `EXPOSE` e `-p`?
8. Qual a diferença entre `depends_on` simples e com `condition: service_healthy`?
9. O que faz `docker compose down -v`? Por que é perigoso?
10. Por que fixar a tag da imagem (`postgres:16`) em vez de `latest`?

Exercícios práticos

1. Rode `docker run hello-world` e um `nginx` na porta 8081; acesse no navegador, veja os logs e remova.
2. Suba um PostgreSQL com volume, crie uma tabela, **remova o contêiner**, suba outro com o mesmo volume e confirme que os dados continuam.
3. Escreva um Dockerfile para uma API Spring Boot simples (do capítulo 6) e rode-a.
4. Reescreva o Dockerfile em **multi-stage** e compare o tamanho com `docker images`.
5. Edite só uma linha de código e refaça o build duas vezes; observe o cache funcionando.
6. Crie um `compose.yaml` com API + PostgreSQL, com `healthcheck` e `.env`.
7. Provoque o erro do `localhost` de propósito, leia a mensagem e corrija.
8. Adicione o `pgAdmin` ou `adminer` ao Compose para inspecionar o banco pelo navegador.

Projeto principal: Sistema Biblioteca

1. Crie o `Dockerfile` multi-stage da `biblioteca-api` e o `.dockerignore`.
2. Crie o `compose.yaml` com `postgres` (volume + `healthcheck`) e `api`; credenciais por `.env` (e um `.env.example` versionado, sem segredos).
3. Garanta que o **Flyway** rode automaticamente na subida e crie o esquema.
4. Adicione `spring-boot-starter-actuator` e exponha `/actuator/health`. Para usá-lo como `healthcheck` da API no Compose, a imagem precisa de uma ferramenta de chamada HTTP (a imagem `eclipse-temurin:21-jre` pode não trazer `curl`; instale-o no Dockerfile com `RUN apt-get update && apt-get install -y curl` ou use outra estratégia) e então `test: ["CMD", "curl", "-f", "http://localhost:8080/actuator/health"]`.
5. Confirme o fluxo completo: `docker compose up -d --build` e teste os endpoints no Postman; depois `down` e `up` novamente para ver que os dados persistem.
6. Documente no README: pré-requisitos, comandos, portas e como parar/limpar tudo.
7. Faça commit em uma branch `feature/docker` e abra um PR.

Mini-projeto: Stack Completa com Cache (API + PostgreSQL + Redis)

Em outro repositório, monte uma pequena API de "catálogo de produtos" com três contêineres:

- API Spring Boot (multi-stage), PostgreSQL com volume e **Redis** como cache (`spring-boot-starter-data-redis` com `@Cacheable`).
- Rede própria, `healthcheck` em todos os serviços e variáveis em `.env`.
- Endpoint `GET /produtos/{id}` que usa o cache; registre no log quando buscar no banco ou no Redis.
- Perfil `docker` do Spring com configurações específicas do ambiente em contêiner.

- README com diagrama da arquitetura e instruções de execução em um único comando.

11. API profissional: DTOs, validação e erros

Explicação geral

Uma API que funciona não é necessariamente uma API **boa**. Entre o "funciona na minha máquina" do capítulo 6 e o código que uma empresa aceita em produção existem quatro práticas que este capítulo ensina:

1. **DTOs**: não expor entidades diretamente.
2. **Validação**: rejeitar dados ruins na porta de entrada.
3. **Tratamento global de erros**: respostas de erro padronizadas e com o status correto.
4. **Documentação**: contrato visível e testável (OpenAPI/Swagger).

Aqui também fecham-se os mapeamentos prometidos nos capítulos anteriores: `LivroNaoEncontradoException` vira `404`, `LivroIndisponivelException` vira `409`, e assim por diante.

11.1 DTOs (Data Transfer Objects)

Um **DTO** é um objeto criado só para **transportar dados** entre o cliente e a aplicação. Usar a entidade JPA diretamente no Controller é uma má prática, por vários motivos:

Problema de expor a entidade	Exemplo
Vaza dados internos	Devolver o `hash` da senha do usuário
Acopla a API ao banco	Renomear uma coluna quebra o contrato com o cliente
Mass assignment	O cliente envia `"id": 1` ou `"disponivel": true` e altera o que não deveria
Laços infinitos no JSON	Relação bidirecional `Usuario` ↔ `Emprestimo`
`LazyInitializationException`	Serializar associação lazy fora da transação

Request e Response separados

Com `record`, DTOs ficam concisos e imutáveis:

```
// o que o cliente ENVIA
public record LivroRequest(String titulo, String autor, Integer ano) { }

// o que a API DEVOLVE
public record LivroResponse(Long id, String titulo, String autor, Integer ano, boolean disponivel) {

    public static LivroResponse de(Livro l) {
        return new LivroResponse(l.getId(), l.getTitulo(), l.getAutor(),
                l.getAno(), l.isDisponivel());
    }
}
```

O fluxo passa a ser:

```
JSON → LivroRequest → (Service) → Livro (entidade) → LivroResponse → JSON
```

Mapeamento entre entidade e DTO

- **Manual** (métodos `de(...)`/`toEntity()`): simples, explícito, sem mágica. Ótimo para começar.
- **MapStruct**: gera os conversores em tempo de compilação, sem reflexão e sem custo de execução (muito usado no mercado).

```
@Mapper(componentModel = "spring")
public interface LivroMapper {
    LivroResponse toResponse(Livro livro);     // entidade → DTO de saída: é o uso mais seguro
}
```

> **DICA** O mapeamento **inverso** (DTO → entidade, `toEntity`) exige que a entidade tenha construtor ou setters acessíveis. Nas entidades desta apostila (construtor `protected` e regras de negócio em métodos), prefira criar a entidade **no Service**, chamando o construtor com validação: `new Livro(req.titulo(), req.autor(), req.ano())`.

> **DICA** Regra prática: o Controller fala em DTOs; o Service converte; a entidade nunca sai do Service. Para endpoints de leitura com muitas associações, devolva DTOs montados dentro da transação (resolve também o `LazyInitializationException`).

11.2 Validação com Bean Validation

Dados de fora **nunca são confiáveis**. A dependência `spring-boot-starter-validation` permite declarar regras com anotações.

```
public record LivroRequest(
        @NotBlank(message = "O título é obrigatório")
        @Size(max = 200, message = "O título deve ter no máximo 200 caracteres")
        String titulo,

        @NotBlank(message = "O autor é obrigatório")
        String autor,

        @NotNull(message = "O ano é obrigatório")
        @Min(value = 1450, message = "Ano inválido")
        @Max(value = 2100, message = "Ano inválido")
        Integer ano
) { }

public record UsuarioRequest(
        @NotBlank String nome,
        @NotBlank @Email(message = "E-mail inválido") String email
) { }
```

Ative a validação no Controller com `@Valid`:

```
@PostMapping
public ResponseEntity<LivroResponse> criar(@Valid @RequestBody LivroRequest request) {
    ...
}
```

Se alguma regra falhar, o Spring lança `MethodArgumentNotValidException` **antes** de entrar no método.

Anotação	Valida
`@NotNull`	Não é nulo
`@NotBlank`	String não nula e com algo além de espaços
`@NotEmpty`	Coleção/string não vazia
`@Size(min, max)`	Tamanho de texto ou coleção

Anotação	Valida
`@Min` / `@Max`	Limites numéricos
`@Positive` / `@PositiveOrZero`	Números positivos
`@Email`	Formato de e-mail
`@Pattern(regexp=...)`	Expressão regular
`@Past` / `@Future`	Datas no passado/futuro
`@Valid` (em atributo)	Valida objetos aninhados

Para parâmetros de URL, coloque `@Validated` na classe do Controller e valide direto: `@PathVariable @Positive Long id`. A violação lança `ConstraintViolationException` (tratada no handler abaixo). Em versões recentes do Spring, a validação de parâmetros de método também pode lançar `HandlerMethodValidationException`, que a classe base `ResponseEntityExceptionHandler` já converte em `400`.

Validação personalizada

```
@Target(FIELD) @Retention(RUNTIME)
@Constraint(validatedBy = IsbnValidator.class)
public @interface Isbn {
    String message() default "ISBN inválido";
    Class<?>[] groups() default {};
    Class<? extends Payload>[] payload() default {};
}
```

Onde validar o quê

Tipo de regra	Onde	Exemplo
Formato e obrigatoriedade	DTO (Bean Validation)	título em branco, e-mail inválido
Regra de negócio	Service (exceção de domínio)	livro indisponível, limite de empréstimos
Integridade final	Banco (`NOT NULL`, `UNIQUE`, `FK`)	e-mail duplicado

Os três níveis se complementam: a validação no DTO dá feedback rápido, o Service protege o negócio e o banco é a última barreira.

11.3 Tratamento global de exceções

Sem tratamento, qualquer exceção vira `500` com um JSON confuso. Com `@RestControllerAdvice`, você centraliza a conversão de exceções em respostas padronizadas, em **um único lugar**.

Formato padrão de erro

```
// não envia "campos": null quando não há erros de campo
@JsonInclude(JsonInclude.Include.NON_NULL)
public record ErroResponse(
        Instant timestamp,
        int status,
        String erro,
        String mensagem,
        String caminho,
        List<CampoInvalido> campos) {

    public record CampoInvalido(String campo, String mensagem) { }
}
```

O handler global

A forma recomendada é **estender** `ResponseEntityExceptionHandler`, uma classe do Spring que já trata as exceções do próprio MVC com o status correto (JSON malformado → `400`, método não permitido → `405`, rota inexistente → `404` etc.). Você só acrescenta as suas exceções e ajusta o formato do corpo.

> **ATENÇÃO** Se você criar apenas um `@ExceptionHandler(Exception.class)` "pega-tudo", ele captura **também** as exceções do próprio Spring (JSON malformado, `405`, validação de parâmetro, `AccessDeniedException` do capítulo 12) e as transforma em `500`. Por isso o pega-tudo abaixo vem **depois** dos tratamentos específicos, e a classe base cuida do restante.

Antes, crie duas classes-base para reduzir o número de handlers (e faça `LivroNaoEncontradoException`, `UsuarioNaoEncontradoException` etc. as estenderem):

```
public abstract class RecursoNaoEncontradoException extends RuntimeException {      // → 404
    protected RecursoNaoEncontradoException(String msg) { super(msg); }
}
public abstract class RegraDeNegocioException extends RuntimeException {            // → 409
    protected RegraDeNegocioException(String msg) { super(msg); }
}
// LivroIndisponivelException e LimiteEmprestimosException passam a estender RegraDeNegocioException

@RestControllerAdvice
public class GlobalExceptionHandler extends ResponseEntityExceptionHandler {

    private static final Logger log = LoggerFactory.getLogger(GlobalExceptionHandler.class);

    @ExceptionHandler(RecursoNaoEncontradoException.class)
    public ResponseEntity<ErroResponse> naoEncontrado(RecursoNaoEncontradoException ex,
            HttpServletRequest req) {
        return erro(HttpStatus.NOT_FOUND, ex.getMessage(), req, null);
    }

    @ExceptionHandler({RegraDeNegocioException.class, OptimisticLockingFailureException.class})
    public ResponseEntity<ErroResponse> conflito(RuntimeException ex,
            HttpServletRequest req) {
        String msg = (ex instanceof RegraDeNegocioException)
                ? ex.getMessage() : "O registro foi alterado por outra operação. Tente novamente.";
        return erro(HttpStatus.CONFLICT, msg, req, null);
    }

    @ExceptionHandler(DataIntegrityViolationException.class)
    public ResponseEntity<ErroResponse> integridade(DataIntegrityViolationException ex,
            HttpServletRequest req) {
        return erro(HttpStatus.CONFLICT, "Violação de integridade dos dados", req, null);
    }

    // @PathVariable/@RequestParam validados
    @ExceptionHandler(ConstraintViolationException.class)
    public ResponseEntity<ErroResponse> parametroInvalido(ConstraintViolationException ex,
            HttpServletRequest req) {
        return erro(HttpStatus.BAD_REQUEST, "Parâmetro inválido", req, null);
    }

    @ExceptionHandler(CredenciaisInvalidasException.class)         // usada no login (capítulo 12)
    public ResponseEntity<ErroResponse> credenciais(CredenciaisInvalidasException ex,
            HttpServletRequest req) {
        return erro(HttpStatus.UNAUTHORIZED, "Credenciais inválidas", req, null);
    }

    @ExceptionHandler(AccessDeniedException.class)                 // @PreAuthorize (capítulo 12)
    public ResponseEntity<ErroResponse> acessoNegado(AccessDeniedException ex,
            HttpServletRequest req) {
        return erro(HttpStatus.FORBIDDEN, "Acesso negado", req, null);
    }

    // rede de segurança: só o que sobrou
    @ExceptionHandler(Exception.class)
    public ResponseEntity<ErroResponse> inesperado(Exception ex,
```

```
            HttpServletRequest req) {
        log.error("Erro inesperado em {}", req.getRequestURI(), ex);      // detalhe só no log
        return erro(HttpStatus.INTERNAL_SERVER_ERROR,
                "Erro interno. Tente novamente mais tarde.", req, null);
    }

    // Validação de @Valid @RequestBody: sobrescreve o método da classe base para listar os campos
    @Override
    protected ResponseEntity<Object> handleMethodArgumentNotValid(MethodArgumentNotValidException ex,
            HttpHeaders headers, HttpStatusCode status, WebRequest request) {
        List<ErroResponse.CampoInvalido> campos = ex.getBindingResult().getFieldErrors().stream()
                .map(f -> new ErroResponse.CampoInvalido(f.getField(), f.getDefaultMessage()))
                .toList();
        return ResponseEntity.badRequest().body(
                corpo(HttpStatus.BAD_REQUEST, "Dados inválidos", path(request), campos));
    }

    // Demais exceções do MVC (JSON malformado, 405, 404...): mantém o status da classe base,
    // mas devolve o NOSSO formato de erro
    @Override
    protected ResponseEntity<Object> handleExceptionInternal(Exception ex, Object body,
            HttpHeaders headers, HttpStatusCode statusCode, WebRequest request) {
        HttpStatus status = HttpStatus.valueOf(statusCode.value());
        return ResponseEntity.status(status).headers(headers)
                .body(corpo(status, status.getReasonPhrase(), path(request), null));
    }

    private ResponseEntity<ErroResponse> erro(HttpStatus status, String msg, HttpServletRequest req,
            List<ErroResponse.CampoInvalido> campos) {
        return ResponseEntity.status(status).body(corpo(status, msg, req.getRequestURI(), campos));
    }

    private ErroResponse corpo(HttpStatus status, String msg, String caminho,
                               List<ErroResponse.CampoInvalido> campos) {
        return new ErroResponse(Instant.now(), status.value(), status.getReasonPhrase(),
                msg, caminho, campos);
    }

    private String path(WebRequest request) {
        return ((ServletWebRequest) request).getRequest().getRequestURI();
    }
}
```

> **DICA** As assinaturas dos métodos sobrescritos (`handleMethodArgumentNotValid`, `handleExceptionInternal`) podem variar de leve entre versões do Spring. Se a IDE reclamar do `@Override`, use o autocompletar para ver a assinatura da sua versão.

Resposta resultante de uma validação:

```
{
  "timestamp": "2026-11-05T21:10:03Z",
  "status": 400,
  "erro": "Bad Request",
  "mensagem": "Dados inválidos",
  "caminho": "/api/v1/livros",
  "campos": [
    { "campo": "titulo", "mensagem": "O título é obrigatório" },
    { "campo": "ano", "mensagem": "Ano inválido" }
  ]
}
```

> **ATENÇÃO** No `500`, **nunca** devolva a mensagem da exceção ou o *stack trace* ao cliente (vaza detalhes internos e abre brechas de segurança). Registre no log e responda com uma mensagem genérica.

Mapeamento recomendado

Situação	Status
JSON malformado, validação falhou, parâmetro inválido	`400`
Recurso não existe	`404`
Regra de negócio violada (livro indisponível, limite excedido, e-mail duplicado) ou conflito de versão (`OptimisticLockingFailureException`)	`409` (ou `422`, se o time preferir)
Credenciais inválidas / não autenticado	`401`
Autenticado, mas sem permissão (`AccessDeniedException`)	`403`
Falha inesperada	`500`

A hierarquia de exceções de domínio (`RecursoNaoEncontradoException` → `404`, `RegraDeNegocioException` → `409`) definida acima é o que mantém o handler enxuto: criar uma nova exceção de negócio **não exige** um novo handler, basta estender a classe-base certa.

11.4 Documentação com OpenAPI (Swagger)

O `springdoc-openapi` gera a documentação a partir do código:

```
<dependency>
  <groupId>org.springdoc</groupId>
  <artifactId>springdoc-openapi-starter-webmvc-ui</artifactId>
  <version>VERSAO_COMPATIVEL</version>    <!-- substitua pela versão compatível (veja o aviso) -->
</dependency>
```

ATENÇÃO **Compatibilidade de versões.** A série 2.x do springdoc é a do Spring Boot 3. Para o **Spring Boot 4**, a discussão oficial do projeto aponta a série 3.x (3.0.0). Pesquise `springdoc-openapi-starter-webmvc-ui` no Maven Central, leia o README do projeto e escolha a versão que corresponde ao seu Boot. Não existe um número certo para todos os casos.

Sem configurar mais nada, você ganha:

- Swagger UI (página interativa para testar): `http://localhost:8080/swagger-ui.html`
- Especificação em JSON: `http://localhost:8080/v3/api-docs`

Enriqueça com anotações:

```
@Tag(name = "Livros", description = "Operações sobre o acervo")
@RestController
public class LivroController {

    @Operation(summary = "Cadastra um livro")
    @ApiResponses({
        @ApiResponse(responseCode = "201", description = "Livro criado"),
        @ApiResponse(responseCode = "400", description = "Dados inválidos")
    })
    @PostMapping
    public ResponseEntity<LivroResponse> criar(@Valid @RequestBody LivroRequest r) { ... }
}
```

DICA Em produção, restrinja ou desative o Swagger UI conforme a política da empresa.

11.5 Paginação, filtros e logs

Paginação padronizada

Devolva um envelope com metadados, não só a lista:

```
public record PaginaResponse<T>(List<T> itens, int pagina, int tamanho,
                                long total, int totalPaginas) {
    public static <T> PaginaResponse<T> de(Page<T> p) {
        return new PaginaResponse<>(p.getContent(), p.getNumber(), p.getSize(),
                p.getTotalElements(), p.getTotalPages());
    }
}
```

Limite o tamanho máximo da página (`spring.data.web.pageable.max-page-size=50`) para impedir que o cliente peça 1 milhão de linhas.

Logs com SLF4J

Use logging, nunca `System.out.println`.

```
private static final Logger log = LoggerFactory.getLogger(EmprestimoService.class);

log.info("Empréstimo realizado: usuario={}, livro={}", usuarioId, livroId);
log.warn("Tentativa de emprestar livro indisponível: livro={}", livroId);
log.error("Falha ao processar devolução", ex);
```

Nível	Uso
`ERROR`	Falha que exige atenção
`WARN`	Situação anormal, mas tratada
`INFO`	Eventos relevantes do negócio
`DEBUG` / `TRACE`	Detalhes para desenvolvimento

> **ATENÇÃO** Nunca registre senhas, tokens ou dados pessoais sensíveis nos logs.

Resumo do capítulo

- **DTOs** (records) separam o contrato da API do modelo interno; entidade nunca sai do Service.
- **Bean Validation** (`@Valid`, `@NotBlank`, `@Email`, `@Min`...) valida formato; regras de negócio ficam no Service; integridade final, no banco.
- `@RestControllerAdvice` centraliza o tratamento de exceções com um **formato de erro padronizado** e o status correto (400, 404, 409, 500).
- No `500`, registre o detalhe no log e devolva mensagem genérica.
- OpenAPI/Swagger documenta e permite testar a API.
- Paginação com envelope e limite máximo; logs com SLF4J nos níveis corretos.

Exercícios teóricos

1. Cite três problemas de devolver a entidade JPA diretamente no Controller.
2. O que é *mass assignment* e como os DTOs ajudam a evitá-lo?
3. Qual a diferença entre `@NotNull`, `@NotEmpty` e `@NotBlank`?
4. Em qual camada validar o formato do e-mail? E a regra "limite de 3 empréstimos"? Por quê?
5. Como funciona o `@RestControllerAdvice`? Que vantagem tem sobre `try/catch` em cada Controller?
6. Por que não devolver o *stack trace* ao cliente?
7. Qual status usar para e-mail duplicado? Justifique.
8. Para que serve limitar o tamanho máximo da página?
9. Cite um exemplo de cada nível de log.

10. Por que `System.out.println` não substitui um logger?

Exercícios práticos

1. Crie `TarefaRequest` e `TarefaResponse` (records) para a API de Tarefas e converta manualmente.
2. Adicione 5 validações diferentes ao `TarefaRequest` e teste com Postman enviando dados inválidos.
3. Crie um `@RestControllerAdvice` que trate `404`, `400` (com a lista de campos) e `500`.
4. Faça uma validação personalizada (por exemplo, `@CpfValido`).
5. Valide um `@PathVariable` com `@Positive` e observe a exceção lançada.
6. Instale o springdoc e documente 3 endpoints com `@Operation`.
7. Troque o conversor manual por **MapStruct** em uma entidade e compare.
8. Adicione logs com SLF4J em um Service e configure o nível por pacote no `application.yml`.

Projeto principal: Sistema Biblioteca

1. Crie os DTOs: `LivroRequest/Response`, `UsuarioRequest/Response`, `EmprestimoRequest`, `EmprestimoResponse` (com nome do usuário e título do livro, sem expor entidades).
2. Aplique **Bean Validation** em todos os Requests (título, autor, ano, e-mail, IDs positivos).
3. Implemente o `GlobalExceptionHandler` (estendendo `ResponseEntityExceptionHandler`) com o mapeamento do `docs/API.md`: crie `RecursoNaoEncontradoException` e `RegraDeNegocioException` como classes-base; `LivroNaoEncontradoException` e `UsuarioNaoEncontradoException` → `404`; `LivroIndisponivelException` e `LimiteEmprestimosException` → `409`; validação → `400`; e-mail duplicado → `409`; conflito de versão (`OptimisticLockingFailureException`) → `409`. Prove com testes que JSON malformado devolve `400` (e não `500`).
4. Responda `PaginaResponse` em `GET /livros` e `GET /emprestimos`, com limite máximo de página.
5. Adicione o **Swagger UI** e documente todos os endpoints.
6. Adicione logs de negócio (`INFO` em empréstimo/devolução, `WARN` em regra violada).
7. Volte aos testes do capítulo 9: complete os testes de Controller com os status `400`, `404` e `409`.
8. Atualize o `docs/API.md` e o README com a versão final do contrato.

Mini-projeto: API de Reservas de Salas

Em outro repositório, implemente do zero **já com as boas práticas deste capítulo**:

- Recursos: `Sala`, `Reserva`; DTOs (records) separados para entrada e saída.
- Validações: horário de término depois do início, `@Future` para data, capacidade positiva.
- Regra de negócio: não permitir reservas sobrepostas na mesma sala (`409` com mensagem clara).
- Tratamento global de erros com o formato padrão e a lista de campos inválidos.
- Swagger UI, paginação e filtros (`?sala=...&data=...`).
- Testes de Service (Mockito) e de Controller (`@WebMvcTest`) para os principais cenários de erro.

12. Segurança básica com Spring Security

Explicação geral

Quase toda API real precisa responder duas perguntas antes de executar uma ação:

- **Autenticação**: *quem é você?* (provar a identidade: login e senha, token)
- **Autorização**: *o que você pode fazer?* (permissões: um aluno não deve remover livros)

O **Spring Security** é o framework do ecossistema Spring para isso. Ele funciona como uma **cadeia de filtros** que intercepta toda requisição **antes** de ela chegar ao Controller. Ao adicionar a dependência, a aplicação inteira fica protegida por padrão (tudo exige login), e você configura o que é público e o que é restrito.

Você não precisa virar especialista em segurança para ser júnior, mas precisa conhecer o básico: **nunca guardar senha em texto puro**, entender o que é um **token JWT**, proteger endpoints por perfil e evitar as falhas mais comuns.

```
Requisição → [Filtro de segurança: o token é válido? tem permissão?] → Controller → Service
                          ↓ não
                  401 (não autenticado) ou 403 (sem permissão)
```

Conceito	Significado
Autenticação (*authentication*)	Verificar a identidade
Autorização (*authorization*)	Verificar permissões (papéis/roles)
Principal	O usuário autenticado na requisição atual
Role / Authority	Perfil de acesso (`ADMIN`, `BIBLIOTECARIO`, `LEITOR`)
Hash	Transformação irreversível de senha
Token	Credencial enviada a cada requisição (como JWT)

12.1 Senhas: o que nunca fazer

> **ATENÇÃO** **Nunca** armazene senha em texto puro, nem com criptografia reversível, nem com hashes rápidos como MD5 ou SHA-1. Use um algoritmo de hash **lento e com sal** (*salt*), feito para senhas: **BCrypt** (ou Argon2).

O Spring Security fornece o `BCryptPasswordEncoder`:

```
@Bean
public PasswordEncoder passwordEncoder() {
    return new BCryptPasswordEncoder();
}

// ao cadastrar:
String hash = passwordEncoder.encode(request.senha());     // $2a$10$N9qo8uLOickgx2ZMRZoMy...

// ao autenticar:
boolean confere = passwordEncoder.matches(senhaDigitada, hashGuardado);
```

O mesmo texto gera hashes diferentes a cada vez (por causa do sal), e não existe "descriptografar": só comparar com `matches`. Na entidade, guarde o hash em uma coluna `senha_hash`, e **nunca** coloque esse campo em DTOs de resposta.

12.2 Configurando o Spring Security

Dependência:

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-security</artifactId>
</dependency>
```

Uma configuração moderna (Spring Security 6) define as regras por URL e verbo:

```
@Configuration
@EnableWebSecurity
@EnableMethodSecurity
public class SecurityConfig {

    @Bean
    public SecurityFilterChain filterChain(HttpSecurity http, TokenService tokens) throws Exception {
        http
            .csrf(csrf -> csrf.disable())                       // API stateless com token (ver nota)
            .sessionManagement(s -> s.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
            .exceptionHandling(e -> e
                // sem token → 401
                .authenticationEntryPoint(new HttpStatusEntryPoint(HttpStatus.UNAUTHORIZED)))
            .authorizeHttpRequests(auth -> auth
                .requestMatchers("/api/v1/auth/**").permitAll()
                .requestMatchers("/swagger-ui/**", "/v3/api-docs/**").permitAll()
                .requestMatchers("/actuator/health").permitAll()
                .requestMatchers(HttpMethod.GET, "/api/v1/livros/**").authenticated()
                .requestMatchers(HttpMethod.POST, "/api/v1/livros/**")
                    .hasAnyRole("ADMIN", "BIBLIOTECARIO")
                .requestMatchers(HttpMethod.DELETE, "/api/v1/**").hasRole("ADMIN")
                .anyRequest().authenticated())
            .addFilterBefore(new JwtAuthFilter(tokens), UsernamePasswordAuthenticationFilter.class);
        return http.build();
    }
}
```

Observações:

- **Stateless**: a API não guarda sessão no servidor; cada requisição traz o token.
- **401 explícito**: sem configurar o `authenticationEntryPoint`, uma requisição sem credenciais pode receber `403` em vez de `401`, dependendo dos mecanismos de login ativos. Configure-o e **teste** (seção 12.6).
- **O filtro JWT não é um** `@Component`: criamos com `new JwtAuthFilter(tokens)` dentro da configuração. Se ele fosse um bean, o Spring Boot também o registraria no contêiner de servlets, e ele rodaria **duas vezes** por requisição.
- **CSRF**: proteção contra requisições forjadas a partir de navegadores que usam *cookies* de sessão. Em APIs stateless que usam o token no cabeçalho `Authorization`, costuma-se desabilitar. Se você usar cookies, **mantenha ligado**.
- `hasRole("ADMIN")` espera a autoridade `ROLE_ADMIN` (o prefixo `ROLE_` é adicionado automaticamente).
- A **ordem** das regras importa: a primeira que casar vale, e `anyRequest()` fica por último. Prefira "negar por padrão" e liberar o mínimo.

Autorização por método

Com `@EnableMethodSecurity`, você protege também no Service:

```
@PreAuthorize("hasRole('ADMIN')")
```

```
public void remover(Long id) { ... }

// principal = UsuarioAutenticado
@PreAuthorize("hasRole('ADMIN') or #usuarioId == authentication.principal.id")
public List<EmprestimoResponse> historico(Long usuarioId) { ... }
```

Para `authentication.principal.id` funcionar, o *principal* precisa ser um objeto com o `id` (e não apenas o e-mail em `String`). Crie uma classe simples e use-a no filtro (seção 12.3):

```
public class UsuarioAutenticado {
    private final Long id;
    private final String email;
    public UsuarioAutenticado(Long id, String email) { this.id = id; this.email = email; }
    public Long getId() { return id; }          // SpEL lê "principal.id" através do getter
    public String getEmail() { return email; }
}
```

(`#usuarioId` refere-se ao parâmetro do método pelo nome; o Spring Boot compila com `-parameters` por padrão, então isso funciona.)

A segunda regra mostra um ponto crucial: **um usuário só deve ver os próprios dados**, a não ser que seja administrador. Esquecer essa checagem é a falha mais comum (veja 12.5).

12.3 JWT: autenticação por token

JWT (*JSON Web Token*) é um token assinado que carrega informações (*claims*) do usuário. O fluxo:

```
1. POST /api/v1/auth/login  {email, senha}
2. Servidor confere a senha (BCrypt) → gera um JWT assinado → devolve ao cliente
3. Cliente envia em toda requisição:  Authorization: Bearer <token>
4. Filtro valida a assinatura e a validade → define o usuário autenticado → segue para o Controller
```

Um JWT tem três partes separadas por ponto (`cabeçalho.payload.assinatura`), codificadas em Base64URL (uma variante do Base64 segura para URLs):

```
// payload (claims)
{ "sub": "ana@x.com", "roles": ["LEITOR"], "iat": 1793900000, "exp": 1793903600 }
```

> **ATENÇÃO** O payload é apenas **codificado**, não criptografado: qualquer pessoa pode lê-lo. **Nunca** coloque senhas ou dados sensíveis dentro do token. A assinatura garante que ele não foi **alterado**, não que seja secreto.

Dependências (biblioteca `jjwt`)

```
<dependency>
  <groupId>io.jsonwebtoken</groupId><artifactId>jjwt-api</artifactId>
  <version>0.12.6</version>       <!-- exemplo: confirme a versão atual -->
</dependency>
<dependency>
  <groupId>io.jsonwebtoken</groupId><artifactId>jjwt-impl</artifactId>
  <version>0.12.6</version><scope>runtime</scope>
</dependency>
<dependency>
  <groupId>io.jsonwebtoken</groupId><artifactId>jjwt-jackson</artifactId>
  <version>0.12.6</version><scope>runtime</scope>
</dependency>
```

Os exemplos abaixo usam a API da série **0.12** do jjwt (`Jwts.builder().subject(...)`, `Jwts.parser().verifyWith(...)`); versões mais antigas têm outra API. Verifique também a compatibilidade do `jjwt-jackson` com a versão do Jackson do seu Spring Boot (o Boot 4 usa Jackson 3); se houver conflito, consulte a documentação do jjwt para o módulo de serialização adequado.

Gerando e validando

```
@Service
public class TokenService {

    private final SecretKey chave;
    private final long validadeMs;

    public TokenService(@Value("${seguranca.jwt.segredo}") String segredo,
                        @Value("${seguranca.jwt.validade-minutos:60}") long minutos) {
        this.chave = Keys.hmacShaKeyFor(segredo.getBytes(StandardCharsets.UTF_8));
        this.validadeMs = minutos * 60_000;
    }

    public String gerar(Long id, String email, List<String> roles) {
        Date agora = new Date();
        return Jwts.builder()
                .subject(email)
                .claim("id", id)
                .claim("roles", roles)
                .issuedAt(agora)
                .expiration(new Date(agora.getTime() + validadeMs))
                .signWith(chave)
                .compact();
    }

    public Claims validar(String token) {          // lança exceção se inválido ou expirado
        return Jwts.parser().verifyWith(chave).build().parseSignedClaims(token).getPayload();
    }
}
```

O segredo vem de **variável de ambiente** (`SEGURANCA_JWT_SEGREDO`), com no mínimo 32 caracteres aleatórios, e jamais vai para o Git.

O filtro que lê o token

```
// Sem @Component: é instanciado em SecurityConfig (evita execução duplicada)
public class JwtAuthFilter extends OncePerRequestFilter {

    private final TokenService tokens;
    public JwtAuthFilter(TokenService tokens) { this.tokens = tokens; }

    @Override
    protected void doFilterInternal(HttpServletRequest req, HttpServletResponse res,
                                    FilterChain chain)
            throws ServletException, IOException {

        String header = req.getHeader("Authorization");
        if (header != null && header.startsWith("Bearer ")) {
            try {
                Claims claims = tokens.validar(header.substring(7));
                List<SimpleGrantedAuthority> autoridades = ((List<?>) claims.get("roles")).stream()
                        .map(r -> new SimpleGrantedAuthority("ROLE_" + r)).toList();
                Long id = ((Number) claims.get("id")).longValue();
                var principal = new UsuarioAutenticado(id, claims.getSubject());
                var auth = new UsernamePasswordAuthenticationToken(principal, null, autoridades);
                SecurityContextHolder.getContext().setAuthentication(auth);
            } catch (JwtException e) {
                // token inválido: segue sem autenticação → o Spring responderá 401
            }
        }
        chain.doFilter(req, res);
    }
}
```

O endpoint de login

```
@PostMapping("/login")
public TokenResponse login(@Valid @RequestBody LoginRequest req) {
    Usuario u = usuarios.findByEmail(req.email())
            .filter(x -> encoder.matches(req.senha(), x.getSenhaHash()))
```

```
                .orElseThrow(() -> new CredenciaisInvalidasException());      // mensagem genérica!
        String token = tokenService.gerar(u.getId(), u.getEmail(), List.of(u.getPerfil().name()));
        return new TokenResponse(token);
    }
```

Observe: erro de e-mail e erro de senha devolvem **a mesma mensagem** ("Credenciais inválidas"). Dizer "e-mail não existe" ajuda atacantes a descobrir contas válidas.

Access token e refresh token

Tokens de acesso devem ter vida **curta** (15 a 60 minutos). Para não obrigar login a toda hora, usa-se um *refresh token* de vida longa, guardado com mais cuidado, que apenas gera novos tokens de acesso. Para um primeiro projeto, o token único com expiração curta já é suficiente; conheça o conceito para entrevistas.

12.4 Perfis de acesso na Biblioteca

Perfil	Pode
`LEITOR`	Consultar livros, ver os **próprios** empréstimos
`BIBLIOTECARIO`	Tudo do leitor + cadastrar livros/usuários, emprestar e devolver
`ADMIN`	Tudo + remover registros e gerir perfis

Quem cria contas? Defina uma regra e documente: o cadastro público (`POST /api/v1/auth/registro`, aberto) sempre cria `LEITOR`; apenas um `ADMIN` pode criar ou promover `BIBLIOTECARIO` e `ADMIN`. Nunca aceite o campo `perfil` vindo do cliente no cadastro público (seria *mass assignment*: qualquer um se cadastraria como administrador).

Armazene o perfil como `enum Perfil` (`@Enumerated(STRING)`), e a coluna `senha_hash` na tabela `usuario` (nova migração Flyway).

12.5 As falhas mais comuns (OWASP)

A **OWASP Top 10** lista os riscos mais frequentes em aplicações web. Os que mais afetam uma API júnior:

Falha	O que é	Como evitar
Controle de acesso quebrado	Usuário A acessa dados do usuário B (`GET /usuarios/2/emprestimos` sendo o usuário 3)	Sempre checar **dono** do recurso e perfil, no Service
Falhas criptográficas	Senhas em texto puro, segredos no código	BCrypt, variáveis de ambiente, HTTPS
Injeção	SQL Injection por concatenação de strings	Parâmetros nomeados, JPA/JPQL com `:param`
Exposição excessiva de dados	Devolver campos internos (hash, tokens)	DTOs de resposta enxutos
Configuração insegura	Swagger e Actuator abertos, `500` com stack trace	Restringir endpoints e esconder erros
Falta de limitação de taxa	Força bruta no login	Limitar tentativas (*rate limiting*, ex.: Bucket4j)
Dependências vulneráveis	Bibliotecas antigas com falhas conhecidas	Atualizar; Dependabot no GitHub

> **DICA** Regra de ouro: **valide tudo que vem do cliente, negue por padrão, exponha o mínimo e não confie em nada que não seja verificado no servidor**. Esconder um botão no front-end não protege nada.

CORS

Se um front-end em outro domínio (ex.: `localhost:3000`) chamará a API, configure o CORS de forma **restritiva**, listando as origens permitidas. Evite `allowedOrigins("*")` em produção, especialmente com credenciais.

```
@Bean
CorsConfigurationSource cors() {
    var cfg = new CorsConfiguration();
    cfg.setAllowedOrigins(List.of("http://localhost:3000"));
    cfg.setAllowedMethods(List.of("GET", "POST", "PUT", "PATCH", "DELETE"));
    cfg.setAllowedHeaders(List.of("Authorization", "Content-Type"));
    var src = new UrlBasedCorsConfigurationSource();
    src.registerCorsConfiguration("/**", cfg);
    return src;
}
```

12.6 Testando segurança

```
@WebMvcTest(LivroController.class)
@Import(SecurityConfig.class)
class LivroControllerSegurancaTest {

    @Autowired private MockMvc mvc;
    @MockitoBean private LivroService livroService;     // o Controller depende dele
    // o SecurityConfig/filtro dependem dele (não vem no @WebMvcTest)
    @MockitoBean private TokenService tokenService;

    @Test
    void semTokenDeveRetornar401() throws Exception {
        mvc.perform(get("/api/v1/livros")).andExpect(status().isUnauthorized());
    }

    @Test
    @WithMockUser(roles = "LEITOR")
    void leitorNaoPodeCadastrarLivro() throws Exception {
        mvc.perform(post("/api/v1/livros").contentType(MediaType.APPLICATION_JSON).content("{}"))
           .andExpect(status().isForbidden());
    }

    @Test
    @WithMockUser(roles = "BIBLIOTECARIO")
    void bibliotecarioPodeCadastrarLivro() throws Exception { /* espera 201 */ }
}
```

Se o teste de `401` devolver `403`, revise o `authenticationEntryPoint` da seção 12.2. Teste sempre **os dois lados**: quem tem permissão consegue, quem não tem recebe `403`, e quem não se identificou recebe `401`.

Resumo do capítulo

- **Autenticação** = quem é; **autorização** = o que pode. Sem token válido, `401`; sem permissão, `403`.
- Senhas: sempre **BCrypt** (hash lento com sal), nunca texto puro; `matches` para comparar.
- Spring Security funciona como cadeia de filtros; configure regras com `SecurityFilterChain`, **negue por padrão**.
- **JWT**: login devolve um token assinado, enviado como `Authorization: Bearer ...`; o payload é legível, então nada sensível dentro; segredo em variável de ambiente; validade curta.
- Autorização por perfil (`hasRole`) e por **dono do recurso** (`@PreAuthorize`).

- Falhas comuns (OWASP): controle de acesso quebrado, injeção, exposição de dados, configuração insegura, dependências desatualizadas.
- Teste os cenários `401`, `403` e sucesso.

Exercícios teóricos

1. Qual a diferença entre autenticação e autorização? Dê um exemplo de cada na Biblioteca.
2. Qual a diferença entre `401` e `403`?
3. Por que usar BCrypt e não SHA-256 simples ou criptografia reversível para senhas?
4. O que é "sal" (*salt*) e por que o mesmo texto gera hashes diferentes?
5. O que significa uma API ser *stateless*? Que vantagem traz?
6. Quais são as três partes de um JWT? Por que não se coloca senha no payload?
7. Por que a mensagem de erro do login deve ser a mesma para e-mail e senha incorretos?
8. O que é controle de acesso quebrado (IDOR)? Dê um exemplo e como preveni-lo.
9. Quando é seguro desabilitar CSRF? Quando não é?
10. Por que `allowedOrigins("*")` é arriscado em produção?
11. Por que o segredo do JWT não pode ficar no código ou no Git?

Exercícios práticos

1. Adicione `spring-boot-starter-security` à API de Tarefas, veja a senha gerada no log e acesse com Postman usando Basic Auth.
2. Cadastre usuários com senha em BCrypt e confirme no banco que o hash é diferente para a mesma senha.
3. Implemente `POST /auth/login` que devolve um JWT, e um filtro que valide o token.
4. Proteja os endpoints: `GET` para autenticados, `POST` e `PUT` para `EDITOR`, `DELETE` para `ADMIN`.
5. Cole um JWT em jwt.io e observe o payload (confirme que ele é legível).
6. Expire o token em 1 minuto, espere e confirme o `401`.
7. Escreva testes `@WebMvcTest` com `@WithMockUser` para `401`, `403` e sucesso.
8. Crie `GET /tarefas/minhas` que devolve só as tarefas do usuário autenticado e prove, com um teste, que o usuário B não acessa as do A.

Projeto principal: Sistema Biblioteca

1. Adicione `senha_hash` e `perfil` (`enum Perfil`: `LEITOR`, `BIBLIOTECARIO`, `ADMIN`) em `usuario` com **migração Flyway** `V5`.
2. Cadastro de usuário com senha validada (tamanho mínimo) e armazenada com **BCrypt**; a resposta nunca inclui a senha.
3. `POST /api/v1/auth/login` devolvendo JWT (validade de 60 minutos); segredo via variável de ambiente no `compose.yaml`/`.env`.
4. Aplique a tabela de perfis da seção 12.4 no `SecurityConfig`.
5. Garanta, com `@PreAuthorize` no Service, que um `LEITOR` só veja o **próprio** histórico de empréstimos.
6. Libere sem login apenas: `/auth/**`, Swagger (em dev) e `/actuator/health`. Adicione o botão **Authorize** (Bearer) no Swagger UI.

7. Escreva testes de segurança: `401` sem token, `403` para perfil errado, `200/201` para perfil correto, e o teste do "leitor tentando ver o histórico de outro".
8. Crie um usuário `ADMIN` inicial por migração/variável de ambiente (nunca com senha fixa em produção) e documente no README.

Mini-projeto: Cofre de Notas Pessoais

Em outro repositório, crie uma API em que cada usuário guarda suas notas privadas:

- Cadastro e login com BCrypt e JWT; perfis `USER` e `ADMIN`.
- CRUD de notas (`titulo`, `conteudo`, `criadaEm`) **sempre filtrado pelo dono**; qualquer tentativa de acessar nota alheia retorna `404` (para não revelar a existência).
- `ADMIN` consegue listar usuários, mas **nunca** o conteúdo das notas.
- Limite de 5 tentativas de login por e-mail a cada 10 minutos (rate limit simples em memória).
- Testes cobrindo: sem token, token expirado, nota de outro usuário, e perfil insuficiente.
- README com um pequeno "modelo de ameaças": 5 ataques possíveis e como o projeto os mitiga.

13. Projeto principal: do código ao portfólio

Explicação geral

Chegou a hora de juntar tudo. Seu roadmap reserva as **semanas 11 e 12** para o projeto principal: um sistema completo para o portfólio. Ao longo dos capítulos você já construiu as camadas separadamente; aqui o objetivo é **elevar o Sistema Biblioteca ao nível de um projeto profissional** e transformá-lo na peça central da sua candidatura.

Um projeto de portfólio forte não precisa ser enorme. Precisa mostrar, de forma verificável:

- **Domínio do essencial**: Java moderno, Spring Boot em camadas, JPA, PostgreSQL.
- **Qualidade**: testes automatizados, tratamento de erros, validação, segurança básica.
- **Operação**: roda com um comando (Docker Compose), tem pipeline de CI.
- **Comunicação**: README claro, API documentada, histórico de commits limpo.

> **DICA** Um recrutador técnico gasta poucos minutos no seu repositório. Ele olha o README, a estrutura de pastas, alguns commits e se há testes. Facilite tudo isso.

13.1 Escopo final do Sistema Biblioteca

Entidades (conforme o roadmap)

Entidade	Principais atributos	Relacionamentos
`Usuario`	id, nome, email, senhaHash, perfil, ativo	1–N `Emprestimo`, 1–N `Reserva`
`Livro`	id, titulo, isbn, ano, quantidadeTotal, quantidadeDisponivel, version	N–N `Autor`, 1–N `Emprestimo`
`Autor`	id, nome, nacionalidade	N–N `Livro`
`Emprestimo`	id, usuario, livro, dataEmprestimo, dataPrevista, dataDevolucao, status, multa	N–1 `Usuario`, N–1 `Livro`
`Reserva`	id, usuario, livro, criadaEm, status	N–1 `Usuario`, N–1 `Livro`

Repare na evolução: o livro agora tem **quantidade de exemplares** (`quantidadeTotal`, `quantidadeDisponivel`) em vez de um simples booleano, o que se aproxima de uma biblioteca real.

Funcionalidades e regras

Do roadmap: cadastro, consulta, atualização e exclusão; empréstimos; devoluções; verificação de disponibilidade; buscas. Regras para completar:

Regra	Detalhe
Limite de empréstimos	No máximo 3 ativos por usuário (`409` ao exceder)
Prazo	14 dias; configurável por propriedade
Multa	R$ 1,50 por dia de atraso, calculada na devolução com `BigDecimal`

Regra	Detalhe
Bloqueio por multa	Usuário com multa pendente ou empréstimo atrasado não pode pegar outro livro
Disponibilidade	Só empresta se `quantidadeDisponivel > 0`; decrementa ao emprestar e incrementa ao devolver
Reserva	Se não houver exemplar, o usuário entra na fila de reserva do livro (FIFO)
Atendimento da reserva	Ao devolver, o primeiro da fila é marcado como `DISPONIVEL_PARA_RETIRADA` por 48 h
Exclusão	Não é possível excluir livro/usuário com empréstimo ativo (`409`); prefira "inativar" a apagar (*soft delete*)
Concorrência	Dois empréstimos simultâneos do último exemplar: só um passa (`@Version` ou bloqueio)

Migração de transição do modelo (do capítulo 8 para a versão 1.0)

O modelo do capítulo 8 tem **um exemplar por livro** (`disponivel` booleano), `autor` como texto e o índice único parcial de empréstimo ativo. O modelo final tem vários exemplares e a entidade `Autor`. Essa mudança é a parte mais delicada do projeto e deve ser feita por **uma nova migração Flyway** (nunca editando as antigas). O número `V` depende das migrações que você já tem (aqui, `V6`):

```
-- V6__evolucao_modelo_v1.sql

-- 1) Vários exemplares por livro (preserva o que já estava emprestado)
ALTER TABLE livro ADD COLUMN quantidade_total      INTEGER NOT NULL DEFAULT 1;
ALTER TABLE livro ADD COLUMN quantidade_disponivel INTEGER NOT NULL DEFAULT 1;
UPDATE livro SET quantidade_disponivel = CASE WHEN disponivel THEN 1 ELSE 0 END;
ALTER TABLE livro ADD CONSTRAINT ck_livro_quantidade
    CHECK (quantidade_disponivel >= 0 AND quantidade_disponivel <= quantidade_total);
ALTER TABLE livro DROP COLUMN disponivel;

-- 2) O índice do capítulo 8 impediria emprestar o segundo exemplar: remova-o
DROP INDEX uq_livro_emprestimo_ativo;

-- 3) Autor como entidade (N-N), migrando os textos que já existem
CREATE TABLE autor (
    id    BIGSERIAL PRIMARY KEY,
    nome  VARCHAR(120) NOT NULL UNIQUE
);
CREATE TABLE livro_autor (
    livro_id  BIGINT NOT NULL REFERENCES livro(id),
    autor_id  BIGINT NOT NULL REFERENCES autor(id),
    PRIMARY KEY (livro_id, autor_id)
);
INSERT INTO autor (nome) SELECT DISTINCT autor FROM livro;
INSERT INTO livro_autor (livro_id, autor_id)
    SELECT l.id, a.id FROM livro l JOIN autor a ON a.nome = l.autor;
ALTER TABLE livro DROP COLUMN autor;

-- 4) Prazo, multa e inativação (soft delete)
ALTER TABLE emprestimo ADD COLUMN data_prevista DATE;
UPDATE emprestimo SET data_prevista = data_emprestimo + 14;
ALTER TABLE emprestimo ALTER COLUMN data_prevista SET NOT NULL;
ALTER TABLE emprestimo ADD COLUMN multa NUMERIC(10,2) NOT NULL DEFAULT 0;
ALTER TABLE usuario ADD COLUMN ativo BOOLEAN NOT NULL DEFAULT TRUE;
ALTER TABLE livro   ADD COLUMN ativo BOOLEAN NOT NULL DEFAULT TRUE;

-- 5) Fila de reservas
CREATE TABLE reserva (
    id          BIGSERIAL PRIMARY KEY,
    usuario_id  BIGINT NOT NULL REFERENCES usuario(id),
    livro_id    BIGINT NOT NULL REFERENCES livro(id),
    criada_em   TIMESTAMP NOT NULL DEFAULT now(),
    expira_em   TIMESTAMP,
    status      VARCHAR(30) NOT NULL
);
CREATE INDEX idx_reserva_livro_status ON reserva(livro_id, status);
```

Observe o padrão seguro para adicionar uma coluna `NOT NULL` a uma tabela que já tem dados: adicionar **sem** `NOT NULL`, preencher com `UPDATE` e só então aplicar `SET NOT NULL`. Depois da migração, ajuste as entidades (`Livro` com `quantidadeTotal`/`quantidadeDisponivel` e `Set<Autor>`) e rode a aplicação com `ddl-auto: validate`: se algo divergir, ela não sobe e mostra o que falta.

13.2 Arquitetura e organização

Estrutura de pacotes sugerida (por camada, simples e clara):

```
com.exemplo.biblioteca
├── config          ← SecurityConfig, OpenApiConfig, propriedades
├── controller      ← LivroController, EmprestimoController, AuthController...
├── dto             ← requests e responses (records)
├── exception       ← exceções de domínio e GlobalExceptionHandler
├── model           ← entidades JPA e enums
├── repository      ← interfaces Spring Data
├── security        ← JwtAuthFilter, TokenService
└── service         ← regras de negócio
```

Alternativa mais avançada: organizar **por funcionalidade** (`livro/`, `emprestimo/`, `usuario/`, cada um com seu controller, service e repository). Funciona bem quando o projeto cresce. Escolha uma e seja consistente.

Decisões que valem registrar no README

Mostre que você pensou, anotando brevemente o *porquê* (um "ADR" informal):

- Por que `Emprestimo` é uma entidade de junção e não `@ManyToMany`?
- Por que bloqueio otimista (`@Version`) para a concorrência de exemplares?
- Por que DTOs com records e Flyway em vez de `ddl-auto=update`?
- O que ficou de fora de propósito (e-mail de notificação, front-end) e como evoluir.

13.3 Roteiro de implementação (semanas 11 e 12)

Semana 11: núcleo e qualidade

1. **Dia 1–2:** revise o modelo, crie as migrações finais (`quantidade`, `Reserva`, `multa`, `ativo`).
2. **Dia 3:** regras de empréstimo e devolução (limite, prazo, multa, bloqueios) no `EmprestimoService`.
3. **Dia 4:** fila de reservas e atendimento ao devolver.
4. **Dia 5:** concorrência: `@Version` e teste simulando duas threads.
5. **Fim de semana:** testes de Service cobrindo todos os cenários da tabela de regras.

Semana 12: acabamento profissional

1. **Dia 1:** relatórios (livros mais emprestados, atrasados, usuários com mais empréstimos) com JPQL/SQL.
2. **Dia 2:** paginação, filtros e ordenação nas listagens; revisão de N+1 com `show-sql`.
3. **Dia 3:** pipeline de **CI** (GitHub Actions, seção 13.4) e badge no README.
4. **Dia 4:** revisão de segurança (checklist OWASP do capítulo 12) e do Swagger.
5. **Dia 5:** README final, coleção Postman, dados de demonstração, tag `v1.0.0`.

Testando concorrência em Java

Para provar que apenas um usuário pega o último exemplar, dispare várias threads ao mesmo tempo. Três ferramentas do `java.util.concurrent` bastam: `ExecutorService` (um *pool* de threads),

`CountDownLatch` (uma "largada" que libera todas juntas) e `AtomicInteger` (contador seguro entre threads).

```
@Test
void apenasUmUsuarioPegaOUltimoExemplar() throws Exception {
    int tentativas = idsDosUsuarios.size();
    ExecutorService pool = Executors.newFixedThreadPool(tentativas);
    CountDownLatch largada = new CountDownLatch(1);
    AtomicInteger sucessos = new AtomicInteger();
    AtomicInteger conflitos = new AtomicInteger();
    List<Future<Void>> futuros = new ArrayList<>();

    for (Long usuarioId : idsDosUsuarios) {
        futuros.add(pool.submit(() -> {
            largada.await();                        // todas as threads esperam aqui
            try {
                service.emprestar(usuarioId, livroId);
                sucessos.incrementAndGet();
            } catch (RegraDeNegocioException | OptimisticLockingFailureException e) {
                conflitos.incrementAndGet();         // perdeu a disputa: esperado
            }
            return null;
        }));
    }
    largada.countDown();                             // liberta todas ao mesmo tempo
    for (Future<Void> f : futuros) f.get();          // espera terminar (propaga erros inesperados)
    pool.shutdown();

    assertThat(sucessos.get()).isEqualTo(1);
    assertThat(conflitos.get()).isEqualTo(tentativas - 1);
}
```

Este teste precisa de um banco real (Testcontainers) e **não** deve estar dentro de uma transação de teste (`@Transactional` na classe de teste): cada thread precisa da sua própria transação.

Exemplo: devolução com multa

> **DICA** O trecho abaixo é um **excerto**: os campos `emprestimos`, `clock`, `props`, `reservaService` e `log` são dependências e atributos da classe `EmprestimoService`, omitidos aqui. A regra de acesso (quem pode devolver) fica no `SecurityConfig` e em `@PreAuthorize`, por isso o método não recebe `Authentication`.

```
@Transactional
public EmprestimoResponse devolver(Long emprestimoId) {
    Emprestimo e = emprestimos.findById(emprestimoId)
            .orElseThrow(() -> new EmprestimoNaoEncontradoException(emprestimoId));

    if (e.getStatus() == StatusEmprestimo.DEVOLVIDO) {
        throw new RegraDeNegocioException("Empréstimo já devolvido");
    }

    LocalDate hoje = LocalDate.now(clock);                      // Clock injetado: testável
    long diasAtraso = Math.max(0, ChronoUnit.DAYS.between(e.getDataPrevista(), hoje));
    BigDecimal multa = props.multaPorDia().multiply(BigDecimal.valueOf(diasAtraso));

    e.registrarDevolucao(hoje, multa);                          // regra dentro da entidade
    e.getLivro().devolverExemplar();                            // quantidadeDisponivel++
    reservaService.atenderProxima(e.getLivro());                // fila de reservas

    log.info("Devolução: emprestimo={}, atraso={} dias, multa={}", emprestimoId, diasAtraso, multa);
    return EmprestimoResponse.de(e);
}
```

13.4 Integração contínua com GitHub Actions

CI (*Continuous Integration*) executa automaticamente build e testes a cada `push` e Pull Request. Garante que a `main` nunca quebre e mostra a recrutadores que você trabalha como profissional.

Crie `.github/workflows/ci.yml`:

```
name: CI

on:
  push:
    branches: [ main ]
  pull_request:

jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - name: Configurar Java 21
        uses: actions/setup-java@v4
        with:
          distribution: temurin
          java-version: '21'
          cache: maven

      - name: Build e testes
        run: ./mvnw -B clean verify

      - name: Relatório de cobertura
        uses: actions/upload-artifact@v4
        with:
          name: jacoco
          path: target/site/jacoco
```

Os testes com Testcontainers funcionam no `ubuntu-latest`, que já traz Docker. As versões das *actions* (`@v4`) são exemplos que envelhecem: confira a versão mais recente de cada uma no repositório oficial antes de usar. Adicione o **badge** ao README:

```
![CI](https://github.com/SEU-USUARIO/sistema-biblioteca/actions/workflows/ci.yml/badge.svg)
```

Boas práticas: proteja a branch `main` (exigir CI verde e PR para merge), ative o **Dependabot** para avisar sobre dependências desatualizadas, e nunca coloque segredos no workflow (use *Secrets* do GitHub).

Opcional: publicar a imagem Docker

Com um segundo job, é possível construir e publicar a imagem no **GitHub Container Registry (GHCR)**. Não é obrigatório para o primeiro emprego, mas é um diferencial.

13.5 Checklist de qualidade final

Código

- ☐ Camadas separadas; Controller sem regra de negócio; entidades fora do Controller.
- ☐ DTOs com validação; `GlobalExceptionHandler` com formato de erro padrão.
- ☐ Sem `System.out.println`, sem `catch` vazio, sem código comentado e sem *TODOs* esquecidos.
- ☐ Nomes claros em português **ou** inglês (escolha um e mantenha).
- ☐ Valores monetários com `BigDecimal`; datas com `java.time` e `Clock` injetado.

Dados e infraestrutura

- ☐ Esquema 100% por migrações Flyway; `ddl-auto=validate`.
- ☐ Sem N+1 nas listagens principais; índices nas FKs e colunas filtradas.
- ☐ `docker compose up` sobe banco e API, com `healthcheck` e volume.
- ☐ Nenhum segredo no repositório; `.env.example` versionado.

Testes

- ☐ Services com testes de sucesso **e** de erro (cobertura acima de 80%).
- ☐ Testes de Controller (`401`, `403`, `400`, `404`, `409`) e de Repository.
- ☐ Pelo menos um teste de fluxo completo e um de concorrência.
- ☐ CI verde na `main`.

Documentação

- ☐ README com descrição, tecnologias, como rodar, endpoints e decisões.
- ☐ Swagger acessível; coleção do Postman em `docs/`.
- ☐ Diagrama de entidades (pode ser feito com Mermaid direto no README).
- ☐ Tag `v1.0.0` e *release* no GitHub com notas.

Exemplo de diagrama Mermaid (renderiza no GitHub)

```
erDiagram
    USUARIO ||--o{ EMPRESTIMO : realiza
    LIVRO   ||--o{ EMPRESTIMO : "é emprestado em"
    USUARIO ||--o{ RESERVA : faz
    LIVRO   ||--o{ RESERVA : "é reservado em"
    LIVRO   }o--o{ AUTOR : "escrito por"
```

(Coloque este conteúdo dentro de um bloco de código marcado como `mermaid` no seu README.)

13.6 Estrutura sugerida do README final

1. **Título, badge de CI e descrição** (o problema que resolve, em 2 a 3 linhas).
2. **Demonstração**: GIF/prints do Swagger ou do Postman.
3. **Tecnologias**: Java 21, Spring Boot, JPA/Hibernate, PostgreSQL, Flyway, JUnit, Mockito, Testcontainers, Docker.
4. **Funcionalidades e regras de negócio** (tabela da seção 13.1).
5. **Como executar** (apenas `git clone`, `cp .env.example .env`, `docker compose up -d`) e **como rodar os testes** (`./mvnw verify`).
6. **Endpoints principais** (resumo + link para o Swagger).
7. **Arquitetura**: diagrama de camadas e de entidades.
8. **Decisões técnicas** e **o que aprendi / próximos passos**.
9. **Autor** e contatos (LinkedIn).

Resumo do capítulo

- O projeto de portfólio prova domínio técnico, qualidade, operação e comunicação.
- Evolua a Biblioteca: exemplares, multa, reserva em fila, bloqueios, soft delete e concorrência.
- Estruture em camadas consistentes e registre as decisões no README.
- Use **CI** (GitHub Actions) com `mvn verify`; proteja a `main`; badge no README.
- Entregue com checklist: código limpo, Flyway, Docker Compose, testes, Swagger, README e tag `v1.0.0`.

Exercícios teóricos

1. Por que modelar `Emprestimo` como entidade em vez de `@ManyToMany` entre `Usuario` e `Livro`?

2. Como evitar que dois usuários peguem o último exemplar ao mesmo tempo? Compare as abordagens.
3. Por que usar *soft delete* (campo `ativo`) em vez de apagar registros?
4. O que é integração contínua e que problema ela previne?
5. Por que injetar um `Clock` ajuda a testar a regra de multa?
6. O que um recrutador procura ao abrir seu repositório?
7. Por que manter um `.env.example` no Git, mas nunca o `.env`?

Exercícios práticos

1. Crie o workflow de CI em um dos mini-projetos anteriores e faça o badge ficar verde.
2. Proteja a branch `main` no GitHub exigindo PR e CI verde; abra um PR para testar.
3. Gere o diagrama de entidades da Biblioteca em Mermaid e coloque no README.
4. Escreva um teste de concorrência (duas threads emprestando o último exemplar) e confirme que apenas uma tem sucesso.
5. Peça a um colega (ou use outra pessoa de confiança) para clonar o repositório e seguir **só o README**. Anote onde ele travou e corrija.
6. Rode `docker compose up` em uma máquina "limpa" (ou após `docker system prune -a`) para validar o passo a passo.

Projeto principal: Sistema Biblioteca (versão 1.0)

Entregue a versão final seguindo o roteiro da seção 13.3 e o checklist da 13.5:

1. Implemente `Reserva`, `quantidade` de exemplares, multa com `BigDecimal` e bloqueio por atraso.
2. Resolva a concorrência dos exemplares e prove com teste.
3. Crie os relatórios (mais emprestados, atrasados, ranking de usuários) e exponha em `/api/v1/relatorios/...` apenas para `BIBLIOTECARIO` e `ADMIN`.
4. Configure CI e proteja a `main`.
5. Publique a `v1.0.0` com notas de lançamento.
6. Escreva o README final (seção 13.6) e grave um GIF curto mostrando o fluxo: login → emprestar → devolver com multa.

Mini-projeto: "Clone" de um Sistema do seu Interesse

Para ter um **segundo projeto** no portfólio (muitos recrutadores gostam de ver dois), escolha um domínio que você curta e construa do zero, com tudo que aprendeu, em ~1 semana:

- Opções: gestão de estoque de uma loja, agenda de barbearia/clínica, controle de treinos, marketplace simplificado, sistema de eventos e ingressos.
- Mínimo: 4 entidades com relacionamentos, autenticação JWT com 2 perfis, 1 regra de negócio com concorrência ou cálculo não trivial, migrações Flyway, Docker Compose, CI, Swagger e testes.
- Diferencial: integração com uma API externa (ex.: ViaCEP, geração de PDF/relatório CSV) ou cache com Redis.
- Mantenha a mesma qualidade de README e checklist do projeto principal.

14. Empregabilidade (dezembro)

Explicação geral

Dominar a tecnologia é metade do caminho. A outra metade é **ser encontrado, ser compreendido e passar nas conversas**. Seu roadmap dedica dezembro a isso: currículo, GitHub, LinkedIn, documentação do projeto principal e candidaturas a **estágio** e **Back-end Java Júnior**, com foco em Itajaí e região.

Pense na busca por emprego como um **funil**: muitas candidaturas → algumas triagens → poucas entrevistas técnicas → uma oferta. Cada etapa tem um objetivo diferente, e este capítulo prepara você para todas.

```
Candidaturas  →  Triagem do RH  →  Conversa com o time  →  Teste/Entrevista técnica  →  Oferta
 (currículo,      (LinkedIn,         (comunicação,            (Java, Spring, SQL,
  GitHub)          currículo)         motivação)               lógica, projeto)
```

> **DICA** Número realista: é normal enviar de 50 a 100 candidaturas até conseguir as primeiras entrevistas. Rejeição e silêncio fazem parte do processo e **não** medem seu valor. Mantenha constância e melhore a cada rodada.

14.1 Currículo de uma página

Para júnior e estágio, **uma página** basta. Recrutadores leem em segundos.

Estrutura

1. **Cabeçalho**: nome, cidade/UF, e-mail profissional, telefone, LinkedIn e GitHub (links clicáveis).
2. **Resumo** (2 a 3 linhas): quem você é e o que busca.
3. **Tecnologias**: agrupadas (Linguagens, Frameworks, Banco de dados, Ferramentas).
4. **Projetos**: o coração do seu currículo, já que você ainda tem pouca experiência.
5. **Formação**: curso, instituição, previsão de conclusão.
6. **Experiências** (se houver, mesmo que não sejam de TI) e **Idiomas**.

Exemplo de resumo

> **DICA** Estudante de [curso] em [instituição], com foco em desenvolvimento back-end Java. Construí APIs REST com Spring Boot, JPA e PostgreSQL, com testes automatizados, Docker e CI. Busco estágio ou vaga júnior para evoluir em ambiente colaborativo.

Como descrever projetos (fórmula: ação + tecnologia + resultado)

Em vez de "Sistema de biblioteca em Java", escreva:

Sistema de Biblioteca (API REST) · Java 21, Spring Boot, PostgreSQL, Docker · [link do GitHub]

- Desenvolvi API REST em camadas com autenticação JWT e perfis de acesso (Leitor, Bibliotecário, Admin).
- Modelei e versionei o banco com Flyway; resolvi concorrência de empréstimos com bloqueio otimista.
- Cobri os serviços com testes JUnit/Mockito (cobertura acima de 80%) e configurei CI no GitHub Actions.
- Empacotei com Docker Compose: aplicação e banco sobem com um único comando.

Dicas

- **Seja honesto.** Liste só o que você consegue explicar numa entrevista. "Conheço Docker" é aceitável; "especialista em Kubernetes" não.
- Evite barras de nível ("Java: 90%") e foto/dados pessoais desnecessários (CPF, RG, estado civil).
- Use verbos de ação e números quando possível (testes, cobertura, endpoints).
- Salve em **PDF** com nome claro: `Nome-Sobrenome-Backend-Java.pdf`.
- Adapte o resumo e a ordem das tecnologias para cada vaga, destacando o que ela pede.
- E-mail sério (`nome.sobrenome@...`), sem apelidos.
- Ortografia impecável: peça para alguém revisar.

14.2 GitHub como vitrine

Seu GitHub é um segundo currículo, só que **verificável**.

Perfil

- Foto profissional, nome real, bio curta ("*Back-end Java* | *Spring Boot* | *PostgreSQL*"), cidade e link do LinkedIn.
- **README de perfil**: crie um repositório com o mesmo nome do seu usuário e escreva uma apresentação curta, tecnologias e links dos melhores projetos.
- **Fixe (pin) de 4 a 6 repositórios**: o Sistema Biblioteca, o segundo projeto grande e os melhores mini-projetos.

Repositórios

- Cada repositório fixado precisa de **README completo** (veja o capítulo 3 e o 13), descrição e *topics* (`java`, `spring-boot`, `postgresql`, `docker`).
- Commits regulares e legíveis; a "grade verde" de contribuições mostra constância.
- Sem segredos no histórico (revise! senhas, tokens e `.env` expostos eliminam candidatos rapidamente).
- Apague ou deixe privados repositórios vazios, quebrados ou de tutoriais copiados sem personalização.
- Evite *forks* sem contribuição própria ocupando o destaque.

> **DICA** Teste final: abra seu GitHub em uma **janela anônima**, como o recrutador faria. O que ele vê nos primeiros 10 segundos?

14.3 LinkedIn

Em muitas empresas da região, o primeiro contato acontece pelo LinkedIn.

Seção	O que fazer
Foto e capa	Foto nítida e simples; capa neutra ou com tema de tecnologia
Título (headline)	"Desenvolvedor Back-end Java Júnior · Spring Boot · PostgreSQL · Docker" (não só "Estudante")
Sobre	4 a 6 linhas: quem é, tecnologias, o que busca; termine com uma chamada ("Aberto a oportunidades de estágio e júnior")
Experiência/Projetos	Mesmos tópicos do currículo, com link do GitHub
Competências	Java, Spring Boot, SQL, Git, REST, Docker, JUnit (peça recomendações)
Link personalizado	`linkedin.com/in/seu-nome`

Seção	O que fazer
Open to Work	Ative o aviso (visível para recrutadores) com os cargos desejados e a localização

Rede de contatos

- Conecte-se com recrutadores de TI, desenvolvedores e líderes das empresas de **Itajaí, Balneário Camboriú, Navegantes, Itapema, Florianópolis e região**, e conecte-se com professores e colegas.
- Envie uma nota curta e personalizada ao conectar ("*Olá, Ana! Sou estudante de ADS e busco estágio em back-end Java. Gostei do trabalho da [empresa] com [área]. Posso me conectar?*").
- Interaja: comente posts técnicos e publique sobre o que aprende (um post por semana sobre um conceito ou projeto já ajuda muito).
- Acompanhe páginas de empresas locais e ative alertas de vagas com as palavras-chave `Java`, `Back-end`, `Estágio`, `Júnior`, `Spring`.

14.4 Onde procurar e como se candidatar

Canal	Dica
LinkedIn Vagas	Alertas diários; filtros por localização, "Candidatura simplificada" e nível
Gupy, Vagas.com, Catho, Indeed	Plataformas muito usadas por empresas brasileiras; mantenha o perfil completo
Sites das empresas ("Trabalhe conosco")	Comece pela lista de empresas de tecnologia da região
Programas de estágio e *trainee*	Têm prazos fixos: crie lembretes
Comunidades e eventos locais	Meetups, hackathons, grupos de Discord/WhatsApp de TI da região
Indicações	O canal mais eficiente: conte a conhecidos que está buscando
Seu curso/universidade	Setor de carreiras, professores e egressos

Organize as candidaturas

Crie uma planilha (ou quadro) com: empresa, vaga, link, data, status (enviada, triagem, entrevista, teste, recusada, oferta), contatos e próximos passos. **Acompanhe**: uma mensagem educada após 7 a 10 dias sem resposta é aceitável.

Carta de apresentação / mensagem

Curta (5 a 7 linhas), específica para a vaga: por que você se interessa pela empresa, 1 ou 2 projetos relevantes com link, e disponibilidade.

> **ATENÇÃO** Candidatar-se a vagas que pedem 3+ anos de experiência não é perda de tempo se o restante bater, mas **priorize** vagas de estágio e júnior reais. Leia a descrição com atenção: se pede um requisito que você não tem, seja honesto e mostre disposição para aprender.

14.5 Preparação para entrevistas técnicas

Etapas comuns

1. **Triagem com RH** (15 a 30 min): motivação, expectativa salarial, disponibilidade, inglês.

2. **Entrevista técnica** (45 a 90 min): perguntas conceituais, discussão do seu projeto, às vezes exercício ao vivo.
3. **Teste prático** (*take-home*): construir uma pequena API em alguns dias.
4. **Conversa com gestor/cultura**: trabalho em equipe, aprendizado, dúvidas.

Tópicos que costumam cair (com o capítulo de referência)

Área	Perguntas frequentes
POO (cap. 1)	Pilares; classe abstrata vs interface; sobrecarga vs sobrescrita; `equals`/`hashCode`
Java (1 e 2)	`==` vs `equals`; `String` imutável; checked vs unchecked; `List`/`Set`/`Map`; Streams; `Optional`; `final`/`static`
Collections (1)	`ArrayList` vs `LinkedList`; `HashMap` por dentro (hash, colisão); `Comparable` vs `Comparator`
Git (3)	`merge` vs `rebase`; resolver conflito; `revert` vs `reset`; fluxo de PR
REST (5)	Verbos; idempotência; `401` vs `403`; `PUT` vs `PATCH`; `200` vs `201` vs `204`
Spring (6, 11, 12)	IoC/DI; beans e escopo; `@Component`/`@Service`/`@Repository`; injeção por construtor; profiles; `@RestControllerAdvice`; Spring Security e JWT
JPA (7)	`@OneToMany`/`@ManyToOne`; lazy vs eager; **N+1**; `@Transactional`; JPQL
SQL (8)	`JOIN`s; `GROUP BY`/`HAVING`; índices; transações e ACID; `EXPLAIN`
Testes (9)	Unitário vs integração; mock; `@WebMvcTest`; TDD
Docker (10)	Imagem vs contêiner; Dockerfile; volumes; Compose

Falando do seu projeto (muito importante)

Prepare e ensaie em voz alta uma apresentação de **3 minutos** do Sistema Biblioteca:

1. **Problema e escopo**: o que o sistema faz e para quem.
2. **Arquitetura**: camadas, banco, autenticação.
3. **Um desafio técnico** que você resolveu (concorrência de exemplares, N+1, migrações, testes) e **como** decidiu.
4. **O que faria diferente** ou melhoraria (sinal de maturidade).

Esteja pronto para qualquer pergunta sobre **qualquer linha** do projeto. Se você não souber, diga "não sei, mas eu investigaria assim...".

Lógica de programação e exercícios ao vivo

Muitas vezes pedem algo simples, resolvido "pensando em voz alta":

- Inverter uma string, contar palavras, verificar palíndromo ou anagrama.
- Encontrar duplicados em uma lista (com `Set`/`Map`).
- FizzBuzz, número primo, Fibonacci.
- Agrupar e contar elementos com Streams.
- Modelar classes de um cenário (estacionamento, caixa eletrônico).

Os problemas clássicos resolvidos e a explicação de Big-O estão no apêndice (capítulo 15). Treine em plataformas como **HackerRank**, **LeetCode (nível fácil)** ou **Codewars**, de 15 a 30 minutos por dia. Foque nas estruturas básicas (`List`, `Map`, `Set`, `String`, ordenação, recursão), não em algoritmos avançados.

Teste prático (*take-home*)

- Leia o enunciado **duas vezes** e liste os requisitos; em dúvida, pergunte.
- Entregue **menos, mas bem feito**: camadas, validação, tratamento de erros, alguns testes, README e `docker compose`.
- Respeite o prazo e o escopo; não adicione complexidade desnecessária.
- Use commits claros em um repositório (privado, a menos que a empresa peça público).

14.6 Comportamento: o que também decide a vaga

Empresas contratam júnior pelo **potencial**. Demonstre:

- **Vontade de aprender**: conte como estudou, o que errou e como corrigiu.
- **Comunicação**: explicar com clareza e de forma estruturada; perguntar quando não entender.
- **Autonomia com humildade**: tentar resolver sozinho, mas pedir ajuda depois de um tempo razoável (e mostrar o que já tentou).
- **Trabalho em equipe**: abertura para *code review* e feedback.

Perguntas comportamentais comuns (e use o método **STAR**: Situação, Tarefa, Ação, Resultado):

- "Fale sobre você." · "Por que Java/back-end?" · "Conte um problema difícil que resolveu."
- "Como lida com prazos?" · "Como aprende algo novo?" · "Onde se vê em 3 anos?"
- "Por que quer trabalhar aqui?" (pesquise a empresa antes!)

Perguntas para fazer ao entrevistador: como é o processo de *code review*? Que tecnologias o time usa? Como funciona a integração de quem entra? Existe mentoria para júniores?

Inglês

Leitura técnica em inglês é indispensável (documentação, Stack Overflow, erros). Muitas vagas pedem inglês básico/intermediário. Leia documentação em inglês diariamente e prepare uma apresentação de 1 minuto sobre você e seu projeto.

Salário e negociação

Pesquise faixas na região (Glassdoor, LinkedIn Salary, pesquisas salariais de TI e conversas com colegas). Para estágio e júnior, foque em **aprendizado, mentoria e tecnologia do time** tanto quanto no valor. Se perguntarem a pretensão, dê uma faixa razoável e flexível.

14.7 Plano de dezembro (4 semanas)

Semana	Foco	Entregas
1	Vitrine	Currículo (PDF) pronto; LinkedIn completo; GitHub arrumado; README do projeto final revisado; lista de 40 empresas-alvo
2	Candidaturas	20 a 30 candidaturas personalizadas; planilha de acompanhamento; 10 contatos novos no LinkedIn; 1 post técnico
3	Preparação	Revisão dos tópicos da seção 14.5; 5 simulados de entrevista (com colega ou gravando-se); 1 exercício de lógica por dia; ensaio da apresentação de 3 min
4	Constância	Mais 20 a 30 candidaturas; acompanhar respostas; refazer o ponto fraco de cada entrevista; continuar estudando e publicando

Rotina diária sugerida (3 a 4 horas): 1 h candidaturas e networking · 1 h lógica e revisão técnica · 1 a 2 h projeto/estudo.

> **DICA** Depois de dezembro, **não pare de estudar**: o passo seguinte natural é aprofundar em Spring Security, mensageria (RabbitMQ/Kafka), cloud (AWS), microsserviços e observabilidade. Mas só depois de conseguir a primeira oportunidade e aprender como a empresa trabalha.

Resumo do capítulo

- Você está num funil: currículo/GitHub → triagem → conversa → entrevista técnica → oferta; prepare cada etapa.
- Currículo de **uma página**, focado em projetos descritos com ação + tecnologia + resultado; seja honesto.
- **GitHub**: perfil caprichado, 4 a 6 repositórios fixados com README forte, sem segredos no histórico.
- **LinkedIn**: título claro, "Open to Work", rede local ativa e alertas de vagas.
- Candidate-se com constância e organização (planilha), personalizando as mensagens; indicações são o melhor canal.
- Entrevista: domine os fundamentos (Java, POO, REST, Spring, JPA, SQL, testes, Docker), ensaie a apresentação do projeto e treine lógica.
- Comportamento importa: vontade de aprender, comunicação e abertura a feedback.

Exercícios teóricos

1. Por que o currículo de júnior deve destacar projetos? Reescreva "Fiz um sistema de biblioteca" usando a fórmula ação + tecnologia + resultado.
2. Que sinais negativos um recrutador pode encontrar num GitHub? Como corrigir cada um?
3. Qual a diferença entre uma candidatura genérica e uma personalizada? Que vantagem tem a segunda?
4. O que é o método STAR? Aplique-o a uma situação real dos seus estudos.
5. Liste cinco perguntas que você faria ao entrevistador e por quê.
6. Por que é aceitável responder "não sei, mas investigaria assim..."?
7. Quais são as etapas de um processo seletivo típico para júnior?

Exercícios práticos

1. Monte o currículo de uma página e peça revisão a duas pessoas (uma da área e uma de fora).
2. Crie o repositório de README de perfil no GitHub e fixe seus melhores repositórios com *topics* e descrições.
3. Atualize o LinkedIn (título, sobre, projetos, competências, link personalizado, *Open to Work*).
4. Faça a planilha de acompanhamento e cadastre 40 empresas-alvo da região de Itajaí, com o link das vagas.
5. Grave-se (áudio ou vídeo) apresentando o Sistema Biblioteca em 3 minutos; assista e corrija o que ficou confuso.
6. Responda **em voz alta**, sem consultar, 15 perguntas da tabela da seção 14.5 e anote as que travou; estude-as e repita.
7. Resolva 10 exercícios fáceis de lógica (HackerRank/LeetCode) em Java.
8. Faça um simulado de entrevista com um colega: 20 min técnicos + 10 min comportamentais, com feedback.

9. Escreva um post curto no LinkedIn sobre algo que aprendeu (por exemplo, "o que é o problema N+1 e como resolvi").

Projeto principal: Sistema Biblioteca

Prepare o projeto para ser **apresentado**:

1. Revise o README com os olhos de um recrutador (seção 13.6) e confirme que `docker compose up` funciona do zero.
2. Faça um **GIF/vídeo curto** (1 a 2 min) mostrando o fluxo principal e adicione ao README.
3. Revise o histórico do Git: mensagens claras, sem segredos; marque a tag `v1.0.0`.
4. Escreva uma seção "Decisões técnicas e aprendizados" com 5 itens.
5. Prepare um **roteiro de demonstração** (login como bibliotecário → cadastrar livro → emprestar → atrasar e devolver com multa → ver relatório) e ensaie.
6. Crie 10 perguntas que um entrevistador poderia fazer sobre o projeto e escreva as respostas.

Mini-projeto: Kit de Candidatura

Em outro repositório (pode ser privado), monte seu **kit de busca de emprego**:

- `curriculo/` com o PDF e o texto-base para adaptar a cada vaga.
- `candidaturas.csv` (ou planilha) com o funil: empresa, vaga, data, status, aprendizados.
- `entrevistas/` com um arquivo por tema (`java.md`, `spring.md`, `jpa-sql.md`, `rest.md`, `docker.md`, `comportamental.md`) contendo perguntas e **suas** respostas, escritas com suas palavras.
- `logica/` com as soluções dos exercícios resolvidos, organizadas por assunto (Strings, Listas, Maps, Recursão).
- `apresentacao.md` com o roteiro de 3 min do projeto, versão em português e uma versão curta em inglês (1 min).
- Atualize semanalmente; ao final de dezembro, revise o que mais caiu nas entrevistas e reforce.

15. Apêndice: fundamentos que completam a base

Explicação geral

Os capítulos anteriores seguem o seu roadmap. Este apêndice cobre o que **entrevistas de júnior costumam cobrar** e o roadmap não detalha: princípios de projeto (SOLID), estruturas de dados e algoritmos básicos, noções de concorrência, Lombok, extras do Spring e o terminal. Estude em paralelo ao projeto final e durante dezembro.

Não é preciso decorar. O objetivo é reconhecer o problema, saber o raciocínio e conseguir explicar em voz alta.

15.1 SOLID: cinco princípios de design

SOLID é um conjunto de princípios para escrever classes fáceis de mudar e de testar. Em entrevistas, o mais comum é pedirem para explicar cada letra com um exemplo.

Letra	Princípio	Em uma frase
S	Responsabilidade única (*Single Responsibility*)	Uma classe deve ter **um motivo para mudar**
O	Aberto/fechado (*Open/Closed*)	Aberta para extensão, fechada para modificação
L	Substituição de Liskov	Um subtipo deve poder substituir o tipo pai sem quebrar o comportamento
I	Segregação de interfaces	Várias interfaces pequenas são melhores que uma gigante
D	Inversão de dependência	Dependa de **abstrações** (interfaces), não de implementações concretas

S: responsabilidade única

Má ideia: um `EmprestimoService` que calcula multa, envia e-mail, gera PDF e salva no banco. Cada tarefa é um motivo diferente para mudar. Separe: `CalculadoraDeMulta`, `Notificador`, `EmprestimoService` (que orquestra).

O: aberto/fechado

Em vez de um `if/else` que cresce a cada nova regra, crie uma interface e uma classe por regra:

```
public interface RegraDeMulta {
    BigDecimal calcular(long diasAtraso);
}

public class MultaPadrao implements RegraDeMulta {
    public BigDecimal calcular(long dias) {
        return BigDecimal.valueOf(dias).multiply(new BigDecimal("1.50"));
    }
}

public class MultaProfessor implements RegraDeMulta {       // nova regra = nova classe
    public BigDecimal calcular(long dias) {
        return BigDecimal.valueOf(dias).multiply(new BigDecimal("0.75"));
    }
}
```

Adicionar um novo tipo de multa **não** exige alterar o código existente.

L: Liskov

Se `Professor extends Pessoa`, qualquer código que funciona com `Pessoa` deve funcionar com `Professor`. Violação clássica: um subtipo que lança `UnsupportedOperationException` num método do pai, ou que enfraquece uma regra (o pai exige e-mail válido e o filho aceita qualquer texto). Se isso acontece, a herança provavelmente era o modelo errado: prefira composição.

I: segregação de interfaces

Uma interface `Repositorio` com 15 métodos obriga quem a implementa a criar métodos vazios. Prefira papéis pequenos, como `Leitura<T>` e `Escrita<T>`.

D: inversão de dependência

O Service depende de uma **interface**, e o Spring injeta a implementação. É exatamente o que você faz com `LivroRepository` e com `Notificador`:

```
public interface Notificador {
    void avisar(String destinatario, String mensagem);
}

public class EmprestimoService {
    private final Notificador notificador;                  // abstração, não "EmailNotificador"

    public EmprestimoService(Notificador notificador) {     // injeção por construtor
        this.notificador = notificador;
    }
}
```

Nos testes, você passa um *mock* de `Notificador`; em produção, uma implementação que envia e-mail. É por isso que injeção de dependência e testes andam juntos.

> **DICA** Princípios não são leis. Aplicá-los em excesso gera classes demais. Use-os quando o código estiver difícil de mudar ou de testar.

Padrões de projeto que você já usa sem perceber

Padrão	Onde aparece
Singleton	Beans do Spring (um por contexto)
Strategy	`RegraDeMulta` acima; `Comparator`
Factory	`List.of(...)`, `LocalDate.of(...)`, `Optional.of(...)`
Builder	`HttpRequest.newBuilder()`, `Jwts.builder()`, `StringBuilder`
Observer	Eventos do Spring, listeners
Dependency Injection	Todo o Spring
Repository	`JpaRepository`

15.2 Estruturas de dados e algoritmos para entrevistas

Entrevistas de júnior raramente pedem algoritmos avançados. Pedem **raciocínio com `String`, `List`, `Map` e laços**, e que você explique a complexidade.

Complexidade (Big-O) em linguagem simples

Big-O descreve como o tempo cresce quando a entrada cresce (n = tamanho da entrada).

Notação	Nome	Exemplo	Se n dobra, o tempo...
O(1)	Constante	`map.get(chave)` (médio), acesso `array[i]`	não muda
O(log n)	Logarítmica	Busca binária	cresce pouquíssimo
O(n)	Linear	Percorrer uma lista	dobra
O(n log n)	Linearítmica	Ordenação eficiente (`Collections.sort`)	pouco mais que dobra
O(n²)	Quadrática	Dois laços aninhados sobre a mesma lista	quadruplica

Estrutura	Acesso por índice	Busca por valor	Inserir no fim	Observação
`ArrayList`	O(1)	O(n)	O(1) amortizado	Padrão para listas
`LinkedList`	O(n)	O(n)	O(1)	Raramente vale a pena
`HashMap` / `HashSet`	não tem	O(1) médio	O(1) médio	Sem ordem; usa `equals`/`hashCode`
`TreeMap` / `TreeSet`	não tem	O(log n)	O(log n)	Mantém ordenado
`ArrayDeque`	não tem	O(n)	O(1)	Fila e pilha

Problemas clássicos, resolvidos

Em cada um, pense primeiro em **voz alta**: quais são os casos de borda (nulo, vazio, um elemento)? Qual a complexidade?

Inverter uma string e verificar palíndromo.

```
static String inverter(String s) {
    return new StringBuilder(s).reverse().toString();
}

static boolean ehPalindromo(String s) {
    String t = s.toLowerCase().replaceAll("[^\\p{L}\\p{N}]", "");   // só letras e dígitos
    int i = 0, j = t.length() - 1;
    while (i < j) {                                 // dois ponteiros: O(n)
        if (t.charAt(i++) != t.charAt(j--)) return false;
    }
    return true;
}
```

Anagramas (mesmas letras, ordem diferente): ordenar os dois e comparar, O(n log n).

```
static boolean saoAnagramas(String a, String b) {
    char[] x = a.replace(" ", "").toLowerCase().toCharArray();
    char[] y = b.replace(" ", "").toLowerCase().toCharArray();
    Arrays.sort(x);
    Arrays.sort(y);
    return Arrays.equals(x, y);
}
```

Dois números que somam um alvo (*two sum*). A solução ingênua usa dois laços, O(n²). Com um `HashMap` guardando o que já foi visto, vira O(n):

```
static int[] doisNumerosQueSomam(int[] nums, int alvo) {
    Map<Integer, Integer> vistos = new HashMap<>();      // valor -> índice
    for (int i = 0; i < nums.length; i++) {
        Integer j = vistos.get(alvo - nums[i]);          // o complemento já apareceu?
        if (j != null) return new int[] { j, i };
        vistos.put(nums[i], i);
```

```
    }
    return new int[0];                                  // não encontrou
}
```

Primeiro caractere que não se repete (`LinkedHashMap` preserva a ordem de inserção):

```
static Character primeiroNaoRepetido(String s) {
    Map<Character, Integer> cont = new LinkedHashMap<>();
    for (char c : s.toCharArray()) cont.merge(c, 1, Integer::sum);
    return cont.entrySet().stream()
            .filter(e -> e.getValue() == 1)
            .map(Map.Entry::getKey)
            .findFirst()
            .orElse(null);
}
```

FizzBuzz (verifique o múltiplo de 15 **primeiro**):

```
static List<String> fizzBuzz(int n) {
    List<String> r = new ArrayList<>();
    for (int i = 1; i <= n; i++) {
        r.add(i % 15 == 0 ? "FizzBuzz" : i % 3 == 0 ? "Fizz"
                : i % 5 == 0 ? "Buzz" : String.valueOf(i));
    }
    return r;
}
```

Contar palavras com Streams:

```
static Map<String, Long> contarPalavras(String texto) {
    return Arrays.stream(texto.toLowerCase().split("\\W+"))
            .filter(p -> !p.isBlank())
            .collect(Collectors.groupingBy(p -> p, Collectors.counting()));
}
```

Busca binária (a lista precisa estar **ordenada**; descarta metade a cada passo, O(log n)):

```
static int buscaBinaria(int[] ordenado, int alvo) {
    int lo = 0, hi = ordenado.length - 1;
    while (lo <= hi) {
        int meio = lo + (hi - lo) / 2;                  // evita estouro de int
        if (ordenado[meio] == alvo) return meio;
        if (ordenado[meio] < alvo) lo = meio + 1; else hi = meio - 1;
    }
    return -1;
}
```

Recursão (revisão)

Uma função recursiva resolve um problema usando uma versão menor do **mesmo** problema. Precisa de um **caso base** (onde para) e de um **caso recursivo** (onde chama a si mesma com um problema menor).

```
static long fatorial(int n) {
    if (n <= 1) return 1;               // caso base
    return n * fatorial(n - 1);         // caso recursivo
}

static long fibonacci(int n) {          // versão iterativa: O(n); a recursiva ingênua é O(2^n)
    long a = 0, b = 1;
    for (int i = 0; i < n; i++) { long t = a + b; a = b; b = t; }
    return a;
}
```

> **ATENÇÃO** Sem caso base (ou com um caso base que nunca é alcançado), a recursão chama a si mesma até estourar a pilha: `StackOverflowError`. Fibonacci recursivo sem memorização repete o mesmo cálculo exponencialmente; por isso a versão iterativa é preferível.

Como se comportar numa entrevista de lógica

1. **Repita o problema** com suas palavras e confirme as regras e os casos de borda.
2. Proponha primeiro a solução **simples** (mesmo $O(n^2)$), diga a complexidade e depois melhore.
3. **Pense em voz alta**; o entrevistador avalia o raciocínio, não só o resultado.
4. Teste com exemplos: caso normal, vazio, um elemento, repetidos.
5. Se travar, diga o que já tentou e peça uma dica. Isso é aceitável.

15.3 Concorrência básica

Servidores atendem muitas requisições **ao mesmo tempo**, cada uma em uma *thread*. Quando duas threads mexem no mesmo dado sem cuidado, ocorre uma **condição de corrida** (*race condition*). É a origem do problema de dois usuários pegando o último exemplar (capítulos 8 e 13).

O problema

`valor++` parece uma operação só, mas são três passos (ler, somar, gravar). Duas threads podem ler o mesmo valor e uma das somas se perde. O defeito é **intermitente**: em muitas execuções o resultado sai certo por acaso, o que o torna difícil de detectar só com testes:

```
class ContadorInseguro {
    private int valor;
    void incrementar() { valor++; }          // NÃO é seguro entre threads
    int getValor() { return valor; }
}
```

As soluções

```
class ContadorSincronizado {
    private int valor;
    synchronized void incrementar() { valor++; }     // só uma thread por vez neste objeto
    synchronized int getValor() { return valor; }
}

class ContadorAtomico {
    private final AtomicInteger valor = new AtomicInteger();
    void incrementar() { valor.incrementAndGet(); }  // operação atômica, sem bloquear
    int getValor() { return valor.get(); }
}
```

Ferramenta	Para quê
`synchronized`	Exclusão mútua simples em um método ou bloco
`AtomicInteger`, `AtomicLong`	Contadores seguros entre threads
`ConcurrentHashMap`	Mapa seguro para uso concorrente (usado no repositório em memória do capítulo 6)
`ExecutorService`	Pool de threads: você envia tarefas e ele as executa
`CountDownLatch`	"Largada" que sincroniza várias threads (veja o teste do capítulo 13)
`volatile`	Garante visibilidade de uma variável entre threads (não torna `valor++` atômico)

Executando tarefas em paralelo

```
ExecutorService pool = Executors.newFixedThreadPool(4);
List<Future<Integer>> resultados = new ArrayList<>();
for (int i = 1; i <= 4; i++) {
    final int n = i;
    resultados.add(pool.submit(() -> n * n));         // tarefa que devolve um valor
}
```

```
for (Future<Integer> f : resultados) System.out.println(f.get());   // espera e lê o resultado
pool.shutdown();                                                    // sempre encerre o pool
```

E no Spring?

- Os beans são singletons compartilhados por todas as requisições: **não guarde estado mutável por usuário** em atributos de `@Service`.
- No banco, a concorrência se resolve com transações, `@Version` (bloqueio otimista) e restrições do banco (capítulo 8).
- Para tarefas em segundo plano, o Spring oferece `@Async` e `@Scheduled`, mas só os use depois de dominar o básico. Java 21 também trouxe **threads virtuais**, um tema para estudar depois do primeiro emprego.

15.4 Lombok

Lombok gera código repetitivo (getters, construtores, `equals`/`hashCode`, `toString`) por anotações, em tempo de compilação. É muito comum em projetos de empresas, então você vai encontrá-lo.

```
@Getter
@NoArgsConstructor(access = AccessLevel.PROTECTED)     // construtor exigido pela JPA
public class Livro {
    private Long id;
    private String titulo;
}
```

Anotação	Gera
`@Getter` / `@Setter`	Getters e setters
`@NoArgsConstructor` / `@AllArgsConstructor` / `@RequiredArgsConstructor`	Construtores (o último é útil para injeção por construtor)
`@ToString` / `@EqualsAndHashCode`	Os métodos correspondentes
`@Builder`	Padrão Builder
`@Slf4j`	Um campo `log` pronto

> **ATENÇÃO** Cuidados em entidades JPA: evite `@Data` e `@EqualsAndHashCode` com relacionamentos (podem disparar consultas ou laços infinitos). Nesta apostila **não** usamos Lombok para você aprender o código real primeiro; com `record` e getters escritos pela IDE, você já passa sem ele. Para usar, a IDE precisa do plugin e o *annotation processing* ativado.

15.5 Extras do Spring que aparecem no mercado

Cada item abaixo merece uma leitura na documentação oficial **da sua versão**: os detalhes de configuração mudam.

Tema	O que é	Quando usar
Actuator	Endpoints de monitoramento (`/actuator/health`, métricas)	Healthcheck do Docker, observabilidade
`ProblemDetail`	Formato padrão de erro HTTP (RFC 9457, antiga 7807), suportado pelo Spring	Alternativa ao `ErroResponse` próprio
`Specification`	Filtros dinâmicos para JPA (`JpaSpecificationExecutor`)	Busca com vários filtros opcionais combinados

Tema	O que é	Quando usar
Projeções	Consultas que devolvem só alguns campos (interfaces ou records)	Listagens leves, sem carregar a entidade inteira
Cache (`@Cacheable`)	Guarda o resultado de um método	Dados que mudam pouco e são lidos muito
Eventos (`ApplicationEventPublisher`)	Desacopla ações (ex.: enviar e-mail após empréstimo)	Reduzir dependências entre Services
`@Scheduled`	Tarefas agendadas	Marcar empréstimos atrasados toda noite
Perfis e configuração externa	Separar dev/prod	Já visto no capítulo 6

Logs úteis

```
logging:
  level:
    root: INFO
    com.exemplo.biblioteca: DEBUG
    org.hibernate.SQL: DEBUG           # mostra o SQL gerado (equivale a show-sql, via logger)
```

15.6 Terminal e Linux básico

Servidores, contêineres e pipelines de CI rodam Linux. Você precisa se virar no terminal (no Windows, use o WSL ou o Git Bash).

Comando	Para quê
`pwd`, `ls -la`, `cd pasta`	Onde estou, listar arquivos, mudar de pasta
`mkdir -p a/b`, `touch arq`, `cp`, `mv`, `rm -r`	Criar pasta/arquivo, copiar, mover, apagar (cuidado com `rm -r`)
`cat arq`, `less arq`, `head`, `tail -f log`	Ler arquivos e **acompanhar logs ao vivo**
`grep -rn "texto" .`	Procurar texto em arquivos
`chmod +x script.sh`	Dar permissão de execução
`ps aux`, `kill PID`, `lsof -i :8080`	Processos; descobrir quem usa a porta 8080
`curl -i URL`	Chamar uma API
`export VAR=valor`, `env`	Variáveis de ambiente
`ssh usuario@servidor`	Acessar outro computador
Pipe (barra vertical)	Encadear comandos: a saída de um vira a entrada do outro (exemplo logo abaixo)

Exemplo de pipe: `ls -la`, seguido da barra vertical e de `grep java`, lista só os arquivos que contêm "java" no nome. Digitado: ls -la | grep java

Resumo do capítulo

- **SOLID**: uma responsabilidade por classe; estender em vez de modificar; subtipos substituíveis; interfaces pequenas; depender de abstrações.
- **Big-O** descreve o crescimento do tempo. `HashMap` troca memória por velocidade; dois laços aninhados costumam ser O(n²).

- Padrões de entrevista: dois ponteiros, `HashMap` para evitar laço duplo, ordenar e comparar, busca binária, recursão com caso base.
- **Concorrência**: `valor++` não é atômico; use `synchronized`, `Atomic*`, `ConcurrentHashMap`; beans do Spring não devem guardar estado mutável.
- Lombok reduz código repetitivo, mas exige cuidado em entidades JPA.
- Saiba o básico de terminal: navegar, ler logs, achar processos e portas.

Exercícios teóricos

1. Explique cada letra do SOLID com um exemplo da Biblioteca.
2. Que sinais indicam violação do princípio de responsabilidade única em um Service?
3. Dê um exemplo de violação de Liskov.
4. Por que "depender de abstrações" facilita os testes?
5. Qual a complexidade de achar um valor em uma `ArrayList` e em um `HashSet`? Por quê?
6. Por que a solução do *two sum* com `HashMap` é O(n) e a ingênua é $O(n^2)$?
7. Por que a busca binária exige uma lista ordenada?
8. O que é uma condição de corrida? Por que `valor++` não é seguro?
9. Qual a diferença entre `synchronized` e `AtomicInteger`?
10. Por que um `@Service` do Spring não deve guardar estado mutável por usuário?
11. Qual o risco de usar `@Data` do Lombok em uma entidade JPA?

Exercícios práticos

1. Refatore um `if/else` de cálculo de multa para o padrão `RegraDeMulta` (item O do SOLID) e escreva testes para cada regra.
2. Implemente e **teste com JUnit** os problemas da seção 15.2, incluindo casos de borda (string vazia, `null`, um elemento, repetidos).
3. Resolva sem ver a solução: remover duplicados de uma lista mantendo a ordem; maior e segundo maior número; verificar parênteses balanceados com `ArrayDeque`.
4. Rode o `ContadorInseguro` com 4 threads incrementando 100.000 vezes cada, **repetindo a execução várias vezes**. Às vezes o resultado será menor que 400.000, às vezes exato: a condição de corrida é **não determinística**, e é isso que a torna perigosa (o bug some nos seus testes e aparece em produção). Corrija com `synchronized` e com `AtomicInteger` e confirme que o resultado é sempre 400.000.
5. Use `ExecutorService` para somar partes de uma lista em paralelo e junte os resultados.
6. Resolva 10 problemas fáceis no HackerRank ou LeetCode, em Java, falando o raciocínio em voz alta.
7. No terminal: acompanhe o log da API com `tail -f`, descubra com `lsof -i :8080` quem usa a porta e finalize esse processo.

Projeto principal: Sistema Biblioteca (refatoração opcional)

1. Extraia uma interface `Notificador` e injete-a em `EmprestimoService`; implemente `LogNotificador` (escreve no log) e teste o Service com um mock.
2. Substitua a lógica de multa por uma interface `RegraDeMulta` com implementações por perfil de usuário (`LEITOR`, `PROFESSOR`).

3. Faça uma revisão SOLID: liste no README duas classes que violavam algum princípio e como você corrigiu.
4. Adicione um `@Scheduled` que marca, toda noite, os empréstimos vencidos como `ATRASADO` (habilite com `@EnableScheduling`).

Mini-projeto: Kit de Algoritmos com Testes

Em outro repositório Maven:

- Uma classe por tema (`Strings`, `Listas`, `Maps`, `Recursao`, `Busca`) com **pelo menos 20 problemas resolvidos**, todos com testes JUnit, incluindo casos de borda.
- Um `README.md` com a complexidade (Big-O) de cada solução e uma frase sobre o raciocínio.
- Um pacote `concorrencia` com os três contadores (inseguro, sincronizado, atômico) e um teste que demonstra a diferença entre eles.
- Desafio: implemente uma pilha e uma fila **à mão** (sem usar `Stack`/`Queue`), com testes, e explique quando usar cada uma.